

**Diseño e Implementación de un Agente en Espacio de
Usuario**
**para la Gestión Dinámica de Frecuencia (DVFS) en
Sistemas**
**Heterogéneos mediante Modelos Ligeros de Machine
Learning**

Yeison Adrián Cáceres Torres

Ricardo Andrés Pérez Porras

Trabajo de Grado para Optar el Título de Ingeniero de Sistemas

Director

Gilberto Javier Díaz Toro

Doctor en Ingeniería

Universidad Industrial de Santander

Facultad de Ingenierías Físico-Mecánicas

Escuela de Ingeniería de Sistemas e Informática

Bucaramanga

2026

Agradecimientos

Resumen

Título: Diseño e Implementación de un Agente en Espacio de Usuario para la Gestión Dinámica de Frecuencia (DVFS) en Sistemas Heterogéneos mediante Modelos Ligeros de Machine Learning.

Autores: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Palabras Clave: DVFS; eficiencia energética; computación de alto rendimiento; modelo Roofline; telemetría microarquitectónica; aprendizaje automático supervisado.

Descripción:

El consumo energético constituye hoy una restricción estructural para el escalamiento de la computación de alto rendimiento (HPC), particularmente en nodos heterogéneos que integran CPU y aceleradores gráficos. El Escalado Dinámico de Voltaje y Frecuencia (DVFS) es el principal mecanismo de control disponible a nivel de hardware, pero su aprovechamiento efectivo depende de decidir, durante la ejecución, cuál punto operativo conviene según el comportamiento de la carga. Los gobernadores nativos del sistema operativo deciden a partir de la utilización agregada del procesador, una señal que no distingue si el rendimiento está limitado por la capacidad aritmética del núcleo o por la transferencia de datos desde memoria; en el segundo caso, sostener la frecuencia máxima incrementa el consumo sin una mejora proporcional del tiempo de ejecución.

Este trabajo propone el diseño, la implementación y la evaluación de un agente en espacio de usuario que infiere el régimen de ejecución de la aplicación a partir de telemetría microarquitectónica y traduce esa inferencia en decisiones discretas de DVFS sobre CPU y GPU, con el objetivo de optimizar el Producto Energía–Retardo (EDP) sin incurrir en una penalización severa del rendimiento ni en una sobrecarga de inferencia que anule el beneficio

energético.

El desarrollo se organiza en cuatro fases: caracterización de cargas y construcción del conjunto de datos; entrenamiento y selección de un clasificador supervisado ligero; implementación del servicio de control; y validación experimental frente a los gobernadores nativos de Linux. El presente documento reporta el instrumento de medición y el protocolo experimental contruidos durante la primera fase — cuyo aporte metodológico central es un procedimiento reproducible para etiquetar como *compute-bound* o *memory-bound* cada ventana de ejecución de CPU o corrida completa de GPU, mediante una calibración empírica del modelo Roofline sobre la plataforma de medición, con trazabilidad explícita de la calidad de cada observación — junto con un primer entrenamiento y análisis del clasificador de GPU, posible ya dentro de esta fase por disponer de su conjunto de entrenamiento definido.

Abstract

Title: Design and Implementation of a User-Space Agent for Dynamic Frequency Management (DVFS) in Heterogeneous Systems through Lightweight Machine Learning Models.

Authors: Yeison Adrián Cáceres Torres; Ricardo Andrés Pérez Porras.

Keywords: DVFS; energy efficiency; high performance computing; Roofline model; micro-architectural telemetry; supervised machine learning.

Description:

Energy consumption has become a structural constraint on the scaling of high performance computing (HPC), particularly on heterogeneous nodes that combine CPUs and graphics accelerators. Dynamic Voltage and Frequency Scaling (DVFS) is the main control mechanism available at the hardware level, but exploiting it effectively requires deciding, at run time, which operating point suits the current workload behaviour. Native operating system governors base that decision on aggregate processor utilisation, a signal that does not distinguish whether performance is limited by the arithmetic capability of the core or by data transfer from memory; in the latter case, sustaining maximum frequency raises power draw without a proportional improvement in execution time.

This work proposes the design, implementation and evaluation of a user-space agent that infers the execution regime of an application from microarchitectural telemetry and translates that inference into discrete DVFS decisions over CPU and GPU, aiming to optimise the Energy-Delay Product (EDP) without incurring a severe performance penalty or an inference overhead that cancels the energy benefit.

The work is organised in four phases: workload characterisation and dataset construction; training and selection of a lightweight supervised classifier; implementation of the control

service; and experimental validation against the native Linux governors. This document reports the measurement instrument and experimental protocol built during the first phase, whose central methodological contribution is a reproducible procedure for labelling execution windows as *compute-bound* or *memory-bound* through an empirical calibration of the Roofline model on the measurement platform, with explicit traceability of the quality of each observation.

Índice general

Agradecimientos	1
Resumen	2
Abstract	4
Introducción	16
Planteamiento y justificación del problema	17
Objetivos	18
1. Marco de referencia	20
1.1. Marco Conceptual	20
1.1.1. Potencia CMOS y fundamento de DVFS	20
1.1.2. Modelo Roofline y regímenes de ejecución	21
1.1.3. Telemetría microarquitectónica con PMU	23
1.1.4. Espacios de ejecución <i>user-space</i> y <i>kernel-space</i>	25
1.1.5. Interfaces de observabilidad	25
1.1.6. DVFS, P-states y mecanismos de actuación	26
1.1.7. Cargas de calibración y modelos de clasificación ligera	27
1.1.8. Hiperparámetros y Optuna	28
1.1.9. Producto Energía–Retardo (EDP)	28
1.2. Estado del Arte	29
2. Metodología	32
2.1. Enfoque y diseño	32
2.1.1. Población, muestra y criterios de inclusión	33
2.1.2. Plataforma experimental	33
2.1.3. Instrumento de medición	33

2.1.4.	Modelo Roofline y etiquetado	35
2.2.	Fase 1: recolección y caracterización	36
2.2.1.	Verificación previa y catálogo	36
2.2.2.	Matriz experimental y control de sesgos	36
2.2.3.	Calidad de los datos y granularidad	37
2.2.4.	Reproducibilidad	37
2.3.	Fase 2: modelos de clasificación y política de frecuencia	38
2.3.1.	Formulación, variables y validación	38
2.3.2.	Decisión selectiva y calibración	39
2.3.3.	Comparación y selección de modelos	39
2.3.4.	Invarianza de las entradas frente a la acción del agente	40
2.3.5.	Derivación de la política de frecuencia	40
2.4.	Fase 3: agente de control DVFS	41
2.4.1.	Arquitectura	41
2.4.2.	Detección de fase, decisión y parámetros	42
2.4.3.	Política, actuación y coordinación	43
2.4.4.	Aplicaciones compuestas y verificación previa	44
2.5.	Fase 4: validación experimental	44
2.5.1.	Diseño de la matriz	44
2.5.2.	Medición y análisis	45
2.5.3.	Escenarios adicionales	46
3.	Resultados	47
3.1.	Resultados de CPU	47
3.1.1.	Composición de fase por kernel	48
3.1.2.	Variación de la composición con la frecuencia	49
3.1.3.	Calidad del conjunto de datos	50
3.1.4.	Clasificador de fase en CPU	52
3.1.5.	Política de frecuencia por clase en CPU	67
3.1.6.	Invarianza del clasificador de CPU frente a la acción del agente . . .	70
3.2.	Resultados de GPU	70
3.2.1.	Validación de control y etiquetado	70
3.2.2.	Diagnóstico del primer conjunto GPU	70
3.2.3.	Reanálisis de campañas históricas con NVML	71
3.2.4.	Candidato operativo de GPU: invarianza frente a la acción del agente	77
3.2.5.	Política de frecuencia por clase en GPU	79

3.2.6.	Continuidad de la ruta CUPTI	82
3.3.	Resultados del agente de control	83
3.3.1.	Candado de reloj de GPU y premisa física de la política	83
3.3.2.	Latencias de conmutación y sobrecarga	84
3.3.3.	Restauración bajo terminación anómala y coordinación	85
3.3.4.	Detección y clasificación de fase de GPU sobre aplicaciones compuestas	85
3.4.	Evaluación del agente frente a REF (Fase 4)	85
3.4.1.	Aplicaciones compuestas	85
3.4.2.	Clasificación de fase con el agente en el lazo	86
3.4.3.	Tiempo, energía y EDP frente a REF	86
3.4.4.	Costo del agente de CPU y configuración de la inferencia	88
3.4.5.	Potencial de la GPU	89
3.4.6.	Escenario C: agente de CPU con un hilo de inferencia	89
3.4.7.	Referencia sin turbo	90
3.4.8.	Escenario E: GPU dominada por fases de memoria	90
3.4.9.	Escenario D: LAMMPS como aplicación de terceros	93
3.4.10.	CloverLeaf CUDA Fortran como aplicación continua	94
3.4.11.	Balance de la Fase 4	96
4.	Discusión	97
4.1.	Validez de la medición	97
4.2.	Generalización de los clasificadores	97
4.3.	Alcance de la ganancia energética	98
5.	Conclusiones	99
5.1.	Limitaciones	100
5.2.	Trabajo futuro	100
A.	Detalle técnico de la metodología	102
A.1.	Diseño de la medición	102
A.1.1.	Dominio de frecuencia y topología SMT	102
A.1.2.	Instrumento y señales	103
A.1.3.	Granularidad, calidad y reproducibilidad	104
A.2.	Fase 1. Recolección y caracterización	105
A.2.1.	Verificación previa	105
A.2.2.	Catálogo y cribado	106
A.2.3.	Calibración y matriz	107

A.3.	Fase 2. Desarrollo y validación de los modelos	107
A.3.1.	Variables y prevención de fuga	107
A.3.2.	Auditoría, muestreo y validación	108
A.3.3.	Derivación de la política	108
A.4.	Detalles adicionales del diseño	109
A.5.	Parámetros y controles del agente	110
A.6.	Análisis y decisiones complementarias	110
A.6.1.	Derivación detallada de la política	110
A.7.	Protocolos completos de la Fase 4	111
A.8.	Detalles adicionales de método y diagnóstico	111
A.8.1.	Selección detallada del catálogo	111
B.	Resultados complementarios	113
B.1.	Detalle del clasificador de CPU	113
B.2.	Detalle del clasificador de GPU	117
B.3.	Detalle de la evaluación del agente	117
B.4.	Tablas complementarias de CPU	122
B.5.	Verificaciones complementarias del agente	124
B.6.	Análisis y decisiones complementarias	125
B.6.1.	Sensibilidad del tamaño de la muestra CPU	125
B.6.2.	Variables adicionales del modelo CPU	125
B.6.3.	Desglose de la política CPU	126
B.6.4.	Modelo del piso de potencia CPU	126
B.6.5.	Contrato operativo del modelo GPU	127
B.6.6.	Sensibilidad del tiempo al reloj GPU	127
B.6.7.	Excepción en la política GPU	128
B.6.8.	Referencia sin turbo	128
B.6.9.	Análisis por repetición de E-B	129
B.7.	Tablas completas de síntesis y política	130
B.8.	Detalles adicionales de método y diagnóstico	131
B.8.1.	Auditoría de frecuencia de la campaña CPU	131
B.8.2.	Selección detallada del umbral CPU	132
B.8.3.	Construcción detallada de la política CPU	132
B.8.4.	Diagnóstico del eje de frecuencia CUPTI	133
B.8.5.	Composición detallada del escenario E	133
	Repositorio y recursos complementarios	139

Índice de tablas

2.1. Plataforma experimental.	33
2.2. Brazos del agente.	42
2.3. Parámetros del agente.	42
2.4. Acciones del agente. Las filas de GPU provienen de la política derivada en la Fase 2; las filas de CPU son variantes experimentales evaluadas en la Fase 4.	43
2.5. Aplicaciones compuestas con verdad de fase.	44
2.6. Matriz experimental de la Fase 4.	45
2.7. Mapa de los escenarios de la Fase 4. El protocolo completo se conserva en el Anexo A.	46
3.1. Variables de entrada del candidato CPU congelado.	54
3.2. Comparación de modelos con hiperparámetros fijos, los seis a su vector final de seis variables. La sensibilidad de cada modelo al tamaño del vector de entrada se reporta por separado en la Tabla B.1. La latencia corresponde a una fila y un hilo, medida en el nodo de destino (paccaA100) con el proceso fijado a cuatro núcleos.	56
3.3. Síntesis de la política CPU. Las ganancias son de EDP frente a REF; la rejilla completa se conserva en la Tabla B.29 del Anexo B.	67
3.4. Modelos GPU después de aplicar el contrato de entradas del daemon. Las métricas se obtienen con LOFO sobre 483 corridas y 16 familias.	78
3.5. Síntesis de la política GPU. Las ganancias son de EDP frente a REF; la rejilla completa se conserva en la Tabla B.30 del Anexo B.	79
3.6. Latencias de conmutación y sobrecarga del agente en paccaA100.	84
3.7. Composición del escenario E. Duraciones aproximadas por fase medidas antes de la matriz.	91
3.8. Resultados del confirmatorio de CloverLeaf CUDA Fortran. Duración y energías son medianas de cinco bloques; las razones se calculan frente a REF. . .	96

3.9. Balance de la Fase 4. El desglose por pregunta se conserva en la Tabla B.31 del Anexo B.	96
A.1. Verificaciones previas agrupadas por familia.	106
A.2. Catálogo resumido de cargas.	106
A.3. Rejilla fina de frecuencia de la campaña definitiva.	107
A.4. Señales de telemetría del instrumento.	109
A.5. Unidad de observación del conjunto de entrenamiento.	109
A.6. Amenazas a la validez y medidas de control.	110
A.7. Protocolos complementarios ejecutados en la Fase 4.	112
B.1. Variantes del vector de entrada bajo el mismo modelo y el mismo protocolo. La diferencia se calcula frente al vector de 6 variables.	113
B.2. Efecto del esquema de partición sobre el resultado, con el mismo modelo y los mismos datos. El $\pm$ es la desviación estándar entre las cinco semillas de muestreo.	114
B.3. Métricas de evaluación empleadas y pregunta que responde cada una.	114
B.4. Valores de todas las métricas para el modelo final, sin abstención. Media de cinco semillas de muestreo, desviación entre semillas e intervalo de confianza del 95 % por remuestreo de las 30 familias.	115
B.5. Resultado del modelo final por nivel de frecuencia. La cobertura es la fracción de intervalos con confianza $\geq 0,85$; la exactitud es global y depende de la composición de clases de cada nivel, a diferencia de la exactitud balanceada por celda.	115
B.6. Rejilla del umbral de confianza. Exactitud balanceada por celda sobre lo decidido y cobertura media por celda, promedio de cinco semillas. La regla elige el máximo con cobertura de al menos 0.60; se congeló 0.85.	116
B.7. Efecto de la decisión selectiva con umbral de confianza 0.85.	116
B.8. Proporción de predicciones del clasificador de CPU que cambia de clase al forzar la frecuencia.	116
B.9. Comparación de modelos candidatos sobre el conjunto GPU ampliado (483 corridas, 16 familias), exactitud balanceada por celda familia por clase.	117
B.10. Factor de inflación de varianza (VIF) de las cinco variables de la referencia estática GPU.	117
B.11. Decisiones del agente de GPU sobre las aplicaciones compuestas (umbral de abstención 0.90).	117

B.12.Fases de las aplicaciones compuestas. Duración media por fase medida en las celdas del brazo REF (6 a 10 fases por kernel); cada aplicación ejecuta dos ciclos.	118
B.13.Clasificación de fase del agente en la matriz inicial (muestreo de 1 ms en CPU). Correctas y erróneas cuentan decisiones; la exactitud es sobre las decididas y la cobertura es el número de fases con al menos una decisión.	118
B.14.Resultados de la matriz inicial. Duración y energías son medianas; las razones son frente a REF. $E_{\text{tot}} = E_{\text{CPU}} + E_{\text{GPU}}$. El valor p es de Mann–Whitney bilateral y solo se calcula con al menos cinco repeticiones por brazo.	119
B.15.Efecto atribuible a la acción de frecuencia: EDP frente a REF del brazo sombra (sin escribir) y del activo, y su cociente. Un cociente menor que 1 indica que actuar mejora el EDP respecto de solo observar.	119
B.16.Costo del brazo sombra frente a REF en CPU, aplicación A (tres repeticiones por configuración), según la configuración del motor de inferencia y del proceso consumidor.	120
B.17.Escenario C (tres repeticiones por brazo). Duración y energías son medianas y las razones son frente a REF.	120
B.18.REF sin turbo relativo a REF, por aplicación compuesta y alcance incluido en C o E. Las razones son medianas de tres repeticiones.	120
B.19.Escenario E (tres repeticiones por brazo). Duración y energías son medianas; las razones son frente a REF y la última columna frente a REF sin turbo. . .	121
B.20.Clasificación de fase del agente en los escenarios C y E, contra las fronteras reales de cada fase.	121
B.21.Resultados del escenario D con LAMMPS. Las razones se calculan frente a REF para cada entrada.	122
B.22.Cobertura y composición del conjunto de intervalos CPU.	122
B.23.Intervalos procesados, rechazados por causa y elegibles en la campaña CPU. .	123
B.24.Flujo de observaciones desde la campaña CPU hasta la evaluación del modelo. .	123
B.25.Sensibilidad del resultado al tope de intervalos por celda de familia y clase. .	123
B.26.Configuración final del clasificador CPU.	124
B.27.Métricas por clase del modelo final sobre los 1 309 755 intervalos evaluados. .	124
B.28.Restauración bajo terminación anómala y coordinación entre procesos. . . .	124
B.29.Ganancia del producto energía–retardo por nivel y clase	130
B.30.Ganancia del producto energía–retardo GPU por nivel y clase, con la familia como unidad	130
B.31.Balance de la Fase 4 por pregunta.	131

Índice de figuras

1.1. Modelo Roofline conceptual	21
1.2. Unidad de Monitorización de Rendimiento (PMU)	23
1.3. Contadores de núcleo y contadores de <i>uncore</i>	24
1.4. Espacios de usuario y de kernel	25
1.5. P-states, C-states y jerarquía DVFS	26
2.1. Fases de la metodología	32
2.2. Recorrido de una campaña a través de las dos capas del instrumento	34
2.3. Obtención de la etiqueta en CPU y en GPU	36
2.4. Validación anidada por familia	39
2.5. Arquitectura del agente	41
3.1. Composición de fase por kernel en CPU	48
3.2. Composición de fase por nivel de frecuencia en CPU	49
3.3. <i>Ridge</i> Roofline según el nivel de frecuencia CPU. El punto de inflexión descendiente de 8.69 FLOP/byte en referencia nativa a 2.92 FLOP/byte en F8. Fuente: elaboración propia.	50
3.4. Intervalos del conjunto de entrenamiento en el plano Roofline	51
3.5. Muestreo estratificado del conjunto CPU	53
3.6. Correlación entre las variables candidatas del clasificador CPU	55
3.7. Comparación de modelos del clasificador CPU	56
3.8. Acierto del modelo final en cada celda de familia y clase	58
3.9. Matrices de confusión del clasificador CPU	60
3.10. Resultado del clasificador CPU por familia	61
3.11. Exactitud y cobertura del clasificador CPU por nivel de frecuencia	62
3.12. Calibración y compromiso de la abstención en CPU	63
3.13. Perfiles de celdas confusables entre clases opuestas	64
3.14. Exactitud según la distancia al <i>ridge</i>	65

3.15. Curva de aprendizaje por número de familias	65
3.16. Ganancia del EDP por nivel y clase, en forma de <i>forest plot</i>	67
3.17. Energía, tiempo y producto energía-retardo por nivel de frecuencia	68
3.18. Potencia de paquete y DRAM frente a la frecuencia	69
3.19. Balance de clases del conjunto GPU inicial	71
3.20. Separabilidad de las clases GPU con señales NVML	72
3.21. Comparación de modelos candidatos GPU	73
3.22. Exactitud de la referencia estática GPU por familia	75
3.23. Calibración y abstención de la regresión logística GPU	76
3.24. Matriz de confusión y validación externa de la referencia estática GPU	77
3.25. Invarianza de los candidatos GPU frente al reloj	78
3.26. Ganancia del EDP GPU por nivel y clase, en forma de <i>forest plot</i>	80
3.27. Tiempo, energía y EDP relativos por nivel de frecuencia GPU	81
3.28. Potencia de GPU frente al reloj SM y piso estático	82
3.29. Rendimiento de GPU con el reloj fijado	84
3.30. Estructura de un ciclo de cada aplicación compuesta. Los números dentro de las barras son la duración media de cada fase en segundos; a la derecha, la fracción del ciclo en fases <i>memory_bound</i>	86
3.31. EDP del nodo, $(E_{\text{CPU}} + E_{\text{GPU}})T$, relativo a REF, por alcance, aplicación (A: vistos, B: inéditos) y brazo, en la matriz inicial. La línea punteada marca la paridad con REF.	87
3.32. Costo del brazo sombra frente a REF en CPU con kernels vistos (A), por configuración de la inferencia y periodo de muestreo.	88
3.33. Izquierda: ganancia de EDP por nivel de reloj de GPU en fases <i>memory_bound</i> (Fase 2, IC95 sobre familias). Derecha: ganancia esperada (fracción de tiempo en memoria por la ganancia de F1) frente a la medida con el agente activo en el alcance GPU.	89
3.34. EDP del nodo relativo a REF en el escenario C, por alcance, aplicación y brazo. La línea punteada marca la paridad con REF.	90
3.35. Escenario E. EDP del nodo (arriba) y energía de GPU (abajo) relativos a REF, con la mediana (barra) y cada repetición (punto), para las aplicaciones de familias vistas (A) e inéditas (B).	93
3.36. EDP mediano de LAMMPS por entrada y brazo, relativo a REF. Fuente: elaboración propia.	94
3.37. EDP de CloverLeaf por bloque y brazo	95
3.38. Descomposición de energía de CloverLeaf	95

A.1. Solicitudes de frecuencia entre hermanos SMT	103
A.2. Ruta de muestreo con estructura circular de un productor y un consumidor .	104

Introducción

En los sistemas de cómputo de alto rendimiento (HPC), el consumo energético se ha consolidado como una restricción crítica para el escalado sostenible, especialmente en arquitecturas heterogéneas CPU–GPU. En este contexto, el Escalado Dinámico de Voltaje y Frecuencia (DVFS) constituye el principal mecanismo de control a nivel de hardware, permitiendo ajustar los estados operativos de los procesadores para optimizar el compromiso entre consumo energético y tiempo de ejecución. Sin embargo, la efectividad de este mecanismo depende de la capacidad del sistema para adaptar dinámicamente la frecuencia al comportamiento cambiante de las aplicaciones, lo cual representa un desafío abierto en entornos HPC modernos.

En la literatura, este problema ha sido abordado principalmente mediante dos enfoques. Por un lado, los gobernadores de energía del sistema operativo, como *ondemand* o *schedutil*, emplean políticas reactivas basadas principalmente en la utilización del procesador, sin considerar la naturaleza microarquitectónica de la carga de trabajo, lo que limita su capacidad de adaptación a cargas de trabajo científicas dinámicas. Por otro lado, diversas propuestas han explorado estrategias heurísticas basadas en métricas microarquitectónicas, utilizando reglas deterministas para ajustar la frecuencia de operación. Aunque estos métodos presentan bajo *overhead*, su dependencia de umbrales estáticos y su limitada capacidad de generalización reducen su efectividad en escenarios donde las fases computacionales varían rápidamente.

A pesar de estos avances, persiste una limitación fundamental: la incapacidad de los enfoques actuales para capturar de forma robusta las transiciones dinámicas entre regímenes *compute-bound* y *memory-bound*, así como para adaptarse a la heterogeneidad CPU–GPU sin intervención manual o ajuste específico por carga de trabajo. Esta brecha evidencia la necesidad de mecanismos de control más flexibles que permitan inferir el régimen computacional de la aplicación en tiempo de ejecución y tomar decisiones de escalado de frecuencia fundamentadas en el perfil de ejecución.

En este contexto, el presente trabajo propone el diseño e implementación de un agente en

espacio de usuario basado en modelos ligeros de aprendizaje automático, capaz de clasificar en tiempo de ejecución las fases de una aplicación como compute-bound o memory-bound a partir de telemetría microarquitectónica, y de ajustar dinámicamente la frecuencia de CPU y GPU para optimizar el Producto Energía–Retardo (EDP). Se plantea demostrar que este enfoque mejora la eficiencia energética frente a los gobernadores nativos de Linux, manteniendo una degradación controlada del rendimiento y una sobrecarga de inferencia mínima y cuantificada mediante métricas experimentales.

Planteamiento y justificación del problema

Existe una ineficiencia energética estructural en la operación de los sistemas de alto rendimiento (HPC), especialmente en infraestructuras que integran aceleradores gráficos (GPU) para ejecutar aplicaciones científicas intensivas. Cuando una aplicación entra en una fase donde el procesamiento depende principalmente de la transferencia de datos desde memoria, mantener la CPU o la GPU a su frecuencia máxima no mejora el rendimiento de manera proporcional. En estas condiciones, los núcleos de cómputo permanecen parcialmente inactivos esperando datos, lo que genera un consumo energético elevado sin un beneficio equivalente en tiempo de ejecución. Esta desalineación entre demanda computacional y frecuencia operativa ha motivado el estudio del escalado de frecuencia como mecanismo para mejorar la eficiencia energética en aplicaciones científicas [31] y en sistemas de gran escala donde la gestión de potencia impacta el comportamiento global del sistema [9].

El problema se agrava por la naturaleza multifásica de las aplicaciones HPC, cuyo perfil de ejecución cambia a lo largo del tiempo de ejecución. En particular, estas aplicaciones alternan entre fases donde el rendimiento está limitado por la capacidad de cómputo del procesador (*compute-bound*) y fases donde está restringido por la transferencia de datos desde memoria (*memory-bound*). Esta distinción es crítica porque, en el régimen *memory-bound*, incrementos de frecuencia tienden a producir mejoras marginales en el tiempo de ejecución, mientras aumentan el consumo energético; en contraste, en fases *compute-bound* la frecuencia sí impacta el desempeño de forma más directa. Por ello, se han propuesto marcos de caracterización para identificar y clasificar estas fases en entornos HPC, con el fin de habilitar decisiones de control más informadas [4, 7].

A partir de este contexto, la gestión eficiente de frecuencia no depende únicamente de la disponibilidad del mecanismo DVFS en el hardware, sino de la capacidad del sistema para decidir, en tiempo de ejecución, cuál configuración resulta más conveniente según el comportamiento de la carga. Aunque se han propuesto mecanismos automatizados para seleccionar

frecuencias óptimas de GPU con el fin de mejorar la eficiencia energética sin comprometer significativamente el desempeño [2], persiste una brecha en la implementación de estrategias que integren de forma coordinada componentes CPU–GPU y que operen durante la ejecución de la aplicación, considerando además el costo computacional asociado al propio proceso de decisión.

Esta brecha resulta especialmente relevante en entornos académicos y científicos, donde el consumo eléctrico incide directamente en los costos operativos, la disponibilidad efectiva de recursos y la sostenibilidad institucional. En este contexto, contar con un mecanismo capaz de observar el comportamiento de la aplicación, inferir su fase de ejecución y ajustar la frecuencia de operación sin intervenir el código fuente constituye una alternativa con valor técnico y práctico.

Desde la perspectiva internacional, la literatura reciente ha mostrado avances en la optimización energética mediante escalado de frecuencia, gestión de potencia y caracterización de cargas HPC [31, 9, 4, 7, 2]. Sin embargo, a nivel nacional y regional, este tipo de soluciones aún no se encuentra ampliamente implementado en infraestructuras académicas de cómputo científico, particularmente bajo esquemas que combinen telemetría de bajo nivel, modelos ligeros de aprendizaje automático y control en espacio de usuario. En consecuencia, el problema no solo es técnicamente relevante, sino también pertinente en términos de aplicabilidad local.

A partir de esta necesidad de optimización energética en sistemas heterogéneos, se formula la siguiente pregunta de investigación que servirá como eje de desarrollo del proyecto:

¿En qué medida el diseño e implementación de un agente en espacio de usuario, guiado por modelos ligeros de aprendizaje automático, permite ajustar la frecuencia de operación (DVFS) en sistemas heterogéneos CPU–GPU para reducir el consumo energético, sin que el costo de la inferencia computacional degrade el rendimiento global de la aplicación, en comparación con los gobernadores nativos de Linux?

Objetivos

Objetivo General

Caracterizar, diseñar, implementar y evaluar un agente en espacio de usuario basado en modelos ligeros de Aprendizaje Automático para el ajuste dinámico de frecuencia (DVFS)

en sistemas heterogéneos CPU–GPU, con el propósito de maximizar la eficiencia energética en aplicaciones científicas sin inducir una penalización severa en su rendimiento global.

Objetivos Específicos

1. Caracterizar el comportamiento computacional y el consumo energético de cargas de trabajo representativas (intensivas en cómputo y en memoria) bajo distintos estados de frecuencia, recolectando telemetría de bajo nivel mediante contadores de rendimiento por hardware e interfaces de potencia estándar (Perf y RAPL para CPU y NVML para GPU).
2. Entrenar y validar modelos clásicos de Aprendizaje Automático (tales como Árboles de Decisión o Bosques Aleatorios) que sean capaces de clasificar, en tiempo de ejecución y con baja latencia de inferencia, las fases de ejecución de las aplicaciones basándose en la telemetría extraída.
3. Desarrollar un servicio de control (*daemon*) en el espacio de usuario que interactúe de forma asíncrona con el sistema operativo, leyendo el estado de los contadores de hardware, ejecutando la inferencia del modelo clasificador y aplicando políticas proactivas de DVFS a través de las interfaces estándar del sistema operativo, en función de la fase de ejecución inferida por el modelo.
4. Evaluar el impacto empírico del sistema propuesto mediante el cálculo del Producto Energía–Retardo (EDP), con el fin de determinar estadísticamente si el ahorro energético compensa la sobrecarga computacional (*overhead*) derivado de la inferencia del modelo en comparación con los gobernadores nativos de Linux.

Capítulo 1

Marco de referencia

1.1. Marco Conceptual

El marco conceptual parte de la relación entre el ajuste de la frecuencia y del voltaje (DVFS), la potencia requerida por el hardware y el tiempo de ejecución, cuyo comportamiento depende del régimen que limita cada carga. Régimen que puede estar condicionado por capacidad de cómputo (compute-bound) o de memoria (memory-bound), dos límites que se pueden distinguir mediante el modelo Roofline y algunas señales del sistema asociadas con este comportamiento durante la ejecución. Señales que funcionan como vector de entrada a un clasificador ligero que intenta inferir el régimen, permitiendo así tomar decisiones de ajuste de frecuencia, teniendo en cuenta el Producto Energía–Retardo (EDP), el cual permite relacionar el consumo energético con el tiempo de ejecución para valorar el efecto de dichas decisiones.

1.1.1. Potencia CMOS y fundamento de DVFS

La potencia dinámica de un circuito CMOS depende del factor de actividad, la capacitancia efectiva, el voltaje de alimentación y la frecuencia de operación. Esta relación es la base física de DVFS y se expresa como:

$$P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f \quad (1.1)$$

donde α representa el factor de actividad, C la capacitancia efectiva conmutada, V el voltaje de alimentación y f la frecuencia de operación [12]. El voltaje y la frecuencia están relacionados dentro de los estados de rendimiento, ya que para trabajar a frecuencias más

altas normalmente se necesita un mayor voltaje. Por esto, al reducir la frecuencia también puede disminuir el voltaje, lo que afecta directamente la potencia dinámica debido a la relación cuadrática de V en la ecuación.

Sin embargo, esta reducción de potencia por sí sola no asegura un menor consumo de energía, porque también interviene el tiempo de ejecución. Si la aplicación tarda más en finalizar al trabajar a una frecuencia menor, parte del ahorro obtenido puede perderse por ese tiempo adicional. Además, durante toda la ejecución sigue existiendo consumo asociado a la potencia estática. Por eso, el efecto energético de cambiar la frecuencia también depende de cómo responda la carga de trabajo.

1.1.2. Modelo Roofline y regímenes de ejecución

El modelo Roofline relaciona el rendimiento alcanzable de una carga con la capacidad de cómputo y el ancho de banda de memoria de la plataforma. La intensidad operacional determina cuál de estos dos límites condiciona el rendimiento y se incorpora al modelo como:

$$P = \min(P_{\text{pico}}, I \cdot BW_{\text{pico}}) \quad (1.2)$$

donde P_{pico} representa el rendimiento aritmético pico, BW_{pico} el ancho de banda pico de memoria e I la intensidad operacional [37]. Cuando el rendimiento queda limitado por $I \cdot BW_{\text{pico}}$, la carga se encuentra en régimen *memory-bound*. Si el límite corresponde a P_{pico} , la carga se encuentra en régimen *compute-bound*. La Figura 1.1 muestra esta separación entre ambos regímenes.

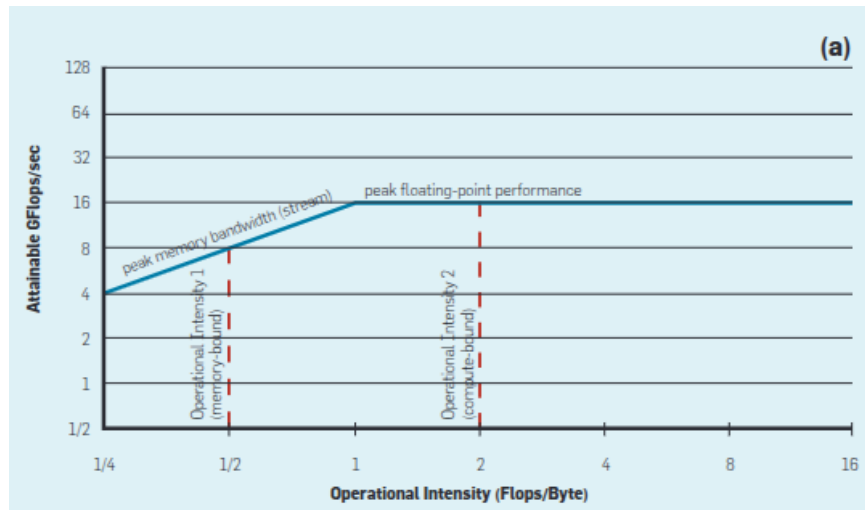


Figura 1.1: Modelo Roofline conceptual con el techo de ancho de banda de memoria y el techo de rendimiento de cómputo. Nota. Tomado de Williams et al. [37, Fig. 1(a)].

La intensidad operacional expresa la cantidad de operaciones realizadas por cada byte transferido desde el nivel de memoria considerado [37, p. 66]. Por ello, no depende únicamente del tamaño del conjunto de datos ni de si este cabe o no en caché. La reutilización de datos puede reducir el tráfico hacia memoria y aumentar la intensidad operacional, incluso cuando el conjunto de trabajo supera la capacidad de la caché [13, Secs. 3–4]. El modelo establece así un límite de rendimiento para cada intensidad, aunque una carga puede permanecer por debajo de ese límite por otros efectos de la ejecución.

La separación entre los dos regímenes ocurre en el punto de inflexión, también denominado *ridge point*. En ese punto, el techo de cómputo y el límite impuesto por el ancho de banda tienen el mismo valor, por lo que el umbral se obtiene como:

$$I_{\text{ridge}} = \frac{P_{\text{pico}}}{BW_{\text{pico}}} \quad (1.3)$$

Una intensidad inferior a I_{ridge} corresponde al régimen *memory-bound*, mientras que una intensidad superior corresponde al régimen *compute-bound* [37]. El *ridge point* funciona como frontera entre ambos regímenes y no como una frecuencia óptima de ejecución. Si alguno de los techos cambia, también puede cambiar la posición de esta frontera.

El techo de cómputo depende de la capacidad aritmética disponible y puede variar con la precisión de las operaciones y con la frecuencia de las unidades de cómputo [3]. El ancho de banda de memoria pertenece a otro dominio de la arquitectura y no tiene que cambiar en la misma proporción cuando se modifica la frecuencia [15, 2]. Por ello, el valor de I_{ridge} utilizado para clasificar una carga debe corresponder a la precisión y al punto de operación bajo el cual se realiza la observación. Los valores nominales sirven como referencia, pero la clasificación requiere techos representativos de las condiciones de ejecución consideradas [28].

La intensidad operacional observada se obtiene a partir del trabajo aritmético y del tráfico hacia memoria medidos durante el mismo intervalo. Para un intervalo i se calcula como:

$$I_i = \frac{\text{FLOPs}_i}{B_i^{\text{DRAM}}} \quad (1.4)$$

donde FLOPs_i representa las operaciones realizadas y B_i^{DRAM} los bytes transferidos hacia o desde memoria principal durante ese intervalo [37]. Los eventos utilizados para obtener los FLOPs dependen de la microarquitectura y deben interpretarse de acuerdo con la semántica de la PMU [36]. El tráfico de DRAM se obtiene desde contadores asociados al controlador

de memoria, cuyo ámbito puede ser diferente al de los contadores de núcleo. Ambas mediciones deben corresponder al mismo intervalo para que la intensidad operacional conserve su significado físico.

1.1.3. Telemetría microarquitectónica con PMU

La clasificación en CPU requiere señales que describan el comportamiento de la ejecución sin reconstruir en línea la intensidad operacional. Las Unidades de Monitorización de Rendimiento (PMU), proporcionan esa observabilidad mediante contadores de hardware que acumulan eventos como instrucciones retiradas, ciclos y actividad de la jerarquía de memoria. Estos contadores pueden ser fijos, asociados a eventos arquitectónicos predefinidos, o programables, configurados para registrar eventos específicos de la arquitectura [35, 18]. La siguiente figura resume esta organización:

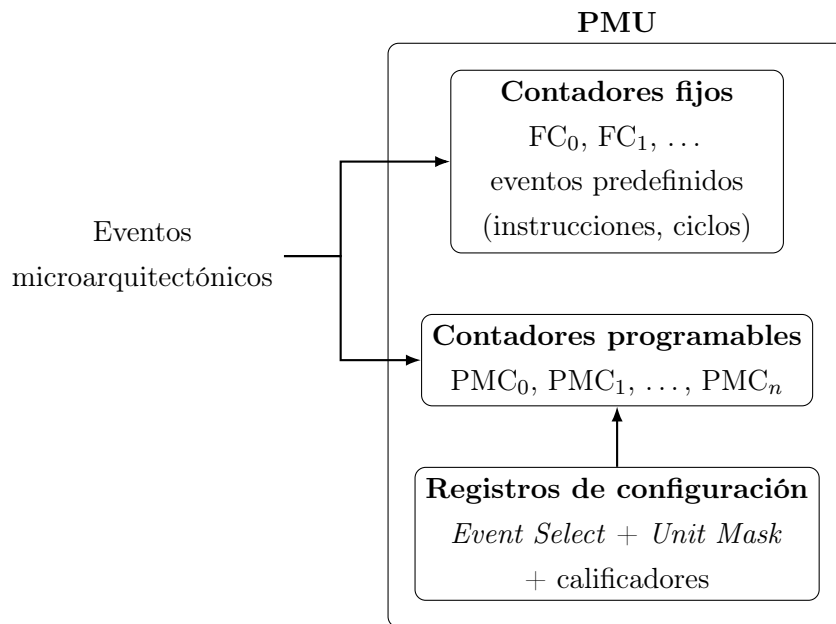


Figura 1.2: Organización conceptual de una Unidad de Monitorización de Rendimiento (PMU) con contadores fijos y programables. Fuente: elaboración propia.

Sin embargo, PMU no interpreta el comportamiento de la aplicación, sino que entrega conteos cuyo valor depende del evento y del dominio observado. Por ello, el mismo valor numérico solo adquiere significado después de considerar qué recurso cuenta, durante qué intervalo y con qué alcance.

Un problema que presentan estos contadores es la multiplexación, que ocurre cuando se solicitan más eventos de los que el hardware puede medir al mismo tiempo. En estas

condiciones, el sistema alterna distintos grupos de eventos y cada contador permanece activo solo durante una parte de la ventana de medición, por lo que el valor registrado se ajusta teniendo en cuenta el tiempo durante el cual estuvo realmente activo [26, 24]. Aunque este mecanismo permite trabajar con más eventos de los disponibles físicamente, la estimación puede perder precisión si la aplicación cambia de fase durante los periodos en los que un evento no está siendo medido, lo que hace conveniente limitar la cantidad de señales observadas de forma simultánea.

Otro detalle que también es importante tener en cuenta es el dominio de los contadores, puesto que existen contadores de núcleo y contadores de *uncore* que difieren en su nivel de acceso y alcance. Los primeros observan recursos asociados a un núcleo y pueden vincularse con la ejecución de una tarea, mientras que los de *uncore* cubren componentes compartidos del zócalo, entre ellos el controlador de memoria integrado y partes de la interconexión, desde donde se puede observar el movimiento efectivo de datos hacia memoria principal. Sin embargo, al tratarse de medidas con alcance compartido, el tráfico registrado por los contadores de *uncore* no puede atribuirse directamente a un proceso individual sin condiciones adecuadas de aislamiento. Esta diferencia entre una medida local de núcleo y una medida compartida también hace que ambas fuentes puedan trabajar con granularidades temporales distintas, por lo que no deben combinarse fila a fila sin establecer antes una correspondencia temporal. La siguiente figura muestra esta diferencia estructural.

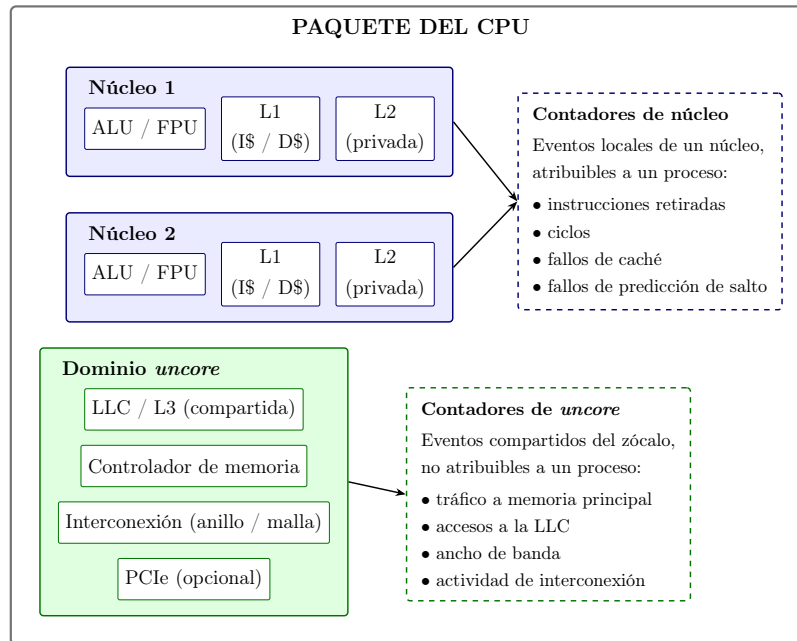


Figura 1.3: Ámbito de atribución de los contadores de núcleo y de *uncore* dentro del paquete del procesador. Fuente: Elaboración propia.

1.1.4. Espacios de ejecución *user-space* y *kernel-space*

Los sistemas operativos modernos emplean distintos niveles de privilegio para regular el acceso a los recursos del sistema, lo que permite distinguir entre los procesos que se ejecutan en *user-space* y aquellos que operan en *kernel-space*. Los procesos en *user-space* trabajan con acceso restringido, mientras que el *kernel-space* concentra las operaciones que requieren acceso directo a recursos protegidos, como la gestión de memoria, la entrada/salida o ciertas funciones del hardware [20, ch. 2]. Debido a esta separación, una aplicación en espacio de usuario no accede directamente a estos recursos y depende de las interfaces que el sistema operativo expone para realizar operaciones controladas [27], y es esta condición la cual también influye en la ubicación de programas de monitoreo y control, ya que determina la forma en que pueden consultar contadores, leer información de energía o solicitar cambios de frecuencia sin modificar estructuras internas del kernel. La siguiente figura detalla más la estructura de estos espacios y la relación conceptual entre ellos y el hardware físico.



Figura 1.4: Relación conceptual entre *user-space*, *kernel-space* y el hardware físico. Fuente: Elaboración propia.

1.1.5. Interfaces de observabilidad

Algunos sistemas operativos proporcionan interfaces que permiten a los programas en espacio de usuario consultar información del hardware sin acceder directamente a sus estructuras internas. En Linux, los eventos de la PMU pueden obtenerse mediante `perf_event_open`, una interfaz que permite configurar los contadores disponibles y consultar sus valores durante la ejecución [24], aunque su disponibilidad depende del soporte del procesador y de los permisos definidos en el sistema.

Estos conteos permiten observar la actividad del procesador, pero no cubren por sí solos la información energética. Para ello puede utilizarse Intel RAPL (*Running Average Power Limit*), un mecanismo incorporado en procesadores Intel que expone contadores de energía para distintos dominios, entre ellos el paquete del procesador y, cuando está disponible, la memoria DRAM [10, 32]. Sus valores representan energía acumulada durante la ejecución, por lo que pueden utilizarse para conocer el consumo asociado a un intervalo y posteriormente relacionarlo con métricas como el EDP.

1.1.6. DVFS, P-states y mecanismos de actuación

El escalado dinámico de voltaje y frecuencia (*Dynamic Voltage and Frequency Scaling*, DVFS) permite modificar el punto operativo del procesador para adaptar su capacidad de cómputo y su consumo energético a las necesidades de la carga. Estos cambios se apoyan en mecanismos de gestión de potencia que reciben una solicitud de rendimiento y la materializan sobre el hardware [16].

En CPU, los *Performance States* o P-states representan distintos niveles de rendimiento mientras el procesador permanece activo en C0. P0 corresponde al mayor nivel de rendimiento y los estados siguientes reducen progresivamente la capacidad disponible. Los C-states, en cambio, representan estados de inactividad con distintos compromisos entre ahorro energético y latencia de reactivación. Sobre los P-states operan los *governors*, políticas software que deciden qué nivel de rendimiento solicitar según el estado del sistema. En Linux, **performance** y **powersave** favorecen extremos distintos del rango disponible, mientras que **ondemand**, **conservative** y **schedutil** ajustan la solicitud de forma dinámica; **userspace**, por su parte, permite que una aplicación solicite explícitamente el nivel deseado [16, 22].

La Figura 1.5 reúne estos dos niveles de funcionamiento. En la parte (a) se representa la relación entre los estados de rendimiento y de inactividad del procesador, mientras que la parte (b) muestra el recorrido de una solicitud desde la interfaz de espacio de usuario, pasando por las políticas y los controladores del sistema, hasta los mecanismos que actúan sobre el hardware. Esta separación permite entender también por qué una decisión de rendimiento no se traduce necesariamente de forma inmediata en la frecuencia observada.

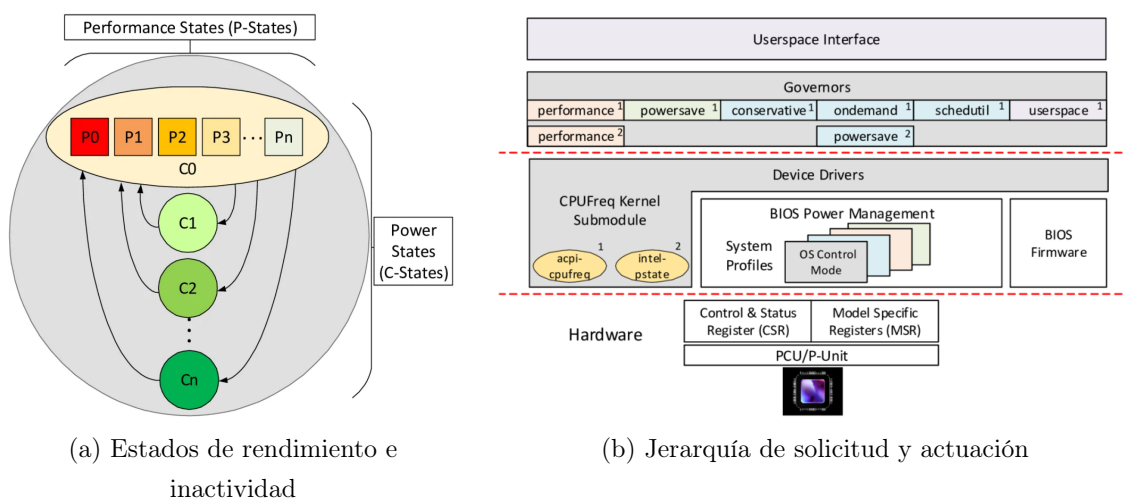


Figura 1.5: Relación entre los estados de rendimiento e inactividad del procesador y la jerarquía mediante la cual una política de rendimiento solicita cambios que posteriormente son materializados por el hardware. Fuente: Tomado de Hebbar y Milenković [16].

En GPU, DVFS mantiene el mismo principio general de ajustar el punto de operación del dispositivo, aunque la actuación suele realizarse sobre dominios de reloj distintos, principalmente el reloj de cómputo y el de memoria. Estos cambios dependen de los mecanismos de gestión propios de la arquitectura y del controlador, que determinan los niveles de frecuencia disponibles y la forma en que una solicitud se aplica sobre el hardware [15, 2]. Al igual que en CPU, la transición entre niveles requiere un tiempo de actuación, por lo que el cambio de frecuencia no es instantáneo [33].

1.1.7. Cargas de calibración y modelos de clasificación ligera

Las aplicaciones de cómputo pueden combinar fases con demandas distintas sobre las unidades de procesamiento y la jerarquía de memoria, por lo que su comportamiento puede observarse mediante cargas o unidades de ejecución más acotadas. Entre ellas se encuentran los *kernels*, que corresponden a operaciones computacionales bien delimitadas, como una multiplicación de matrices, una transformada de Fourier o un stencil, y permiten observar con mayor claridad cómo una operación utiliza los recursos del sistema [37]. También se encuentran los *benchmarks*, que proporcionan cargas utilizadas para medir o comparar el comportamiento de una plataforma bajo condiciones reproducibles, y los *microbenchmarks*, que se construyen para aislar propiedades más específicas, como el ancho de banda de memoria, la latencia de acceso o la capacidad de ejecución aritmética [14].

Las observaciones obtenidas durante estas ejecuciones pueden representarse mediante un conjunto de características y una clase asociada, relación que puede aprenderse mediante clasificación supervisada para después inferir la clase de nuevas observaciones. Cuando este proceso debe realizarse con un costo reducido, conviene trabajar con modelos de clasificación ligera, entendidos como modelos cuya complejidad permite mantener un tiempo de inferencia bajo sobre un número reducido de características.

Existen varios modelos ligeros y no todos representan de la misma forma la relación entre las características. Por ejemplo, la regresión logística establece una frontera lineal y de bajo costo, por lo que sirve como referencia cuando la separación entre clases puede describirse mediante relaciones relativamente simples [19, Ch. 4]. Un árbol de decisión, en cambio, permite introducir relaciones no lineales mediante divisiones sucesivas de las características y conserva una estructura fácil de interpretar, aunque su complejidad aumenta a medida que crece la profundidad del árbol [19, Ch. 8].

A partir de los árboles también pueden construirse modelos de ensamble, como Random Forest, que combina varios árboles generados mediante muestreo y selección aleatoria de

características, reduciendo la dependencia de una única partición y permitiendo representar interacciones no lineales entre las variables [6]. Extra-Trees parte de una estructura similar, pero introduce mayor aleatoriedad en la selección de los puntos de corte [11], por lo que ambos modelos pueden trabajar con relaciones más complejas que las representadas por una frontera lineal.

XGBoost también utiliza árboles de decisión, pero los construye de manera secuencial, de modo que cada nuevo árbol busca corregir parte de los errores acumulados por los anteriores [8]. Esto le permite representar relaciones más complejas, aunque incorpora un mayor número de parámetros y su costo depende de la configuración del ensamble. En conjunto, estas familias cubren distintos niveles de complejidad, desde una frontera lineal hasta conjuntos de árboles capaces de representar interacciones no lineales, manteniendo como consideración común el costo necesario para realizar la inferencia.

1.1.8. Hiperparámetros y Optuna

Los hiperparámetros controlan aspectos de la estructura y del proceso de entrenamiento de un modelo, por lo que su elección puede modificar tanto la capacidad de ajuste como su complejidad. En modelos como Random Forest, Extra-Trees o XGBoost, estos valores pueden incluir cantidades de árboles, profundidades, tasas de aprendizaje u otros parámetros de configuración, lo que genera espacios de búsqueda con combinaciones muy diferentes entre sí. Explorar todas las posibilidades mediante una rejilla puede resultar costoso, de modo que suelen emplearse métodos de optimización que orientan la búsqueda hacia configuraciones más prometedoras.

Optuna es un marco de optimización de hiperparámetros que realiza esta exploración de forma secuencial [1]. Entre sus estrategias se encuentra el estimador de Parzen estructurado (TPE), que utiliza los resultados obtenidos en evaluaciones anteriores para proponer nuevas configuraciones con mayor probabilidad de mejorar el objetivo [5]. De esta manera, la búsqueda no necesita recorrer todas las combinaciones posibles y puede adaptarse a espacios que mezclan variables enteras, continuas y categóricas.

1.1.9. Producto Energía–Retardo (EDP)

Los cambios de frecuencia mediante DVFS afectan tanto el consumo de energía como el tiempo de ejecución, por lo que reducir una de estas magnitudes no significa necesariamente que el resultado sea mejor. Es por esto que el Producto Energía–Retardo (*Energy–Delay Product*, EDP) relaciona ambas en una sola expresión:

$$EDP = E \cdot T$$

donde E representa la energía consumida y T el tiempo de ejecución [21]. Según el valor obtenido, un EDP menor representa una relación más favorable entre energía y tiempo. Sin embargo, ahorrar energía no siempre resulta conveniente si para lograrlo la ejecución tarda mucho más, ya que durante ese tiempo adicional el sistema continúa consumiendo energía, incluida la asociada a la potencia estática.

El EDP reúne la energía y el tiempo en un solo valor, aunque ambas magnitudes todavía deben revisarse por separado, porque dos configuraciones pueden obtener un EDP parecido aun cuando una consuma menos energía y la otra tenga un menor tiempo de ejecución. Por eso, la métrica permite comparar distintos puntos de operación teniendo en cuenta ambos efectos al mismo tiempo.

1.2. Estado del Arte

La gestión energética en sistemas de alto rendimiento ha evolucionado desde enfoques estáticos de reducción de potencia hacia estrategias cada vez más adaptativas, orientadas a preservar el rendimiento sin incurrir en costos energéticos desproporcionados. Dentro de este proceso, el escalado dinámico de voltaje y frecuencia (DVFS) se ha consolidado como uno de los mecanismos más estudiados para modificar el punto operativo de procesadores y aceleradores. Su aplicación, sin embargo, no conduce necesariamente al mismo resultado para todas las cargas de trabajo, ya que una reducción de frecuencia puede disminuir la potencia instantánea, pero también prolongar el tiempo de ejecución y anular el beneficio energético esperado. Esta dependencia ha llevado a que buena parte de la investigación pase de aplicar DVFS de forma uniforme a buscar mecanismos capaces de reconocer el comportamiento de la carga y ajustar la frecuencia de acuerdo con sus características.

Las primeras aproximaciones que analizaron DVFS sobre regiones concretas de una aplicación ya mostraban que mantener una misma frecuencia durante toda la ejecución podía desaprovechar oportunidades de ahorro. Calore et al. [7], al estudiar distintos kernels y una aplicación HPC completa sobre una CPU Intel Haswell y una GPU NVIDIA K80, observaron que el efecto energético variaba entre regiones y arquitecturas. En lugar de mantener un único estado operativo, evaluaron cambios de frecuencia a nivel de función, mostrando que la conveniencia de cada configuración dependía del comportamiento de la región ejecutada y de la plataforma sobre la cual se realizaba el cálculo.

Esta dependencia de la carga llevó a buscar formas de anticipar su respuesta sin recorrer todas las frecuencias disponibles. Guerreiro et al. [15] estudiaron aplicaciones GPGPU a partir de métricas recolectadas en una configuración inicial y utilizaron esa caracterización para estimar cómo responderían ante cambios en los dominios de cómputo y memoria. Con ello redujeron la necesidad de explorar exhaustivamente el espacio DVFS y mostraron que la información de rendimiento podía utilizarse para agrupar aplicaciones según su sensibilidad a la frecuencia y orientar la selección hacia configuraciones energéticamente favorables.

Sobre una idea similar, Ali et al. [2] propusieron un método automatizado y portable para seleccionar frecuencias de GPU a partir de métricas obtenidas inicialmente en la configuración predeterminada del dispositivo. Con esta caracterización construyeron modelos de potencia y tiempo de ejecución para estimar el comportamiento de la aplicación en otros estados DVFS sin ejecutarla en cada uno de ellos. La selección consideró de forma conjunta el consumo energético y la degradación del desempeño, evitando interpretar una reducción de potencia como una mejora cuando venía acompañada de un incremento importante en el tiempo de ejecución. En las plataformas evaluadas, la propuesta reportó reducciones promedio de energía de 29.6 % con una pérdida de rendimiento de 5.2 % sobre NVIDIA GA100 y de 22.6 % con una pérdida de 4.7 % sobre NVIDIA GV100 [2].

Aunque estos enfoques permiten reducir el espacio de búsqueda, parten principalmente de una caracterización con la que se anticipa el comportamiento posterior de la aplicación. Cuando la carga cambia durante la ejecución aparece otra necesidad, reconocer el régimen que domina en cada momento. Antici et al. [4], mediante MCBound, abordaron esta identificación como un problema de clasificación entre cargas *memory-bound* y *compute-bound*, utilizando información de ejecuciones HPC para caracterizar automáticamente nuevos trabajos. Su evaluación sobre datos del supercomputador Fugaku mostró que esta separación puede realizarse con una puntuación F1-macro de al menos 0.89 y con un costo reducido para el funcionamiento del sistema [4]. Este resultado aporta evidencia de que la naturaleza computacional de una carga puede inferirse automáticamente sin depender únicamente de una clasificación manual basada en el conocimiento previo de la aplicación.

La clasificación de MCBound resulta especialmente cercana al problema de identificación del régimen, aunque su objetivo no es gobernar directamente los estados DVFS de CPU o GPU. Aun así, permite separar dos partes que aparecen de manera recurrente en los trabajos revisados, primero reconocer el comportamiento de la carga y luego decidir cómo utilizar esa información para modificar el punto de operación. La segunda parte ha sido explorada desde distintas aproximaciones orientadas a adaptar la frecuencia durante la propia ejecución.

Simsek et al. [31], por ejemplo, estudiaron esta actuación dinámica en simulaciones astrofísicas ejecutadas sobre GPU. Los autores instrumentaron SPH-EXA para medir el comportamiento energético de diferentes regiones y modificar la frecuencia del acelerador en puntos determinados de la ejecución. Frente a una política de frecuencia uniforme, esta adaptación permitió reducir el consumo energético por GPU hasta en 7.82 %, con una degradación del rendimiento de aproximadamente 2.95 %. El trabajo muestra que existe un beneficio potencial al cambiar la frecuencia durante la ejecución, aunque la decisión permanece asociada a regiones previamente conocidas e instrumentadas dentro de la aplicación.

Un paso más hacia un control *online* completo aparece en DRLCAP, un marco de *frequency capping* para GPU basado en aprendizaje por refuerzo profundo. DRLCAP monitoriza información a nivel de sistema, detecta cambios de fase y ajusta el límite de frecuencia sin intervención manual, logrando una reducción promedio de 22 % en la energía de GPU con menos de 3 % de degradación de rendimiento en arquitecturas NVIDIA [34]. Este resultado muestra que el control de frecuencia durante la ejecución puede automatizarse a partir de señales observadas en línea, aunque mediante un enfoque de aprendizaje por refuerzo profundo y concentrado exclusivamente en GPU.

Los trabajos revisados muestran una transición desde la aplicación de configuraciones DVFS sobre regiones conocidas hacia mecanismos que utilizan información de ejecución para anticipar o adaptar la frecuencia. En este recorrido, la caracterización de la carga ha permitido reducir búsquedas exhaustivas, mientras que enfoques como MCBound muestran que el régimen computacional también puede inferirse mediante clasificación. De forma paralela, los trabajos de Simsek et al. [31] y Wang et al. [34] evidencian que la frecuencia puede ajustarse durante la propia ejecución, aunque mediante estrategias ligadas a regiones previamente instrumentadas o a modelos de aprendizaje más complejos.

Aun con estos avances, gran parte de los enfoques recientes de predicción y control dinámico se concentra en GPU, mientras que CPU y GPU presentan diferencias tanto en sus mecanismos de frecuencia como en las señales disponibles para observar la ejecución. Calore et al. [7] consideran ambas arquitecturas, pero Guerreiro et al. [15], Ali et al. [2], Simsek et al. [31] y Wang et al. [34] profundizan principalmente en aceleradores gráficos. Esta diferencia deja espacio para estudiar estrategias que integren ambas plataformas y utilicen una caracterización de bajo costo como apoyo para decisiones DVFS durante la ejecución, sin depender de una instrumentación específica de cada aplicación.

Capítulo 2

Metodología

2.1. Enfoque y diseño

La investigación es cuantitativa, experimental y aplicada. Es experimental porque manipula de forma deliberada el estado de frecuencia de CPU y GPU sobre un conjunto controlado de cargas y observa su efecto sobre el tiempo, la energía y el Producto Energía–Retardo. Es aplicada porque su producto es un agente de software funcional cuya utilidad se evalúa empíricamente frente a gobernadores nativos del sistema operativo y del controlador de la GPU. El trabajo se organiza en cuatro fases encadenadas (Figura 2.1): la Fase 1 produce el conjunto de datos etiquetado; la Fase 2 entrena y valida los clasificadores y deriva las tablas de política; la Fase 3 implementa el agente; la Fase 4 lo evalúa. Los esquemas y cuadros de verificación complementarios se reúnen en el Anexo A.

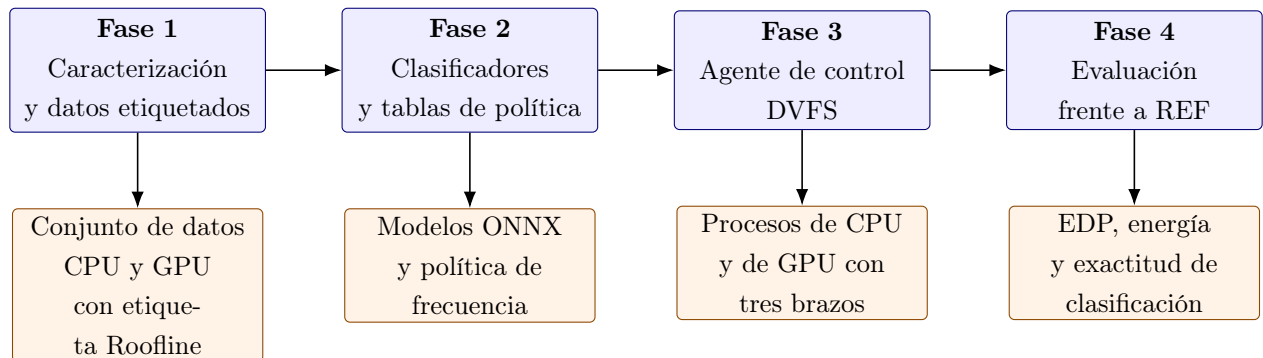


Figura 2.1: Fases de la metodología y producto de cada una. Fuente: elaboración propia.

2.1.1. Población, muestra y criterios de inclusión

La población son las aplicaciones para sistemas heterogéneos CPU–GPU; se estudió una muestra intencional de cargas reproducibles con regímenes contrastantes. Cada carga debía verificar su resultado, exponer trabajo de punto flotante medible por el instrumento y admitir una etiqueta de intensidad operacional. La unidad de diversidad y de validación fue la familia algorítmica: variantes de tamaño del mismo algoritmo no se trataron como familias independientes. El catálogo y su cribado se detallan en el Anexo A.

2.1.2. Plataforma experimental

Todos los experimentos se ejecutan en un único nodo, con asignación exclusiva del gestor de recursos para que ninguna carga ajena comparta el procesador durante las mediciones (Tabla 2.1). La plataforma se eligió porque expone la energía por interfaz interna sin privilegios administrativos (RAPL) y un acelerador instrumentable con NVML. Las actuaciones de frecuencia de CPU se restringen a los procesadores lógicos delegados, los núcleos 0 a 5 de un zócalo y sus hilos hermanos, porque el dominio de frecuencia es el núcleo físico y escribir un solo hilo no fija el reloj (anexo, sección A.1.1).

Tabla 2.1: Plataforma experimental.

Componente	Descripción
CPU	2 × Intel Xeon Gold 5315Y (Ice Lake-SP), 8 núcleos físicos por zócalo, 2 hilos por núcleo, 2 nodos NUMA
Frecuencia	<code>intel_pstate</code> , 0.8 a 3.2 GHz (3.6 GHz con <i>turbo</i> activo), dominio por núcleo físico
Energía de CPU	Intel RAPL, dominios de paquete y de DRAM por zócalo
GPU	NVIDIA A100 PCIe 40 GB, telemetría por NVML
Ejecución	Nodo en asignación exclusiva (Slurm)

2.1.3. Instrumento de medición

El instrumento propio tiene dos capas (Figura 2.2). La capa inferior, en C++, lanza la carga como proceso hijo detenido antes de su primera instrucción, abre los contadores de hardware sobre ese proceso con herencia y solo entonces lo reanuda, de modo que ninguna instrucción queda fuera de la observación y la medida abarca todo el árbol de procesos e hilos de la carga. Un hilo productor lee los contadores a intervalos regulares y deposita las muestras en una estructura circular de un productor y un consumidor; un hilo consumidor, en otro

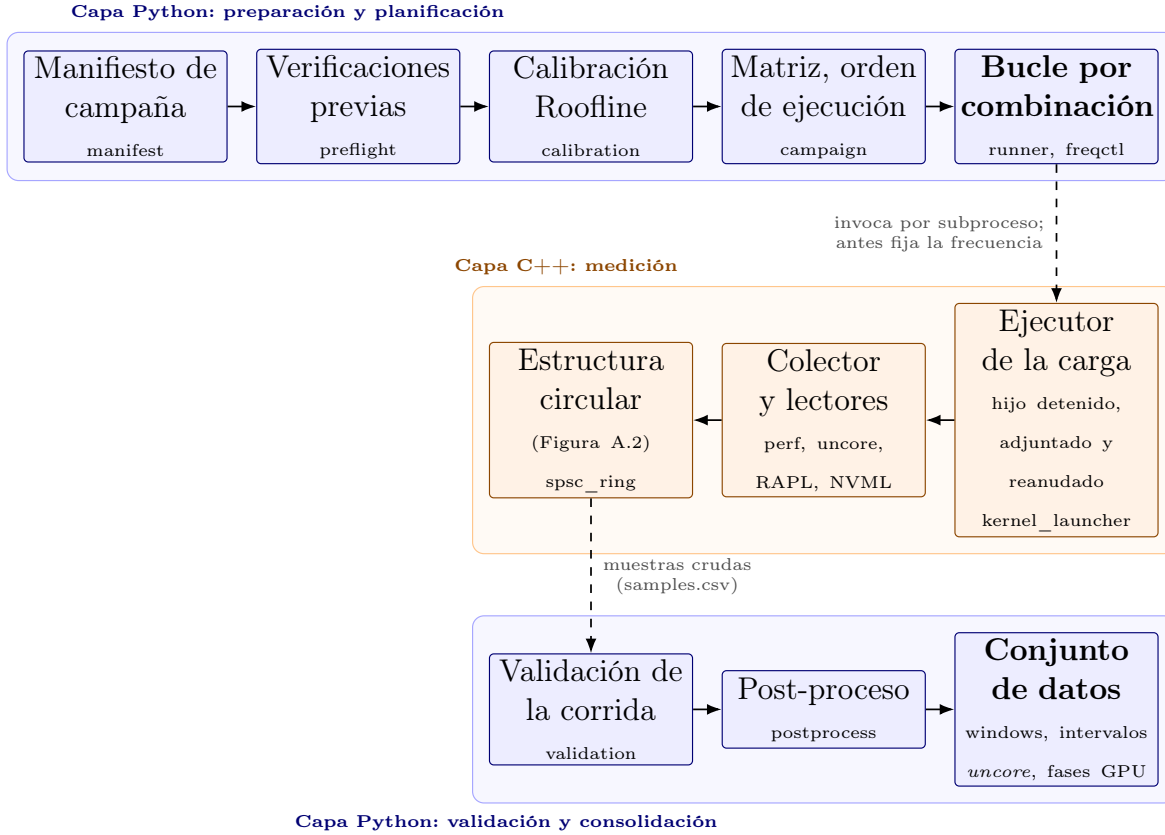


Figura 2.2: Recorrido de una campaña a través de las dos capas del instrumento. La capa Python prepara la campaña (banda superior), invoca por subprocesso el ejecutor de la capa C++ una vez por combinación de carga y frecuencia (banda intermedia), y consolida las muestras crudas que este produce en el conjunto de datos (banda inferior). La capa C++ concentra la medición sensible a la latencia; la capa Python, la lógica experimental, donde priman la claridad y la trazabilidad. Fuente: elaboración propia.

procesador lógico, las persiste (Figura A.2, anexo A). La capa superior, en Python, ejecuta la lógica experimental: interpreta el manifiesto de la campaña, verifica la plataforma, calibra, planifica cada combinación, aplica el estado de frecuencia, valida cada corrida y construye el conjunto de datos.

La Tabla A.4 resume las señales. Los contadores de CPU se limitan a nueve eventos simultáneos, dimensionados contra el presupuesto de contadores físicos del nodo, verificado empíricamente, para evitar la multiplexación y su extrapolación. Una **ventana de muestreo** es el intervalo entre dos lecturas consecutivas; su contenido es la diferencia de las lecturas acumuladas. El tráfico a DRAM se mide con los contadores del controlador de memoria (*uncore*), cuyo intervalo práctico es de 10 ms o más, ya que el manual de perf advierte de overhead crítico y desestabilidad sub-10 ms [25]. En GPU, el recolector solicita lecturas NVML cada 5 ms mediante `gpu_interval_ns=5 000 000`; sin embargo, la actualización física

de los sensores del controlador tiene una cadencia efectiva aproximada de 100–120 ms. Por ello, las muestras NVML se usan como telemetría agregada y no como ventanas GPU con resolución de 5 ms.

2.1.4. Modelo Roofline y etiquetado

La etiqueta de cada observación (*compute-bound* o *memory-bound*) resulta de comparar su intensidad operacional I con el punto de inflexión del modelo Roofline (ecuación (1.3)), calibrado en la plataforma por precisión aritmética y, en las campañas con barrido, por nivel de frecuencia. Los techos se calibran al inicio de cada campaña, con la misma asignación de núcleos que las corridas, mediante microbenchmarks de ancho de banda y de cómputo, y se contrastan con una referencia declarada. La etiqueta se deriva sin juicio manual, y la expectativa cualitativa de régimen que declara cada carga se usa solo como contraste.

Los dos dispositivos obtienen I de forma distinta (Figura 2.3). En CPU, los FLOPs se miden por ventana con los sub-eventos de `FP_ARITH_INST_RETIRED` ponderados por el ancho vectorial (ecuación (A.1)), y los bytes con los comandos CAS del controlador de memoria; I se calcula por intervalo, en vivo. En las campañas de GPU empleadas en este trabajo, NVML no proporciona una medición de FLOPs ni de bytes por ventana o por lanzamiento. Por ello, la intensidad operacional se caracteriza fuera de línea con Nsight Compute [30] (ecuación (A.2)) y se asocia a cada corrida. Una etiqueta con granularidad de fase requeriría combinar el perfilado de contadores con una traza temporal de lanzamientos mediante CUPTI, mecanismo que no forma parte de las observaciones aceptadas para este conjunto. Para un kernel determinista, con tamaño, precisión, entradas y configuración de lanzamiento fijos, esta caracterización se trata como una propiedad operacional estable tras verificar la convergencia entre los dos puntos de mayor trabajo, cuyo cambio relativo debe ser inferior al 1 %. Esta condición no afirma que el tráfico observado sea invariante ante cualquier configuración de ejecución. La caracterización distingue la precisión efectivamente ejercitada y excluye las cargas de aritmética entera o de precisión mixta no resoluble.

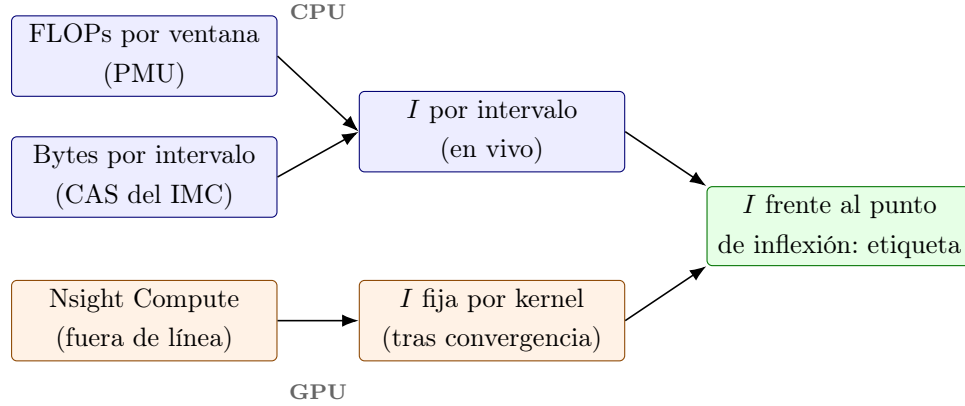


Figura 2.3: Obtención de la etiqueta Roofline. En CPU la intensidad se mide en vivo por intervalo del controlador de memoria; en GPU se caracteriza una vez por kernel, fuera de línea. Fuente: elaboración propia.

2.2. Fase 1: recolección y caracterización

2.2.1. Verificación previa y catálogo

Antes de medir, una batería de verificaciones automáticas, parte del comando que lanza la campaña, confirma las capacidades del entorno, la integridad del catálogo y la disponibilidad de la instrumentación (Tabla A.1, anexo A). Las verificaciones bloqueantes detienen la campaña antes de tocar el sistema: se prefiere no producir datos a producir datos cuya validez no puede sostenerse. El catálogo definitivo es una lista versionada y congelada de familias elegibles, tras un cribado con una rejilla de frecuencia reducida de tres niveles que descarta candidatos, comprueba la utilidad de su señal y estima su cobertura del plano Roofline. Cada clase debe quedar representada por al menos cinco o seis familias distintas por dispositivo. Con esto en mente, el conjunto de CPU reúne 30 familias, y el de GPU 16 (anexo, secciones A.2.2 y A.2.2).

2.2.2. Matriz experimental y control de sesgos

La matriz resulta de combinar las cargas del catálogo, los estados de frecuencia (Tabla A.3 del Anexo A) y las repeticiones independientes del binario. Los niveles se verifican contra los relojes soportados por el nodo. En CPU, fijar un nivel requiere acotar el rango por núcleo y desactivar el *turbo* global para evitar ajustes autónomos del gestor de energía. Cada ventana se clasifica como válida, no confiable, no verificable (margen tras transición) o no aplicable (nivel nativo).

Para mitigar la latencia de asentamiento del reloj, la cual fue medida bajo carga en

10–11 ms para CPU y 70–80 ms para GPU, se verificó cada nivel dentro de una tolerancia de $\pm 5\%$ y se aplicó un margen de guarda de 0.15 s al inicio y final de la región medible.

Las medidas contra el sesgo son cinco. Cada combinación se repite tres veces (validado empíricamente como suficiente). El orden de ejecución se aleatoriza con una semilla registrada en el manifiesto. Se incluyen repeticiones de una carga de referencia, cuya dispersión relativa señala inestabilidad del entorno. El eje de frecuencia de un dispositivo se mide con el otro dispositivo en su *governor* nativo, porque fijar la CPU en su nivel mínimo mientras se barre la GPU alarga las cargas de GPU y contamina la comparación. Y toda campaña reanudable conserva una huella del protocolo, de modo que dos manifiestos con protocolos distintos no se mezclan (anexo, sección A.2.3).

2.2.3. Calidad de los datos y granularidad

Ninguna observación se descarta en silencio. Cada ventana conserva una anotación de calidad (válida, sin diferencia previa, excluida por calentamiento, contadores degradados, intensidad indefinida, energía inválida o sin lectura de frecuencia) y el filtrado es una decisión del análisis. El transitorio de arranque de cada carga se identifica en dos etapas sobre una señal representativa del estado de carga (IPC en CPU, utilización en GPU): el primer instante desde el cual el coeficiente de variación se mantiene en 5 % o menos en dos ventanas móviles consecutivas y, como respaldo, un método de detección de puntos de cambio. Ese intervalo solo actúa como filtro posterior y nunca condiciona la adquisición. Cada corrida completa se valida además por integridad global: sin pérdidas en la estructura de muestreo, suma de verificación correcta y criterio de éxito propio cumplido (anexo, sección A.1.3).

La unidad de observación del entrenamiento se declara por dispositivo (Tabla A.5), porque en ambos casos la ventana del instrumento no coincide con la unidad que posee una etiqueta con respaldo físico. En CPU es un **intervalo del controlador de memoria**, con las variables recalculadas sobre los deltas agregados del intervalo y no promediando cocientes. En GPU es una **corrida completa**, con agregados robustos (mediana, dispersión) de las muestras válidas posteriores al calentamiento. Las lecturas repetidas de una misma corrida no se tratan como ejemplos independientes.

2.2.4. Reproducibilidad

Cada campaña queda definida por un manifiesto declarativo bajo control de versiones (cargas, estados de frecuencia, repeticiones, período de muestreo, asignación de núcleos, semilla y referencias de calibración). Junto con los datos se persisten el perfil de capacidades del

nodo, la calibración y los metadatos de cada corrida. Una campaña interrumpida se reanuda saltando las corridas ya aceptadas, protegida por la huella del protocolo (anexo, sección A.1.3).

2.3. Fase 2: modelos de clasificación y política de frecuencia

2.3.1. Formulación, variables y validación

Se formula una clasificación binaria supervisada por dispositivo; la salida es la probabilidad de la clase *memory-bound*. En CPU, las seis variables son cinco tasas derivadas de los contadores del intervalo (IPC, MPKI, tasa de fallos de caché, fracción de ciclos detenidos por memoria e instrucciones por segundo) y la frecuencia observada, conservada como covariable porque la misma carga puede cambiar de posición respecto del punto de inflexión cuando cambia el techo de cómputo. Antes de entrenar, se revisan las correlaciones entre las columnas candidatas. Los pares con correlación absoluta superior a 0.85 (común en la literatura), y la colinealidad restante, evaluada mediante el factor de inflación de la varianza, orientan el descarte de variables redundantes. El conjunto final se registra en un contrato versionado por dispositivo (anexo, sección A.3.1).

La validación es *leave-one-family-out*: en cada pliegue se retira por completo una familia, que no participa en el ajuste, la selección de representación ni la elección del umbral (Figura 2.4). Las celdas de familia y clase se limitan a 1000 intervalos con una semilla fija, y cada observación se pondera de forma inversa al tamaño de su celda (peso medio uno), de modo que ninguna celda domina por volumen. La métrica principal es la exactitud balanceada por celda de familia y clase,

$$EB = \frac{1}{|\mathcal{C}|} \sum_{(f,c) \in \mathcal{C}} \frac{a_{fc}}{n_{fc}}, \quad (2.1)$$

donde $\mathcal{C}$ es el conjunto de celdas observadas (cada combinación de familia f y clase c), $|\mathcal{C}|$ es el número de celdas, a_{fc} son los aciertos en cada celda y n_{fc} las observaciones totales de esa celda. Está definida aunque una familia tenga una sola clase y no depende de la prevalencia. La incertidumbre se reporta por dos vías: cinco semillas de submuestreo, y remuestreo con reemplazo de las familias (2000 réplicas) para los intervalos de confianza del 95 % y las comparaciones pareadas.

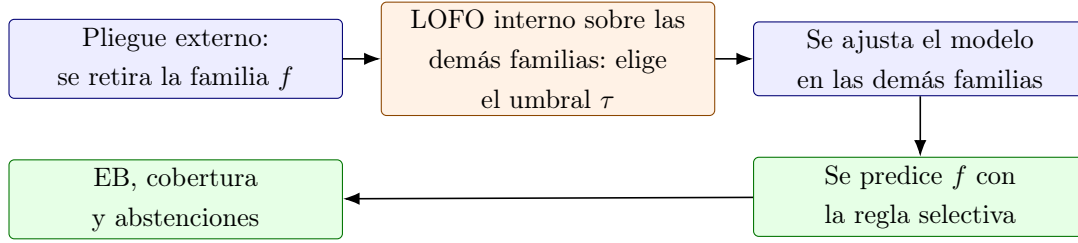


Figura 2.4: Validación anidada *leave-one-familia-out*. La familia retirada no interviene en ninguna decisión de ajuste. Fuente: elaboración propia.

2.3.2. Decisión selectiva y calibración

Con p_i la probabilidad de *memory-bound* y $q_i = \max(p_i, 1 - p_i)$ la confianza, la regla selectiva entrega la clase solo si $q_i \geq \tau$ y en caso contrario emite *revisar*:

$$\hat{y}_i = \begin{cases} \text{memory-bound}, & p_i \geq 0,5 \wedge q_i \geq \tau, \\ \text{compute-bound}, & p_i < 0,5 \wedge q_i \geq \tau, \\ \text{revisar}, & q_i < \tau. \end{cases} \quad (2.2)$$

El umbral τ se elige sobre una rejilla mediante un LOFO interno, maximizando la exactitud balanceada por celda calculada únicamente sobre las observaciones aceptadas, con una cobertura media por celda mínima de 0.60. Las salidas *revisar* se consideran abstenciones y se excluyen del cálculo de las métricas de clasificación sobre las predicciones aceptadas. Su efecto se cuantifica mediante la cobertura, definida como la proporción del total de observaciones para las cuales el modelo emite una clase. La confianza se evalúa además mediante el error de calibración esperado, que divide los valores de confianza en diez rangos de igual amplitud y compara, en cada rango, la confianza media con la proporción real de aciertos,

$$\text{ECE} = \sum_{b=1}^{10} \frac{n_b}{N} |\text{conf}_b - \text{acc}_b|, \quad (2.3)$$

donde N es el número total de observaciones evaluadas, n_b es el número de observaciones del rango b , conf_b es la confianza media de ese rango y acc_b es su proporción de aciertos.

2.3.3. Comparación y selección de modelos

Se compararon siete modelos bajo el mismo protocolo: una regla mayoritaria, un árbol de profundidad uno, uno de profundidad seis, regresión logística, *random forest*, *extra trees* y XGBoost, con hiperparámetros fijos. La búsqueda de hiperparámetros con Optuna se evaluó

anidada en cada pliegue externo y se descartó: con un número limitado de familias, el criterio interno que maximiza no discrimina de forma confiable la configuración que generaliza mejor. Para explorar si el desempeño observado estaba limitado principalmente por la capacidad del modelo o por la información contenida en las variables se aplicaron cuatro diagnósticos complementarios. Entre ellos se evaluó un clasificador de vecinos más cercanos sin ajuste, utilizado para examinar la separabilidad local de las clases en el espacio de variables (anexo, sección A.3.2). La selección del modelo final se realizó después de comparar el desempeño bajo el protocolo LOFO y de verificar la invarianza de las variables frente a la acción del agente. Los modelos y umbrales seleccionados se reportan en el capítulo de resultados.

2.3.4. Invarianza de las entradas frente a la acción del agente

La comparación por exactitud identifica candidatos que generalizan fuera de familia, pero un clasificador integrado en un controlador debe usar entradas que no cambien como consecuencia de su propia acción. En GPU se audita esa condición con dos pruebas sobre los datos ya disponibles. La prueba contrafactual sustituye el valor de una entrada potencialmente afectada por la actuación y cuantifica los cambios de predicción. La prueba de exclusión repite la validación LOFO sin dicha entrada. Toda variable que no supera ambas pruebas se excluye del contrato operativo.

Los modelos se vuelven a evaluar con el contrato de entradas resultante. La selección exige exactitud balanceada, calibración y cobertura suficientes bajo LOFO; la latencia de inferencia se informa como propiedad de implementación, mientras que la latencia completa desde la detección hasta la actuación se mide separadamente durante la evaluación del agente. En CPU se aplica la misma auditoría a la frecuencia observada, mediante una sustitución contrafactual coherente con el cambio de frecuencia que puede ejecutar el agente. Los resultados de ambas auditorías se presentan en las secciones 3.2.4 y 3.1.6.

2.3.5. Derivación de la política de frecuencia

La política de frecuencia se deriva de la campaña de la Fase 1, por separado del clasificador. Cada nivel se compara con REF sobre el mismo kernel y la ganancia agregada se estima por *bootstrap* de kernels. La regla de elección se valida dejando una familia fuera; cualquier refinamiento se congela antes de probarse en familias externas. En GPU, los kernels próximos al punto de inflexión se marcan como ambiguos sin excluirlos. El emparejamiento, las pruebas y los criterios de refinamiento se especifican en el Anexo A.

2.4. Fase 3: agente de control DVFS

La Fase 3 no vuelve a entrenar clasificadores ni vuelve a escoger frecuencias. Recibe como entradas congeladas tres artefactos de la Fase 2: el modelo ONNX de cada dispositivo, su umbral de abstención y la tabla de política. El modelo y el umbral proceden de la validación LOFO y de la auditoría de invarianza. La tabla de política procede, de forma independiente, de la comparación pareada de EDP entre niveles de frecuencia. Esta separación evita que el desempeño del clasificador se interprete como evidencia de que una frecuencia determinada mejora el EDP, o que la política de frecuencia determine la selección del modelo. La Fase 3 implementa detección, decisión, actuación, coordinación y restauración con esos artefactos sin modificarlos; la Fase 4 evalúa el conjunto ya congelado.

2.4.1. Arquitectura

El agente son dos procesos independientes en C++, uno por dispositivo, coordinados por una señal de un solo bit (Figura 2.5). Se separaron porque los bucles operan a escalas distintas (ventanas de milisegundos en CPU, fases de segundos en GPU), porque un fallo en uno no impide que el otro restaure el dispositivo que le corresponde, y porque los experimentos de un solo dispositivo no cargan la biblioteca de gestión del otro. Ambos leen los modelos, umbrales y tabla de política congelados al inicio de la ejecución; el lanzador los traduce a las banderas de cada proceso.

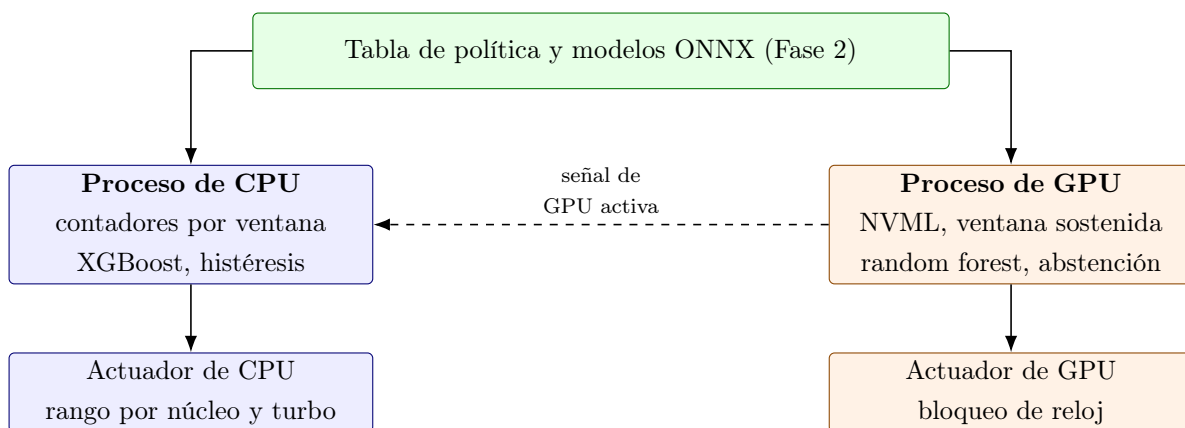


Figura 2.5: Arquitectura del agente: dos procesos, una tabla de política compartida y una señal de coordinación. Fuente: elaboración propia.

Cada proceso admite los tres brazos de la Tabla 2.2. El brazo *base* es la referencia contra la que se compara todo, y el brazo *sombra* mide la sobrecarga del agente sin su efecto sobre el hardware.

Tabla 2.2: Brazos del agente.

Brazo	Agente	Comportamiento
Base	No corre	El dispositivo queda bajo su <i>governor</i> nativo (<code>intel_pstate</code> con turbo en CPU, gestión automática del controlador en GPU)
Sombra	Muestrea, clasifica y decide	Registra cada decisión y nunca escribe sobre el hardware
Activo	Muestrea, clasifica, decide y escribe	Aplica la frecuencia decidida y restaura el estado al terminar

2.4.2. Detección de fase, decisión y parámetros

El agente de GPU delimita fases con la utilización NVML, reúne 3 s de actividad y decide una vez por fase. Si se abstiene, libera el reloj. El agente de CPU decide por ventana y exige 50 propuestas consecutivas antes de cambiar de nivel. Los umbrales y periodos se reúnen en la Tabla 2.3; la máquina de decisión se verificó por separado de la biblioteca de inferencia.

Tabla 2.3: Parámetros del agente.

Proceso	Parámetro	Valor y razón
GPU	Umbral de actividad	5 % de utilización
GPU	Ventana sostenida	3 s; una decisión por fase
GPU	Cadencia de muestreo	50 ms
GPU	Umbral de abstención	0.90 (cobertura media por celda ≥ 0.60); seleccionado en la Fase 2 (sección 3.2.4)
GPU	Permanencia mínima entre cambios	3.7 s, 10 veces la latencia de conmutación medida en la Fase 3 (sección 3.3.2)
CPU	Umbral de decisión	0.85; congelado en la Fase 2 (sección 3.1.4)
CPU	Histéresis	50 ventanas consecutivas
CPU	Piso con GPU activa	3.2 GHz; verificado en la Fase 3 (sección 3.3.3)

2.4.3. Política, actuación y coordinación

La Tabla 2.4 separa la política operativa derivada en la Fase 2 de las variantes experimentales de CPU. La salida operativa de la Fase 2 para CPU es *no actuar* en ambas clases, por lo que no se presenta F0 ni F1 como recomendación de despliegue. Para comprobar que esa decisión no oculta un efecto al cerrar el lazo de control, la Fase 4 incorpora un brazo de CPU experimental que fija F0 en cómputo y F1 en memoria, y una variante *activo-F0* que mantiene F0 en ambas. Estas variantes no actualizan la tabla derivada ni alteran el modelo; se comparan contra el *governor* nativo.

Tabla 2.4: Acciones del agente. Las filas de GPU provienen de la política derivada en la Fase 2; las filas de CPU son variantes experimentales evaluadas en la Fase 4.

Dispositivo	Clase	Acción
CPU	<i>compute_bound</i>	Fija F0 (3.2 GHz, turbo desactivado); experimental
CPU	<i>memory_bound</i>	Baja a F1 (2.9 GHz) con confianza mínima de 0.85; experimental
GPU	<i>memory_bound</i>	Fija el reloj en 1260 MHz (F1)
GPU	<i>compute_bound</i>	No actúa: libera el reloj al <i>governor</i> nativo
GPU	<i>revisar</i>	No actúa: libera el reloj al <i>governor</i> nativo (abstención)

El actuador de CPU reutiliza las reglas de restricción de rango por núcleo del módulo de la Fase 1 (delegados y hermanos de *SMT*), desactiva el turbo global con el envoltorio administrativo del nodo y escribe los núcleos en paralelo. Registra el estado previo, verifica cada escritura por relectura, falla cerrado ante cualquier inconsistencia y restaura el estado inicial al terminar, incluso ante **SIGTERM** o **SIGINT**; su equivalencia con el módulo de la Fase 1 se comprobó con una prueba de paridad. El actuador de GPU fija y libera el reloj con la herramienta de administración del controlador, con la misma disciplina de restauración.

La coordinación entre procesos es una regla de protección distinta de la tabla de política. Cuando la señal de actividad GPU está presente, impone temporalmente un piso de CPU de 3.2 GHz y no espera la histéresis. No reclasifica la fase de CPU ni convierte F0 en una acción recomendada por la Fase 2. Se adopta para evitar que las señales de una CPU que espera a la GPU desencadenen una reducción de frecuencia que prolongue la fase GPU. La Fase 3 verifica que el piso se aplique y se restaure; su efecto sobre EDP se mide por separado en la Fase 4 con el brazo sin piso.

2.4.4. Aplicaciones compuestas y verificación previa

Las aplicaciones compuestas registran con el reloj monotónico las fronteras reales de fase para puntuar al agente (Tabla 2.5). A usa familias vistas y B familias inéditas; una tercera aplicación alterna CPU y GPU. Cada fase supera los 3 s de observación y se separa de la siguiente por 1 s de reposo. Los kernels inéditos de GPU usan parámetros ampliados del mismo binario verificado para sostener la actividad.

Tabla 2.5: Aplicaciones compuestas con verdad de fase.

Alcance	Aplicación	Fases (clase)
CPU	A (vistos)	<code>dgemm_n2048</code> (<i>compute</i>), <code>npb_cg</code> (<i>memory</i>)
CPU	B (inéditos)	<code>cpu_xsbench_omp</code> (<i>memory</i>), <code>cpu_rsbench_omp</code> (<i>compute</i>)
GPU	A (vistos)	<code>gpu_cutlass_simt_dgemm_n4096</code> (<i>compute</i>), <code>gpu_rajaperf_stream_triad</code> (<i>memory</i>)
GPU	B (inéditos)	BabelStream con <code>-numtimes 1000</code> (<i>memory</i>), rodinia_lavamd con <code>-boxes1d 100</code> (<i>compute</i>)

Antes de la evaluación se verificó el agente por dispositivo y con ambos a la vez: latencia de conmutación, sobrecarga, restauración bajo terminación anómala inducida y cumplimiento del piso de coordinación (sección 3.3).

2.5. Fase 4: validación experimental

2.5.1. Diseño de la matriz

La matriz inicial compara el agente contra el *governor* nativo en tres ejes: el alcance (solo CPU, solo GPU o ambos a la vez sobre una aplicación que alterna fases), el tipo de kernel (vistos frente a inéditos, que mide la brecha de generalización con las mismas celdas) y el brazo (Tabla 2.6). El orden de las celdas se aleatoriza con semilla fija y cada una ejecuta la aplicación dos ciclos completos. El brazo *sombra* aísla la sobrecarga del agente; *activo-F0* y *activo sin piso* se incluyen como brazos experimentales porque la política de CPU resultó *no actuar* en la Fase 2 y se quiere medir si esa conclusión se sostiene con el agente en el lazo. La matriz inicial usó cinco repeticiones en el alcance conjunto y tres en los demás casos. Con tres observaciones por brazo, el análisis es descriptivo mediante razones de medianas y valores individuales; una comparación no apareada de tres contra tres no puede producir evidencia exacta bilateral inferior a 0.10.

Tabla 2.6: Matriz experimental de la Fase 4.

Alcance	Brazos	Vistos Inéditos Repeticiones		
CPU	base, sombra, activo, activo-F0	A	B	3
GPU	base, sombra, activo	A	B	3
Conjunto	base, sombra, activo con piso, activo sin piso	A	B	3 (5 en la matriz inicial)

2.5.2. Medición y análisis

En todos los escenarios, cada celda registra duración, energía de CPU (RAPL de ambos paquetes) y energía de GPU (contador NVML) por separado. Así, el EDP del nodo, $EDP = (E_{CPU} + E_{GPU}) T$, se calcula y se desagrega después; también se calcula $E_{GPU} T$. La ventana energética abarca desde el arranque de la aplicación hasta su fin, con los procesos del agente ya activos, por lo que incluye su sobrecarga. Cada celda verifica la finalización de la aplicación y de los procesos del agente que correspondan, y restaura los estados de frecuencia que modificó.

En la matriz y en el escenario E, las aplicaciones compuestas registran las fronteras de fase conocidas y las decisiones de cada proceso (instante, clase, confianza y escritura); la aplicación se fija a los núcleos 0 a 3 y se libera el reloj de GPU entre celdas. LAMMPS también se ejecuta en esos núcleos, pero sus fases son naturales y no se registran fronteras. CloverLeaf se ejecuta como un proceso CUDA Fortran continuo con ocho CPU asignadas; no se le impone ni se le atribuye una frontera de fase. Para CloverLeaf se conserva además la salida numérica, muestras NVML cada 250 ms y el estado de reloj antes y después de cada celda.

Para cada combinación de alcance, kernels y brazo se calcula la mediana de duración, energías y EDP y su razón frente al brazo *base* (menor es mejor). En la matriz inicial conjunta, que tiene cinco observaciones no apareadas por brazo, el EDP se contrasta con Mann–Whitney bilateral. Los dos confirmatorios de cinco bloques, E-A y CloverLeaf, se analizan por bloque: se informan las razones por bloque y la prueba exacta bilateral de signos. Con cinco pares, incluso cinco diferencias del mismo signo dan $p = 0,0625$, por lo que el valor se reporta sin declarar significancia al nivel 0.05.

La clasificación se puntúa contra las fronteras reales solo en las aplicaciones compuestas: cada decisión se atribuye a la fase que contiene su instante y se reportan correctas, erróneas y abstenciones por alcance, tipo de kernel y dispositivo, junto con la cobertura. La diferencia de

exactitud entre A y B mide la generalización dentro de ese protocolo. En CPU las decisiones son por ventana, así que la exactitud se pondera por ventanas y no por fases. En LAMMPS y CloverLeaf se reportan las decisiones y la actuación observada, pero no una exactitud de clasificación, pues no existe una verdad de fase independiente con esa granularidad.

2.5.3. Escenarios adicionales

Además de la matriz inicial, que se reporta completa y es la referencia, se ejecutaron los protocolos complementarios de la Tabla 2.7; sus condiciones completas se conservan en la Tabla A.7 del Anexo A. Cada protocolo se fijó antes de sus mediciones y todos los ejecutados se reportan, cualquiera que sea su resultado. El escenario E se definió con el criterio de selección de sus familias (ganancia de EDP con F1 de al menos 10 % en la Fase 2, más una variante con familias no vistas) y se verificó con una corrida previa de una repetición por brazo antes de la matriz completa.

La referencia secundaria sin turbo se incorporó en las campañas complementarias C y E. En ese brazo no se inicia ningún daemon: se desactiva el turbo de CPU inmediatamente antes de la ventana de medición, se verifica el estado solicitado y se restaura al terminar la celda. Se ejecutan tres repeticiones para cada combinación de aplicación y alcance incluida. El control conserva el *governor* nativo y el reloj nativo de la GPU, por lo que solo permite atribuir al turbo de CPU una diferencia entre referencias; no representa una frecuencia fija de GPU.

La Tabla A.6 resume las principales amenazas a la validez y la medida que la controla.

Tabla 2.7: Mapa de los escenarios de la Fase 4. El protocolo completo se conserva en el Anexo A.

Escenario	Contraste principal
A	Matriz inicial por dispositivo, familias vistas e inéditas y brazo
C	Agente de CPU con un hilo de inferencia
E	GPU con fases largas de memoria, familias vistas (E-A) e inéditas (E-B)
REF sin turbo	Línea base secundaria para aislar el turbo de CPU
D	LAMMPS con cuatro entradas y fases naturales
F	Premedición diagnóstica de la solicitud de F1 fijo
Confirmatorio E-A	Cinco bloques con REF, sombra, agente y solicitud de F1 fijo
CloverLeaf	Cinco bloques con REF, sombra y agente en aplicación continua

Capítulo 3

Resultados

Este capítulo organiza la evidencia por dispositivo. Primero se presentan los resultados de la campaña final de CPU. Después se reportan la validación del instrumento GPU, el diagnóstico exploratorio de su primer conjunto de entrenamiento y el cambio posterior de la estrategia de medición. Esta distinción evita presentar resultados intermedios como conclusiones finales. Las tablas y figuras de sensibilidad, desglose y trazabilidad se conservan en el Anexo B; el cuerpo principal mantiene la evidencia necesaria para seguir cada conclusión.

3.1. Resultados de CPU

El conjunto de CPU procede de una campaña que cubrió 49 configuraciones de kernel en el nivel nativo y nueve niveles fijos de frecuencia, con tres repeticiones por combinación, y sus 1470 corridas finalizaron aceptadas. En esta campaña, el warmup se calibró a partir de las ventanas medidas para cada kernel y el conjunto se reprocesó con esos valores; los transitorios fueron pequeños, con un máximo de 4.8 % de la duración de corrida y valores inferiores a 2 % en la mayoría de los kernels.

La unidad que alimenta el conjunto de entrenamiento CPU es el intervalo real de *uncore* de aproximadamente 10 ms (sección A.1.3). La Tabla B.22 resume la cobertura obtenida después del filtrado de calidad. De 1 424 531 intervalos procesados, 1 309 755 cumplieron simultáneamente los criterios de telemetría y de frecuencia. Los restantes se conservaron para auditoría, pero no recibieron una etiqueta de entrenamiento.

3.1.1. Composición de fase por kernel

Las cargas y su suite de origen se enumeran en la Tabla A.2. La Figura 3.1 muestra que el conjunto cubre ambos regímenes y conserva configuraciones próximas al *ridge*.

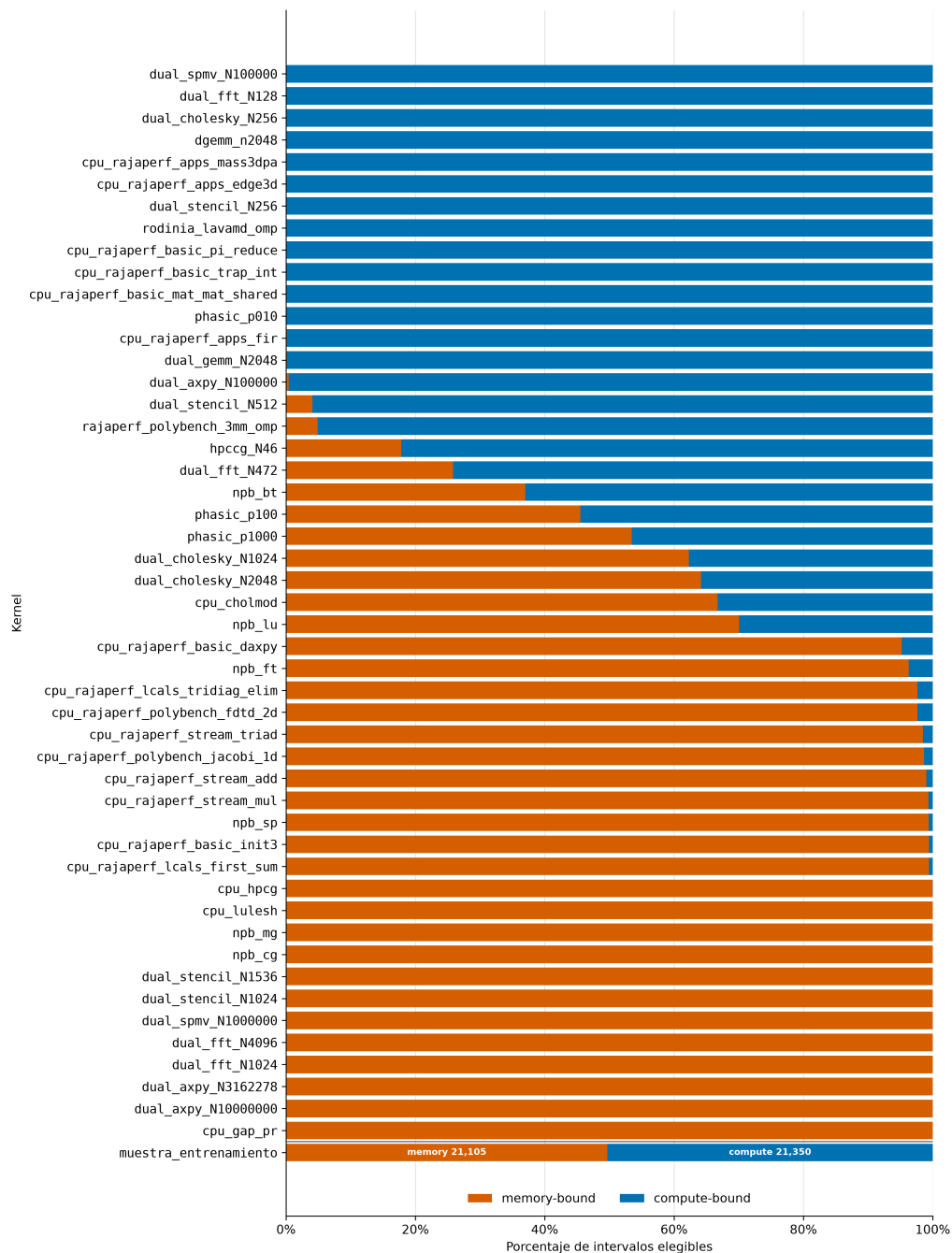


Figura 3.1: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* para cada configuración de kernel de la campaña CPU. La barra final resume la muestra usada por el clasificador, con 21 350 intervalos *compute_bound* y 21 108 intervalos *memory_bound*. Fuente: elaboración propia.

La composición intermedia no se concentra en un solo algoritmo. *npb_bt*, *npb_lu*, *cpu_cholmod*, *hpccg_cpu_N46*, las variantes *phasic* y los tamaños dirigidos de FFT y Cholesky aportan intervalos de ambos regímenes. Estas configuraciones evitan que el conjunto quede limitado a ejemplos alejados del punto de inflexión.

Tres kernels de patrón de acceso irregular de Rodinia (*kmeans*, *srad* y *particlefilter*) se midieron por separado y resultaron casi puramente *memory-bound*; como no aportaban ejemplos *compute-bound* ni mixtos, quedaron fuera del conjunto de entrenamiento por no añadir más que redundancia *memory-bound*.

3.1.2. Variación de la composición con la frecuencia

La proporción global de intervalos *compute_bound* aumentó desde 29.0% en el nivel nativo hasta 46.4% en F8, el nivel fijo más bajo. La Figura 3.2 presenta esta variación. El resultado es consistente con un *ridge* que disminuye cuando baja el techo de cómputo, de modo que una parte de los intervalos cercanos a la frontera cambia de etiqueta. La figura agrega todas las configuraciones y debe leerse junto con la composición por kernel, porque las cargas no aportan el mismo número de intervalos.

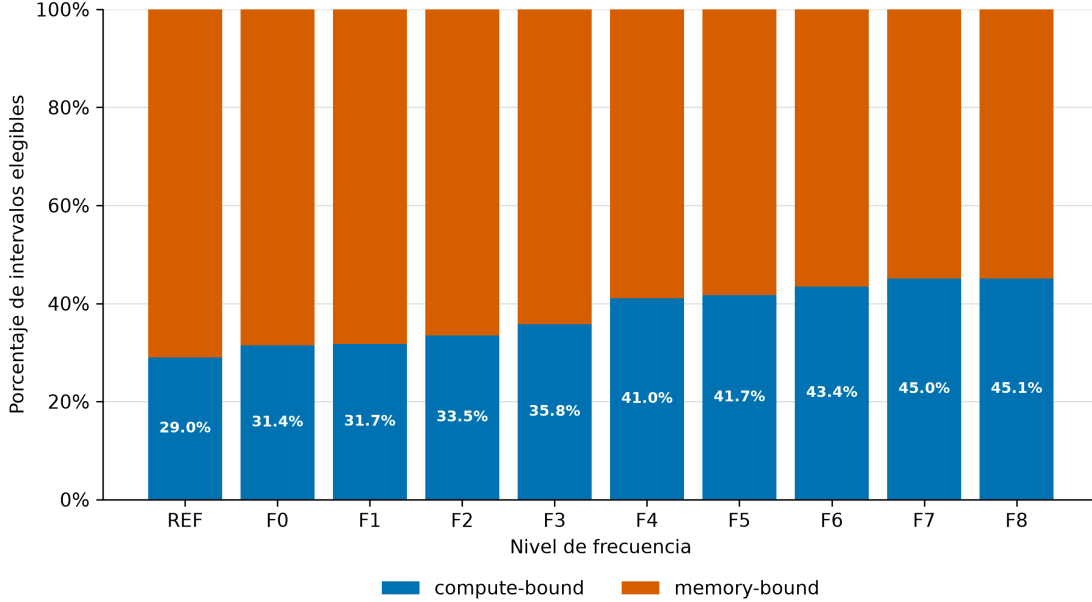


Figura 3.2: Porcentaje de intervalos elegibles clasificados *compute_bound* y *memory_bound* en cada nivel de frecuencia de la campaña CPU. Fuente: elaboración propia.

3.1.3. Calidad del conjunto de datos

Las 1470 corridas CPU se aceptaron con turbo desactivado y verificación del reloj observado dentro de $\pm 5\%$. Se descartaron los márgenes de transición (unos 10 ms) y de inicio y final (0.15 s), además de las ventanas con frecuencia o telemetría no utilizable. En total, el 91.9% de los 1 424 531 intervalos procesados quedó elegible (Tabla B.23). La prueba de actuación confirmó que era necesario restringir también los hilos hermanos de *SMT*.

Medición de bytes y etiqueta. Los bytes movidos en memoria se leen de los contadores de *uncore* del controlador de memoria (sección A.1.2), y son la única fuente de bytes de la etiqueta. La etiqueta de cada intervalo compara los FLOPs medidos con esos bytes contra el *ridge* calibrado para su nivel de frecuencia. La Figura 3.3 presenta cómo descende el *ridge* conforme baja la frecuencia. El presupuesto simultáneo es de nueve eventos de PMU y no diez, porque el vigilante NMI del sistema operativo reserva un contador físico por núcleo. Los contadores de *uncore* son de alcance de sistema, por lo que cada campaña exige el nodo en exclusiva; con esa condición, la cadencia de intervalos se sostuvo en los ~ 10 ms esperados a lo largo de toda una corrida de prueba. Antes de la campaña completa se ejecutaron dos campañas de humo a escala reducida (18 y 21 combinaciones) que ejercitaron el mismo camino de código y se aceptaron por completo; con clasificaciones coherentes: la multiplicación densa de matrices fue *compute-bound* en el 100 % de sus ventanas y los kernels de NAS resultaron predominantemente *memory-bound*.

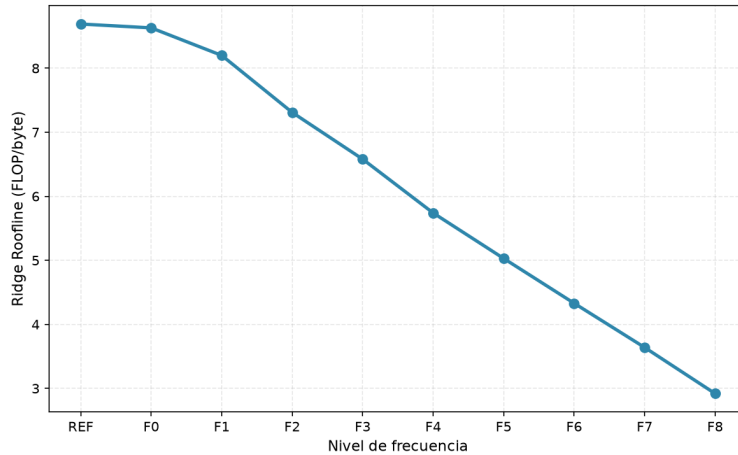


Figura 3.3: *Ridge* Roofline según el nivel de frecuencia CPU. El punto de inflexión descende de 8.69 FLOP/byte en referencia nativa a 2.92 FLOP/byte en F8. Fuente: elaboración propia.

La Figura 3.4 sitúa los intervalos del conjunto de entrenamiento directamente en el plano Roofline, contra el techo calibrado de cada nivel. Los FLOPs y bytes por intervalo no forman

parte del vector de entrada del clasificador (sección A.3.1), por lo que esta vista se reconstruye aparte, sobre las mismas corridas de la campaña, y se muestra solo para REF y F8 como los extremos del rango de frecuencia. La nube *memory-bound* queda, en su mayoría, bajo la rampa de ancho de banda a la izquierda del *ridge*, y la nube *compute-bound* bajo el techo plano de cómputo a la derecha; ambas quedan muy por debajo del techo teórico, porque los kernels reales sostienen una fracción de la tasa aritmética pico que un microbenchmark de registro alcanza (sección 1.1.2). La franja de intervalos que cruza el *ridge* en ambos sentidos es la misma ambigüedad que cuantifica la Figura 3.14.

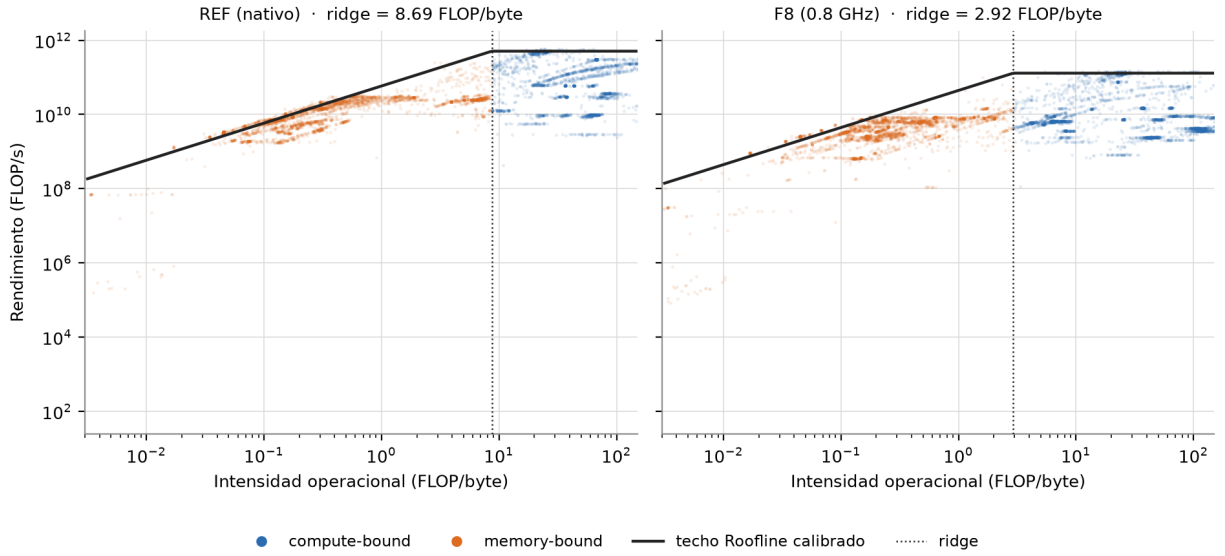


Figura 3.4: Submuestreo estratificado (4000 por nivel y clase, semilla 20260918) de los intervalos elegibles de REF y F8 en el plano intensidad–rendimiento, con el techo Roofline calibrado de cada nivel. Fuente: elaboración propia, reconstruido desde `training_cpu_intervals.csv` de la campaña `pacca_cpu_final_20260913`.

Validación de los techos Roofline. Los techos del modelo Roofline se calibraron de forma empírica sobre la propia plataforma (sección 1.1.2). Para verificar esa calibración con un mecanismo independiente se empleó Intel Advisor, que contabiliza FLOPs y bytes mediante instrumentación binaria y no mediante contadores de PMU [17]. El ancho de banda calibrado coincide con el de Advisor dentro de 12 a 15 % (58.4 a 58.8 GB/s frente a 64.0 a 66.8 GB/s) y el techo de cómputo (417.8 a 555 GFLOP/s) es consistente con la referencia de Advisor, que midió 577.7 GFLOP/s sobre los mismos seis núcleos. El punto de inflexión del nivel nativo queda entre 7.0 y 9.3 FLOP/byte, compatible con los 8.69 FLOP/byte usados en la campaña, y desciende con el reloj hasta 2.92 FLOP/byte en F8, porque baja el techo de cómputo mientras el ancho de banda sigue limitado por memoria.

3.1.4. Clasificador de fase en CPU

Esta sección sigue el orden de construcción del clasificador: la muestra de entrenamiento, las variables de entrada, la elección del modelo, las métricas con que se evalúa y, por último, el resultado. El procedimiento está descrito en las secciones A.3.1 a A.3.2.

Dos convenciones se usan desde la primera cifra y se definen por completo solo en la sección 3.1.4: toda evaluación sigue el protocolo *dejar una familia fuera (LOFO)*, es decir, cada familia algorítmica se retira por completo, el modelo se ajusta con las 29 restantes y se evalúa exclusivamente sobre la retirada; y la métrica principal es la **exactitud balanceada por celda**, el promedio de la fracción de aciertos de cada combinación de familia y clase, que pesa cada celda por igual con independencia de cuántos intervalos aporte.

Muestra de entrenamiento

El modelo CPU se ajusta con 42 458 intervalos de 30 familias y un máximo de 1000 por celda de familia y clase; la evaluación usa todos los intervalos elegibles de la familia retirada (Tabla B.24). La Figura 3.5 muestra el submuestreo por familia. La sensibilidad entre topes de 100 y 30 000 mantuvo la exactitud entre 0.720 y 0.739, sin tendencia creciente (Tabla B.25). La diferencia entre sortear por celda o repartir el tope entre niveles de frecuencia fue compatible con cero. La unidad de generalización sigue siendo la familia, no cada intervalo.

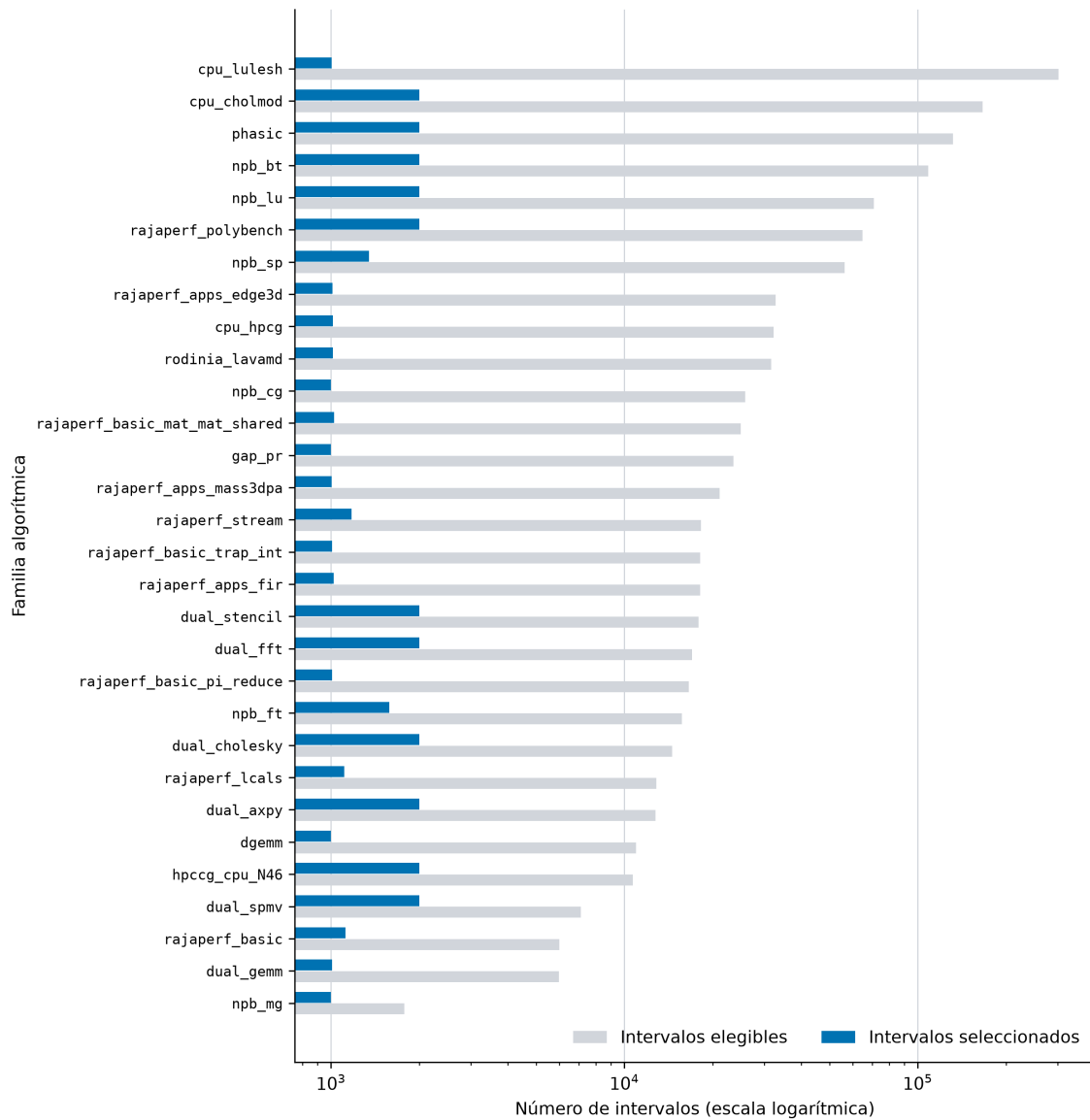


Figura 3.5: Intervalos elegibles y seleccionados por familia algorítmica. El eje logarítmico permite comparar familias cuyo volumen original difiere en varios órdenes de magnitud. Fuente: elaboración propia.

Variables de entrada

El modelo usa las seis variables de la Tabla 3.1: cinco tasas normalizadas calculadas con los contadores del intervalo y la frecuencia física observada.

Tabla 3.1: Variables de entrada del candidato CPU congelado.

Variable	Cálculo	Interpretación
<code>ipc</code>	$N_{\text{inst}}/N_{\text{cyc}}$	Trabajo retirado por ciclo
<code>mpki</code>	$1000N_{\text{miss}}/N_{\text{inst}}$	Fallos de caché por millar de instrucciones
<code>cache_miss_rate</code>	$N_{\text{miss}}/N_{\text{ref}}$	Fracción de referencias que fallan
<code>stall_mem_ratio</code>	$N_{\text{stall}}/N_{\text{cyc}}$	Fracción de ciclos detenidos por memoria
<code>ips</code>	$N_{\text{inst}}/\Delta t$	Tasa de instrucciones del intervalo
<code>freq_khz_observed</code>	Mediana de las lecturas cu- biertas	Frecuencia física observada

Quedaron fuera las columnas con las que se calcula la etiqueta (FLOPs, bytes de DRAM, intensidad operacional y *ridge*), porque con ellas el modelo no predeciría la etiqueta sino que la calcularía; los identificadores de kernel, nivel y repetición, que permiten reconocer la carga en lugar de su régimen; las señales de calidad del contador, que describen la adquisición y no la carga; y los deltas crudos, que dependen de la duración del intervalo y son redundantes frente a sus tasas.

La auditoría de correlación (sección A.3.2) se calculó sobre las doce columnas candidatas, antes de descartar las seis redundantes (Figura 3.6): el par más alto es 0.89 (Pearson), entre MPKI y las referencias de caché por millar de instrucciones, dos normalizaciones distintas de la misma señal de fallos de caché, y de ese par se retuvo MPKI. Sobre el vector final de seis variables, ningún par supera ya la correlación de 0.85: la correlación absoluta más alta es 0.74 (de Spearman), negativa, entre IPC y la fracción de ciclos detenidos por memoria, y la siguiente es 0.72 entre IPC e IPS. El factor de inflación de la varianza máximo es 5.5 con la correlación de Pearson y 7.3 con la de rangos, ambos de IPC, por debajo del umbral de alerta de 10, de modo que no fue necesario retirar ninguna variable adicional.

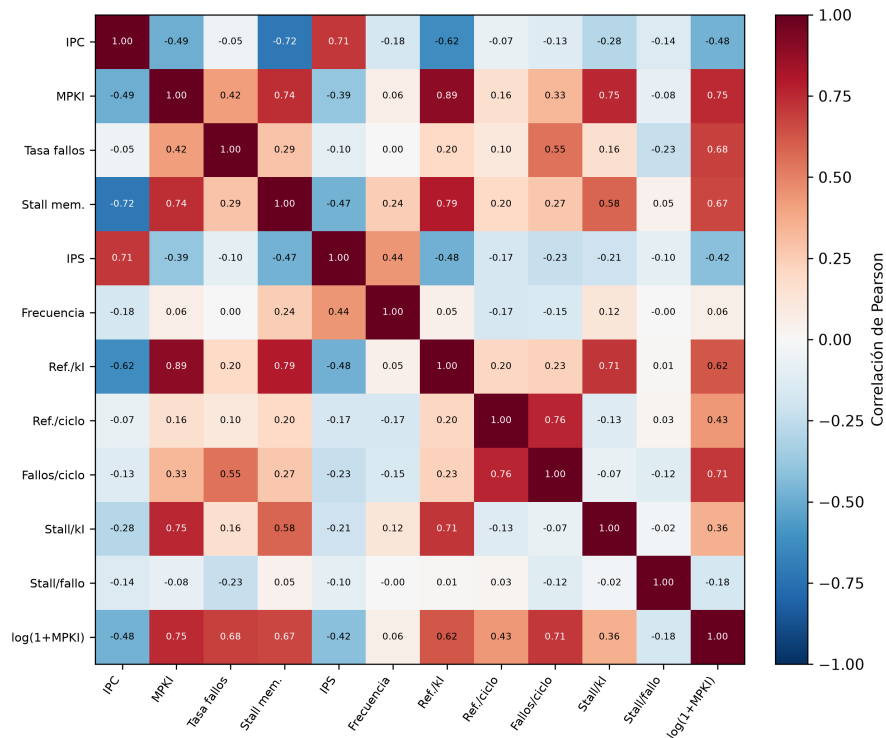


Figura 3.6: Correlación de Pearson entre las doce columnas candidatas, sobre los 1 309 755 intervalos elegibles, calculada antes de la selección del vector final de seis variables. Los pares más altos (MPKI con las dos normalizaciones de referencias de caché, 0.89 y 0.75) motivan retirar las columnas derivadas y conservar la medición más directa. Fuente: elaboración propia.

Ampliar el vector de seis a doce entradas redundantes cambió la exactitud balanceada de 0.728 a 0.724 (IC95 de la diferencia, -0.026 a $+0.011$; Tabla B.1). Por ello se conservaron seis variables. La redundancia perjudicó más a la regresión logística, sin ofrecer a XGBoost una mejora medible de inferencia o precisión.

La frecuencia observada podría servir de atajo para deducir la etiqueta, porque el *ridge* depende de ella. No ocurre: un modelo que solo la recibe alcanza 0.592, apenas por encima del azar, y retirarla del vector completo reduce el resultado solo 0.008 (IC95 de -0.018 a $+0.001$). En términos prácticos, el clasificador es un detector de presión sobre la jerarquía de memoria: al permutarlas, las variables que más deterioran el resultado son MPKI (0.055), la tasa de fallos de caché (0.038) y, en menor medida, la frecuencia (0.021), de modo que las seis columnas contienen bastante menos información que seis variables independientes.

Selección de modelo e hiperparámetros

La Tabla 3.2 y la Figura 3.7 comparan los candidatos con hiperparámetros fijos, cinco semillas y el mismo protocolo LOFO. XGBoost, Extra Trees y Random Forest quedan dentro del mismo intervalo de incertidumbre; la regresión logística rinde menos. Se seleccionó XGBoost por su menor latencia entre los modelos de árboles, más de trece veces inferior a la de los dos bosques.

Tabla 3.2: Comparación de modelos con hiperparámetros fijos, los seis a su vector final de seis variables. La sensibilidad de cada modelo al tamaño del vector de entrada se reporta por separado en la Tabla B.1. La latencia corresponde a una fila y un hilo, medida en el nodo de destino (paccaA100) con el proceso fijado a cuatro núcleos.

Modelo	Exact. bal.	IC95	F1 macro	Latencia p99
Regla mayoritaria	0.464	0.423 a 0.492	0.205	–
Árbol de profundidad 1	0.630	0.566 a 0.700	0.713	109 μ s
Regresión logística	0.680	0.618 a 0.742	0.765	475 μ s
Extra Trees	0.732	0.666 a 0.804	0.717	5477 μ s
Random Forest	0.737	0.675 a 0.803	0.763	5453 μ s
XGBoost	0.728	0.669 a 0.790	0.760	404 μ s

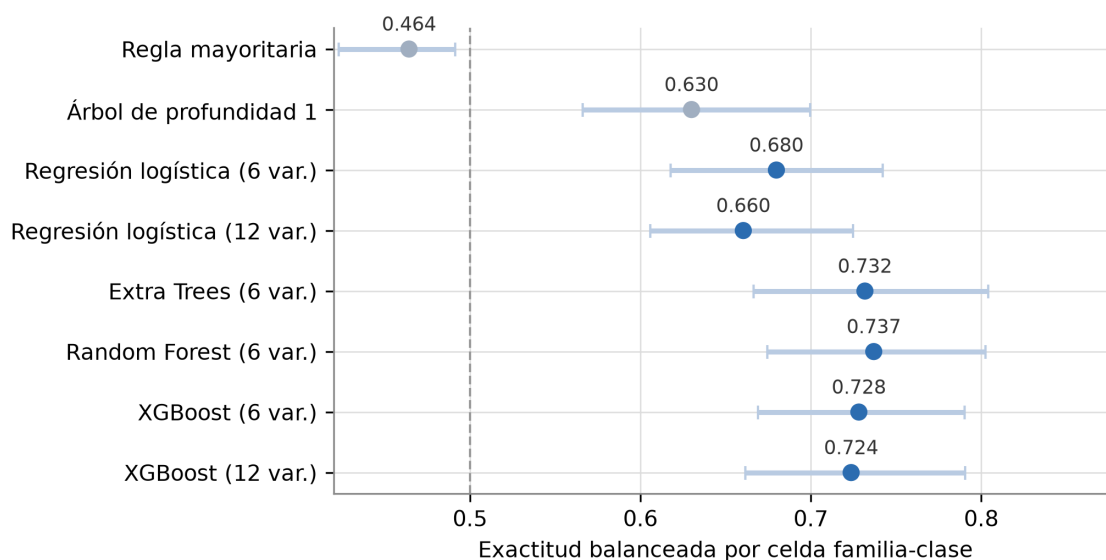


Figura 3.7: Exactitud balanceada por celda de familia y clase de cada modelo, con intervalo de confianza del 95 % obtenido por remuestreo de las 30 familias. Los intervalos se solapan entre todos los modelos con capacidad suficiente. Fuente: elaboración propia.

Búsqueda de hiperparámetros. Sobre esa base se ejecutó una búsqueda con el muestreador TPE de Optuna [1], anidada dentro de cada pliegue: con la familia retirada apartada, se puntúan las configuraciones con un LOFO interno sobre las 29 restantes, se ajusta la mejor y solo entonces se evalúa la familia retirada. La anidación impide que el resultado se infle por escoger la configuración que mejor se ajusta a la familia reportada. Se evaluaron 15 configuraciones por pliegue para XGBoost y 8 para la regresión logística.

Para XGBoost la búsqueda no mejora de forma distinguible la configuración fija: la diferencia pareada por familia es de +0,019, con un intervalo de confianza del 95 % de $-0,007$ a $+0,050$ que incluye el cero (14 familias mejoran, 8 empeoran, de 30). Para la regresión logística el efecto es, en cambio, estadísticamente distinguible de cero (IC95 de $+0,0004$ a $+0,0042$; 5 familias mejoran, 2 empeoran), pero de una magnitud irrelevante, $+0,002$: la búsqueda encuentra una configuración mejor de forma consistente, no por azar, y aun así no cambia en ninguna medida práctica el modelo desplegable. El puntaje interno de cada pliegue se mueve en un rango estrecho (de 0.734 a 0.761) mientras el resultado externo va de 0.487 a 1.000, de modo que el criterio interno no distingue configuraciones buenas de malas para una familia nueva. Por eso se conservó la configuración fija de XGBoost, que además es la más simple de reproducir. La conclusión, coherente con la comparación de modelos y con las variantes del vector de entrada, es que el desempeño no está limitado por la capacidad del modelo ni por el ajuste de sus hiperparámetros.

La configuración congelada es la de la Tabla B.26.

Métricas de evaluación

Esquema de evaluación. Un *pliegue* consiste en retirar una familia completa, ajustar el modelo con las 29 restantes y clasificar después los intervalos de la familia retirada. El procedimiento se repite 30 veces, una por familia, y ningún intervalo de la familia evaluada interviene en el ajuste ni en ninguna decisión de diseño. Responde la pregunta operativa pertinente: si el agente encuentra un algoritmo que nunca vio, ¿acierta su régimen? La alternativa más débil, acertar sobre otra ejecución de un algoritmo ya conocido, da resultados muy distintos con el mismo modelo y los mismos datos (Tabla B.2): repartir intervalos al azar produce 0.874, porque dos intervalos consecutivos de una misma corrida son casi idénticos y el modelo reconoce la carga en lugar de su régimen. Todas las cifras de este capítulo corresponden a dejar una familia fuera.

Qué es una celda. Una *celda* es la combinación de una familia algorítmica y una clase de fase; por ejemplo, los intervalos *compute-bound* de `cpu_cholmod` forman una celda y sus

Métricas empleadas. La Tabla B.3 define las nueve métricas. Para una clase c , VP_c son los intervalos de esa clase correctamente clasificados, FP_c los de otra clase clasificados como c y FN_c los de c clasificados como otra clase. Las ocho primeras agrupan todos los intervalos y por ello pesan cada familia según su volumen; la última pesa cada celda por igual.

Por qué la exactitud por celda es la métrica principal. No depende del volumen, de modo que una corrida larga no decide el resultado; no depende de la proporción entre clases dentro de una familia; y está definida también para las cuatro familias de una sola clase (`dgemm`, `gap_pr`, `npb_cg` y `npb_mg`), donde el F1 macro calculado dentro de cada familia queda acotado a 0.5 aunque todos sus intervalos se clasifiquen bien. La Tabla B.4 reúne todas las métricas del modelo final sin abstención.

La exactitud balanceada clásica (0.760) y la F1 macro agrupada (0.760) coinciden entre sí; la exactitud por celda (0.728) queda unos tres puntos por debajo porque, al pesar cada celda por igual, castiga las celdas de muy pocos intervalos en las que el modelo falla. La exactitud global de 0.769 es la más alta de las tres, pero es la que más depende de la composición del conjunto y no debe leerse como la capacidad del modelo.

Límite de la métrica. La exactitud por celda asigna el mismo peso a una celda de 5 intervalos que a una de 300 000. Diez de las 56 celdas tienen menos de 100 intervalos evaluados: suman 128 intervalos y pesan 17.9 % de la métrica; excluirlas la subiría de 0.728 a 0.777. Se conservan porque retirar celdas después de ver que fallan sería seleccionar datos por el resultado, y porque son informativas: muestran que las clases raras dentro de una familia casi pura no se reconocen.

Incertidumbre. Se informan dos fuentes. La semilla del submuestreo se varió cinco veces. La composición del catálogo se evaluó mediante remuestreo con reemplazo de las 30 familias, con 2000 réplicas. Esta segunda fuente domina y es la que debe usarse al comparar configuraciones: con 30 familias, cualquier diferencia menor que 0.03 es indistinguible del ruido.

Evaluación del modelo final

Se evaluó el modelo sobre los 1 309 755 intervalos elegibles, cada uno clasificado por el modelo del pliegue que no vio su familia, promediando las cinco semillas de muestreo. Si debe emitir siempre una clase, la exactitud balanceada por celda es 0.728, la F1 macro agrupada 0.760 y la exactitud global 0.769. La Tabla B.27 desagrega el resultado por clase y la Figura 3.9 muestra las matrices de confusión. En el promedio agrupado por intervalo, la sensibilidad

de *memory-bound* (0.806) supera en nueve puntos a la de *compute-bound* (0.714); en el promedio por celda de familia y clase la diferencia se invierte y se estrecha (0.700 y 0.759, respectivamente). Ambos promedios describen la misma asimetría: los intervalos *compute-bound* de una familia nueva se parecen más a intervalos *memory-bound* de familias ya vistas que en sentido contrario (sección 3.1.4), y no al desbalance del ajuste, que está corregido por la ponderación.

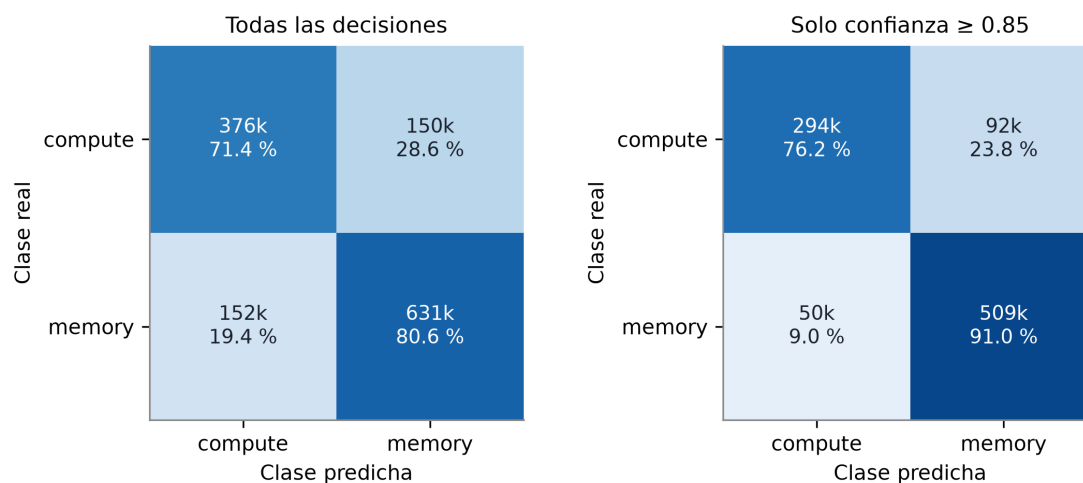


Figura 3.9: Matrices de confusión del modelo final. A la izquierda, cuando se obliga a emitir una clase en todos los intervalos; a la derecha, solo sobre los intervalos con confianza mayor o igual a 0.85. El porcentaje está calculado por fila, es decir, respecto de la clase real. Fuente: elaboración propia.

El resultado no es uniforme entre familias (Figura 3.10). Catorce familias superan 0.9 de exactitud y cinco quedan por debajo de 0.55, con *npb_ft* en 0.314. Las mejores son las de comportamiento extremo y estable; las peores, aquellas cuyo perfil de contadores coincide con el de familias de la clase contraria.

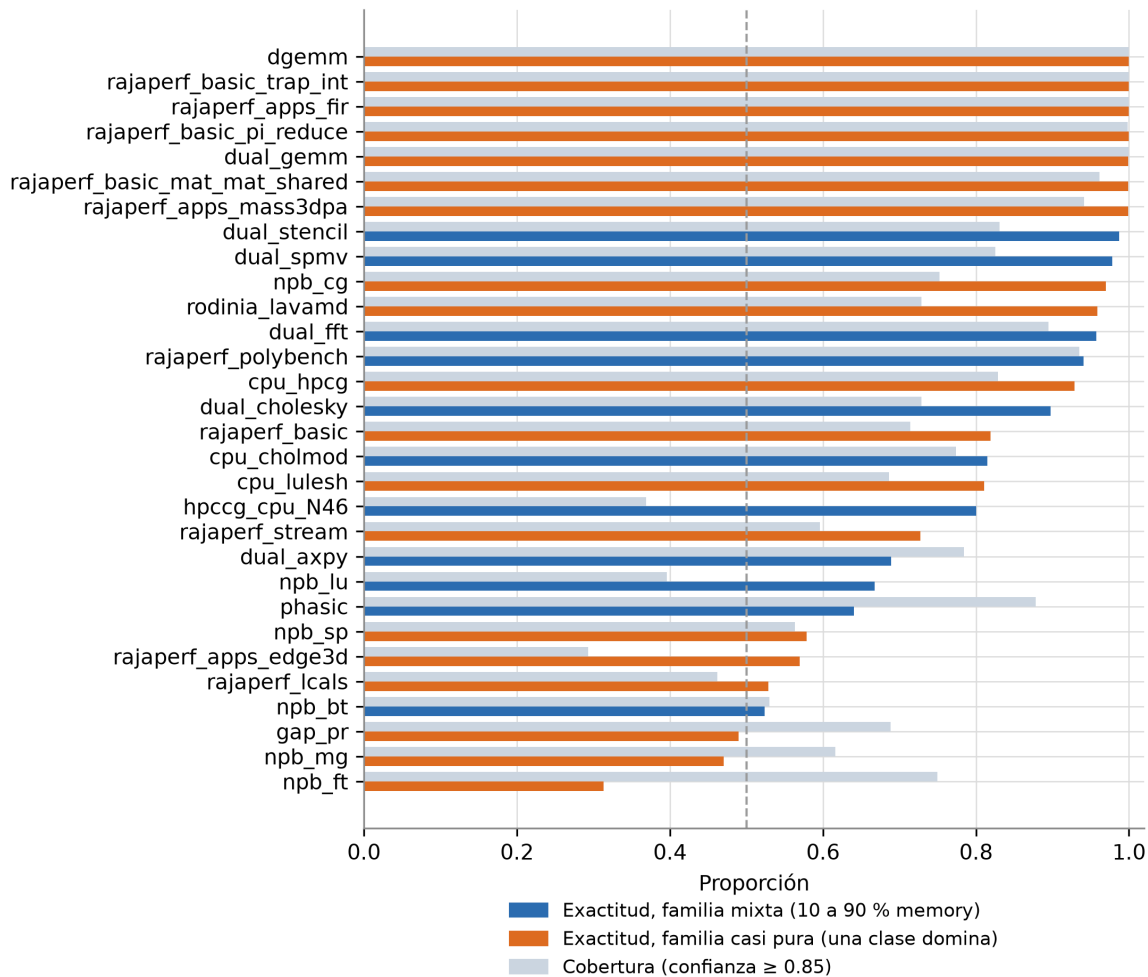


Figura 3.10: Exactitud del modelo final y cobertura de la decisión automática para cada familia retirada. El color distingue las familias mixtas (entre 10 y 90 % de intervalos memory-bound) de las casi puras. Fuente: elaboración propia.

La Figura 3.11 resume el desempeño por nivel. Por nivel de frecuencia, en la métrica principal el resultado va de 0.823 en el nivel nativo a 0.657 en F7 (Tabla B.5) y decrece, en general, al bajar el reloj. En los niveles bajos el *ridge* desplaza más intervalos hacia la clase de cómputo (la fracción *memory-bound* baja de 0.71 a 0.54) y la cobertura de la decisión automática cae de 0.85 a 0.64, lo que es consistente con que allí más intervalos quedan cerca de la frontera entre clases. La exactitud global repunta en F8 (0.757, frente a 0.710 en F7), pero ese repunte es casi enteramente un efecto de composición de clases: en la exactitud balanceada por celda, que no depende de cuántos intervalos aporta cada clase, F8 (0.659) queda prácticamente igualado con F7 (0.657) y no rebota. F8 es también el nivel con más

intervalos (256 531, frente a 85 447–186 309 en los demás), de modo que su exactitud global pesa más hacia la clase *compute-bound* mayoritaria en ese régimen, que el modelo reconoce mejor a esa frecuencia.

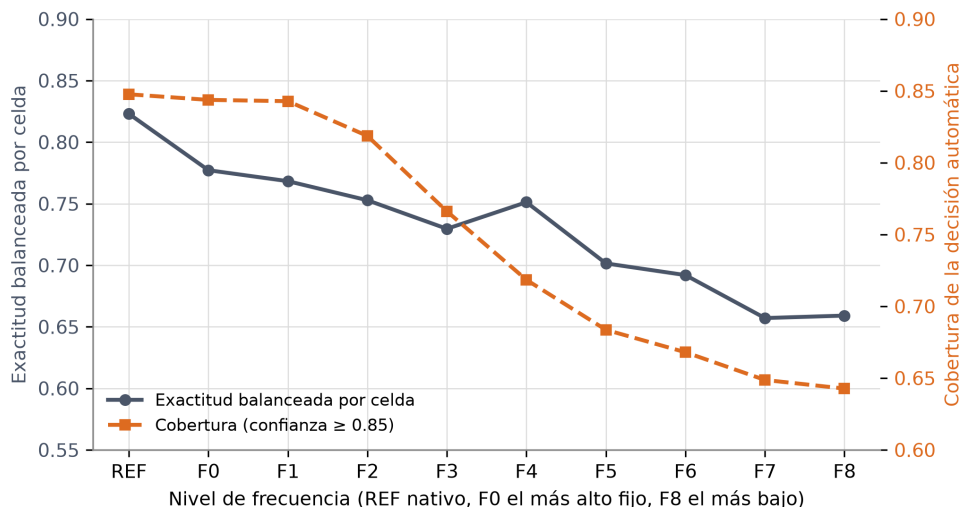


Figura 3.11: Exactitud balanceada por celda y cobertura de la decisión automática (confianza $\geq 0,85$) en función del nivel de frecuencia. Ambas caen de forma conjunta al bajar el reloj. Fuente: elaboración propia.

Decisión selectiva y costo de inferencia

El umbral de abstención CPU se eligió con LOFO interno y un piso de cobertura media por celda de 0.60 (Tabla B.6). Aunque la rejilla proponía 0.90, se congeló 0.85: el aumento de exactitud de selección era solo de 0.003 y reducía la cobertura en 6.5 puntos. La evaluación externa del modelo ya congelado se presenta a continuación; el criterio interno y ese resultado no son la misma estimación.

Con el umbral 0.85 ya congelado, evaluado externamente sobre el modelo final, la cobertura es del 72.2 % de los intervalos (75.1 % promediando por familia); el resto recibe *revisar*. Sobre lo que decide, la exactitud balanceada por celda sube de 0.728 a 0.738, la F1 macro agrupada de 0.760 a 0.842 y la sensibilidad de *compute-bound* de 0.714 a 0.762 (Tabla B.7); la ganancia por celda es modesta y la mayor parte de la mejora se ve en las métricas agrupadas. Abstenerse al azar en una proporción equivalente de intervalos deja la exactitud balanceada en 0.716 y la F1 macro en 0.752, sin mejora, de modo que la ganancia proviene de que la confianza identifica casos difíciles y no de decidir menos.

El límite de la abstención es que la confianza está mal calibrada fuera de la familia de entrenamiento (Figura 3.12): la confianza media declarada es 0.890 frente a una exactitud

observada de 0.764, con un error de calibración esperado de 0.126. Hay familias que fallan *con* confianza alta y a las que el umbral no protege: entre los intervalos con confianza mayor o igual a 0.9, *npb_ft* acierta el 32 % y *npb_mg* el 35 %.

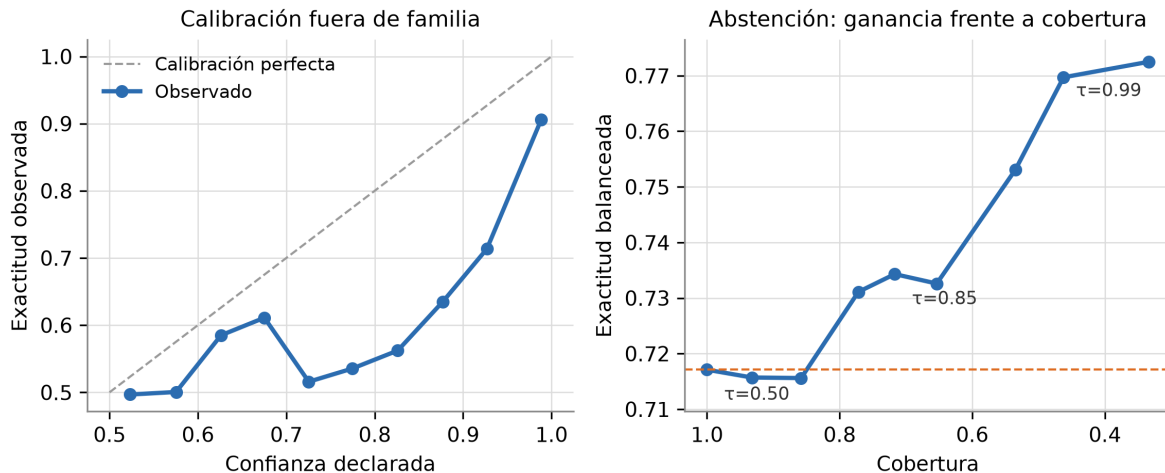


Figura 3.12: A la izquierda, confianza declarada frente a exactitud observada sobre familias no vistas; la curva por debajo de la diagonal indica sobreconfianza. A la derecha, exactitud balanceada alcanzable en función de la cobertura al variar el umbral; ambas curvas usan una sola semilla de muestreo, por lo que sus valores difieren ligeramente de los promedios de cinco semillas de la Tabla B.7. Fuente: elaboración propia.

La salida *revisar* activa una política conservadora: el sistema conserva la frecuencia vigente y espera dos intervalos *uncore* válidos consecutivos antes de aceptar una caracterización física. En cuanto al costo, la inferencia de una fila toma 243 μ s de mediana y 404 μ s en el percentil 99 para XGBoost, frente a 450 y 475 μ s de la regresión logística y 5.3 y 5.5 ms de los bosques, medidos en paccaA100. Estas cifras incluyen la sobrecarga del intérprete de Python y son una cota superior para una implementación integrada en el agente.

Dónde está el límite del resultado

Cuatro resultados independientes indican que el límite proviene de la información contenida en los contadores y no del modelo.

El primero es que un clasificador sin ajuste, de 25 vecinos más cercanos sobre cuatro tasas estandarizadas, alcanza 0.684 con el mismo protocolo, y 0.699 si recibe además la frecuencia: queda a menos de 0.045 del modelo final. El segundo es la existencia de perfiles gemelos entre familias de clase opuesta (Figura 3.13). El caso más simétrico enfrenta a *gap_pr*, *memory-bound*, con la fase *compute-bound* de *phasic*: la primera tiene MPKI de 0.82, IPC de 1.11

y tasa de fallos de 0.76; la segunda, MPKI de 0.94, IPC de 0.97 y tasa de fallos de 0.72. El par más cercano en distancia estandarizada, sin embargo, es otro: la celda rara *memory-bound* de *rajaperf_basic_mat_mat_shared* (apenas el 0.1 % de sus intervalos; la familia es, en conjunto, *compute-bound*) queda a la mitad de distancia de la celda *compute-bound* típica de *rajaperf_basic_pi_reduce*, dos kernels de la misma suite. Ningún modelo que reciba solo esas señales puede separar estos pares, y las tres familias involucradas están entre las de peor resultado.

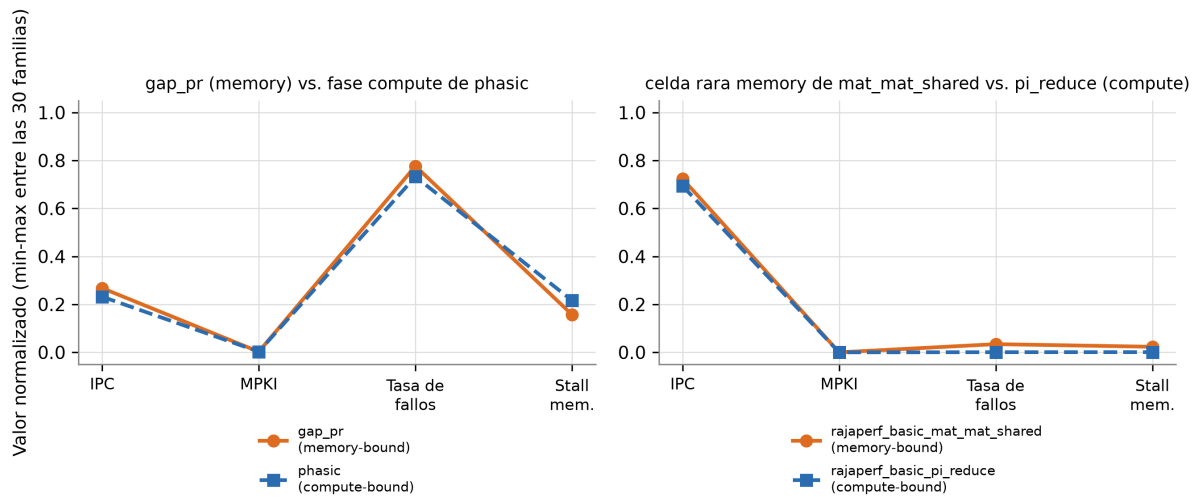


Figura 3.13: Cuatro variables de entrada, normalizadas al rango de las 30 familias, para los dos pares de celdas más confusables entre clases opuestas. Fuente: elaboración propia.

El tercero es la posición respecto del *ridge* (Figura 3.14). Los intervalos situados a menos de un factor dos del punto de inflexión son el 12.5 % del total y se clasifican con 0.521 de exactitud, frente a 0.799 en el resto; concentran el 25.3 % de los errores. La ambigüedad cerca de la frontera explica una parte del error, pero tres cuartas partes de los fallos ocurren lejos del *ridge*, donde la etiqueta no es ambigua. Ese error residual, además, no es simétrico alrededor de la frontera: en la franja inmediatamente *compute-bound* (de 0 a 0.5 octavas por encima del *ridge*) la exactitud es 0.318, mientras que en la franja simétrica del lado *memory-bound* (de -0.5 a 0 octavas) es 0.766. El modelo no se equivoca por igual a ambos lados de la frontera: cerca de ella, empuja sistemáticamente hacia *memory-bound*, lo mismo que ya se observa en el déficit de sensibilidad de *compute-bound* frente a *memory-bound* (sección 3.1.4), y es consistente con los perfiles gemelos del párrafo anterior, en los que la fase *compute-bound* de *phasic* se confunde con *memory-bound*.

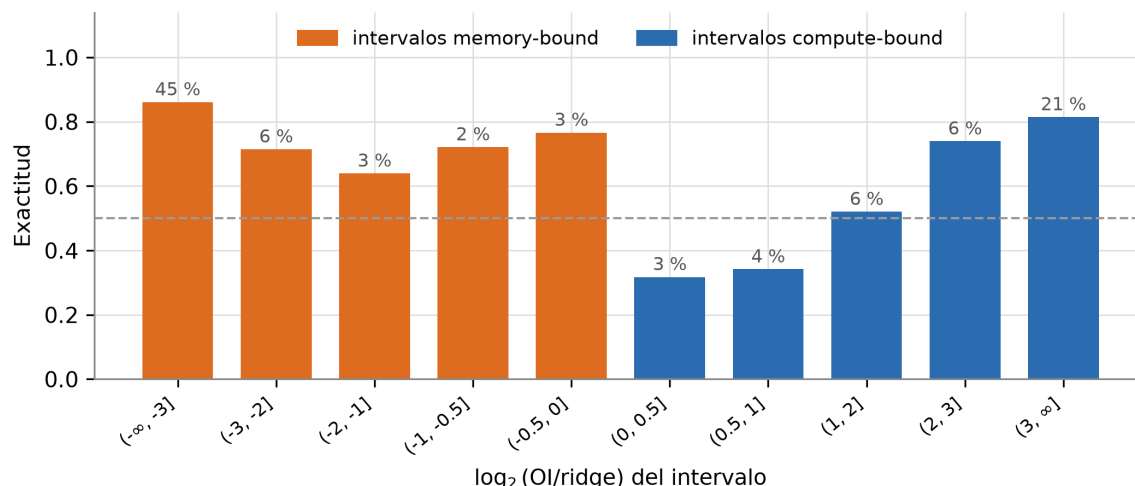


Figura 3.14: Exactitud del modelo final por franja de $\log_2(\text{OI/ridge})$. El porcentaje sobre cada barra indica qué fracción de los intervalos cae en esa franja. Las franjas negativas corresponden a intervalos *memory-bound* y las positivas a *compute-bound*. Fuente: elaboración propia.

El cuarto es la curva de aprendizaje por número de familias (Figura 3.15). Al ajustar con subconjuntos de 3 a 23 familias, el resultado sobre la familia retirada sube de 0.615 a 0.718, lo que implica aproximadamente 0.037 de ganancia cada vez que se duplica el catálogo. La diversidad algorítmica gobierna el resultado, y la mejora disponible por esa vía, dentro del alcance de este trabajo, es pequeña.

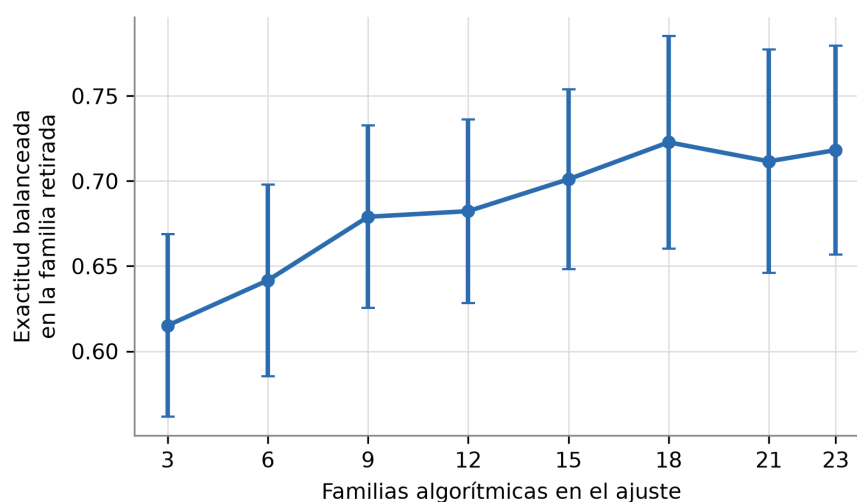


Figura 3.15: Exactitud balanceada sobre la familia retirada en función del número de familias empleadas en el ajuste. Cada punto promedia las 30 familias retiradas y tres repeticiones de la selección; las barras indican el intervalo de confianza del 95 % entre familias. Fuente: elaboración propia.

Como referencia de límite superior, el mismo modelo con la intensidad operacional medida como entrada adicional alcanza 0.9989, frente a 0.728 sin ella. No es una mejora del modelo, porque la intensidad operacional es la cantidad con la que se define la etiqueta; por eso queda fuera del vector de entrada y solo muestra que la brecha entre 0.728 y 1 es información que los contadores de rendimiento no transportan.

Síntesis del clasificador CPU

El modelo es un XGBoost de 100 árboles y profundidad 6 que recibe seis variables derivadas de los contadores de rendimiento (cinco tasas normalizadas y la frecuencia observada), se ajusta dando el mismo peso total a cada combinación de familia y clase, y responde *compute-bound*, *memory-bound* o *revisar* según su confianza supere o no 0.85. Ante un algoritmo que nunca vio clasifica correctamente el 72.8 % de los casos en la métrica balanceada por celda, cifra que sube a 73.8 % si se le permite abstenerse en el 27.8 % de los intervalos. La referencia sin capacidad de discriminar queda en 0.464 y la inferencia cuesta 404 μ s en el peor percentil medido.

Tres decisiones de diseño resultaron indiferentes: el algoritmo de aprendizaje, el tamaño del vector de entrada y los hiperparámetros. Elegir Random Forest en lugar de XGBoost, usar seis variables en lugar de doce o ajustar los hiperparámetros mueve el resultado menos de lo que lo mueve cambiar de conjunto de familias. Lo que gobierna el resultado es la diversidad del catálogo frente a la información de los contadores: la etiqueta se define por FLOPs sobre bytes de DRAM, mientras que el modelo observa fallos de caché, ciclos detenidos e instrucciones retiradas, que no guardan una relación fija con esa razón cuando cambia el algoritmo.

Quedan tres líneas de trabajo, fuera del alcance de esta fase. La primera es dar contenido a la salida *revisar*: en lugar de solo esperar, el agente de la Fase 3 podría medir la etiqueta física de una fracción mínima de intervalos de la carga en ejecución; esa capacidad se diseña y evalúa allí, con su propio método y protocolo, y no se anticipa aquí con una cifra suelta. La segunda es ampliar el catálogo con familias elegidas por su distancia a los perfiles ya cubiertos y reservar algunas para una prueba externa sellada; las seis familias de cómputo denso de RAJAPerf provienen de una misma suite, de modo que su independencia entre sí es menor que la de familias de suites distintas. La tercera es medir la latencia dentro del agente y no en el intérprete.

Los datos, las figuras y las tablas de esta sección se reproducen con `fase2_clasificador/analysis/cpu_quality_report.py` y `docs/libro/scripts/generar_figuras_modelo_cpu.py`, sobre los artefactos de `docs/libro/datos/cpu_calidad_30fam`.

3.1.5. Política de frecuencia por clase en CPU

La política CPU compara la mediana de tres repeticiones por kernel y frecuencia con su propio REF. Un nivel debía mejorar el EDP agregado con evidencia estadística y al menos 1 % para adoptarse; ese mínimo se fijó después de ver el resultado y se declara como tal. Se excluyeron las variantes *phasic*, cuyo trabajo cambia con la frecuencia pese a su duración fija. La Tabla 3.3 resume la decisión; la rejilla completa está en la Tabla B.29 del anexo.

La política resultante es *no actuar* en ambas clases (Tabla 3.3 y Figura 3.16). La Figura 3.17 separa la energía y el tiempo que producen ese resultado. En cómputo, F0 mejora 0.56 % frente a REF, menos que el efecto mínimo relevante de 1 %; todos los niveles inferiores empeoran el EDP. En memoria tampoco hay ganancia agregada a menor reloj. Algunos kernels STREAM presentan ahorros puntuales entre 4 y 11 % en su mejor nivel, pero el signo y la magnitud varían entre niveles y repeticiones; no justifican una regla común.

Tabla 3.3: Síntesis de la política CPU. Las ganancias son de EDP frente a REF; la rejilla completa se conserva en la Tabla B.29 del Anexo B.

Clase	Evidencia decisiva	Política
Cómputo	F0: +0,56 % (IC95 [0,27; 1,38]), por debajo del mínimo relevante de 1 %; F1–F8 empeoran	No actuar
Memoria	F1: –5,4 % (IC95 [–6,8; –3,4]); ningún nivel bajo mejora el agregado	No actuar

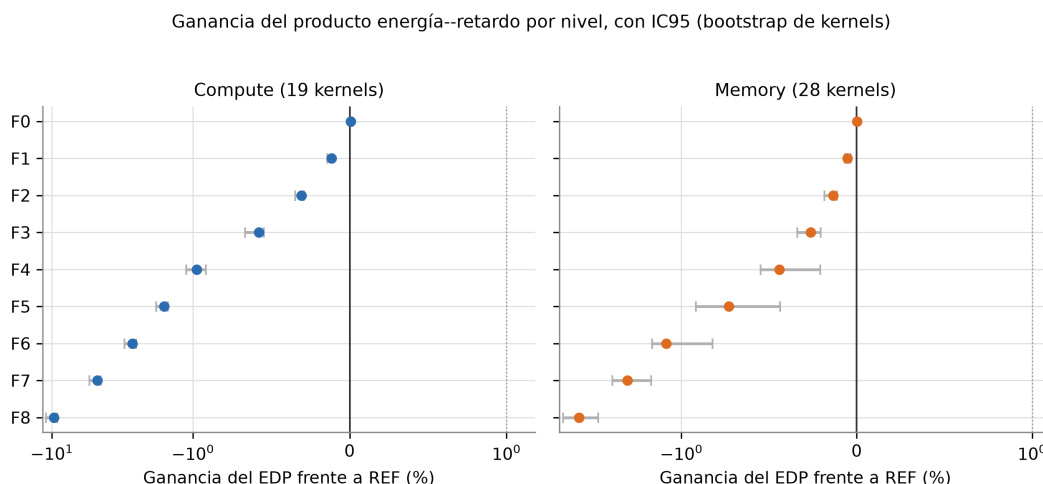


Figura 3.16: La rejilla completa de la Tabla B.29, en forma gráfica. El eje horizontal es simétrico-logarítmico para mostrar a la vez F0, cercano a cero, y F8, con una pérdida de casi un orden de magnitud. Ningún nivel deja el intervalo de confianza por encima de cero de forma útil. Fuente: elaboración propia.

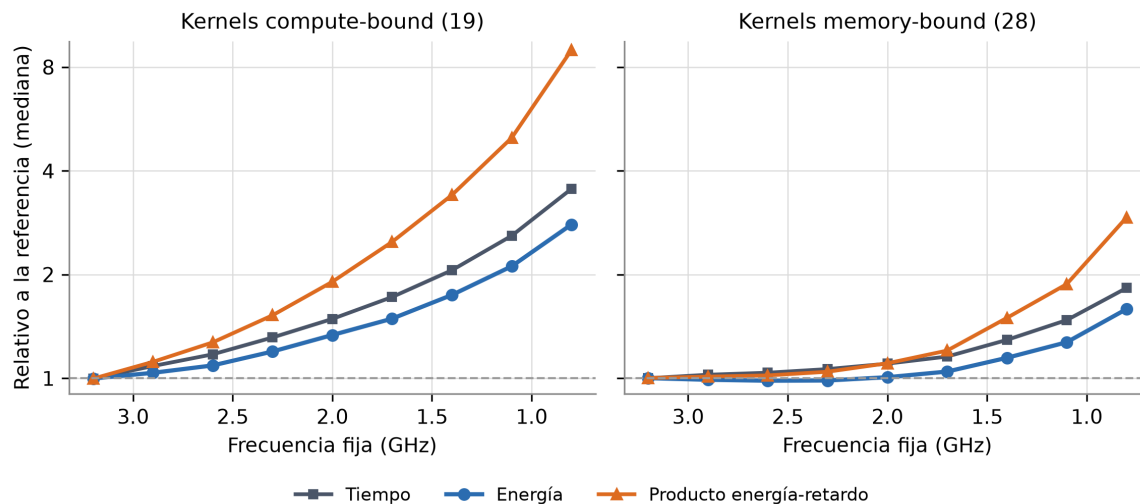


Figura 3.17: Mediana entre kernels de la energía (paquete y DRAM), el tiempo y el producto energía-retardo de cada nivel fijo, relativos a la referencia nativa, para las dos clases. El eje vertical es logarítmico. Fuente: elaboración propia.

El intervalo de confianza *bootstrap* y el valor p de Wilcoxon de la Tabla B.29 responden preguntas distintas y pueden discrepar en apariencia: el primero remuestrea kernels para acotar la ganancia agregada, y el segundo prueba, kernel por kernel, si el signo de la diferencia frente a REF es consistente. Es el caso de F1 y F2 en memory, donde el IC95 excluye el cero pero p no baja de 0.05: con solo 28 kernels y una ganancia heterogénea entre ellos, ambas lecturas son compatibles.

El mecanismo es un piso de potencia de unos 85 W, equivalente al 84 % de la potencia a 3.2 GHz (Figura 3.18). Reducir la frecuencia a 0.8 GHz ahorra solo 15 % de potencia mientras alarga la ejecución. Incluso un oráculo que eligiera el mejor nivel medido por kernel ganaría 2.2 % en promedio; 22 de 47 kernels quedarían por debajo del efecto mínimo de 1 %. Esta cota es optimista porque el mejor nivel también captura ruido de medición.

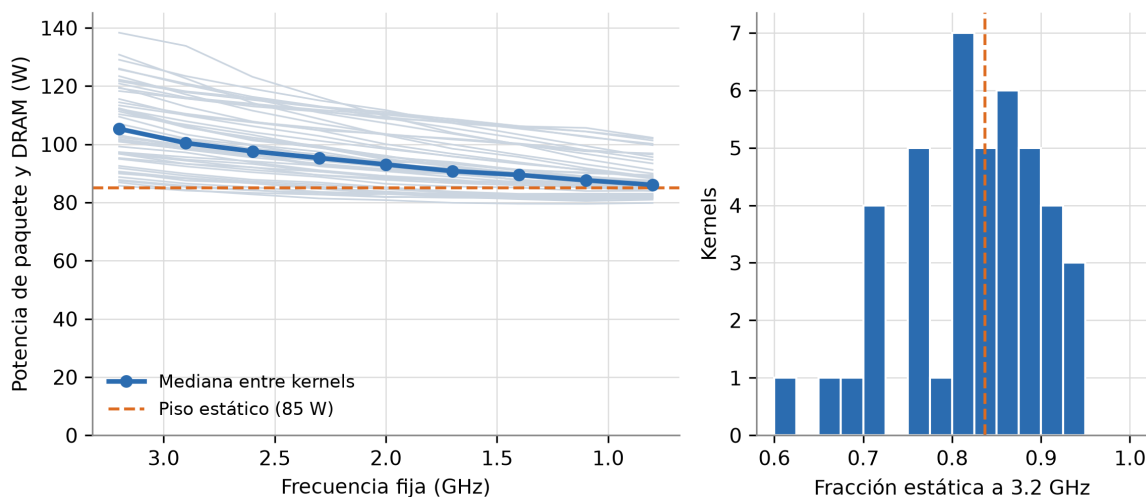


Figura 3.18: A la izquierda, potencia media de paquete y DRAM de cada kernel (líneas grises) y su mediana en cada nivel fijo, con el piso estático ajustado. A la derecha, distribución entre kernels de la fracción de la potencia a 3.2 GHz que corresponde al piso. Fuente: elaboración propia.

Qué tamaño de efecto podía detectar este diseño

Un resultado de *no actuar* solo es informativo si se acompaña del tamaño de efecto que el experimento podía distinguir, porque de lo contrario no separa la ausencia de efecto de la falta de sensibilidad. Tomando la familia algorítmica como unidad, que es la agrupación con la que se valida el clasificador, la desviación estándar del logaritmo del producto energía-retardo entre repeticiones de una misma combinación tiene mediana de 1.05 %, mientras que la dispersión entre las 19 familias *memory_bound* es de 5.35 % en F1 y 9.70 % en F2. Es decir, la varianza que limita la conclusión es heterogeneidad física entre cargas, entre cinco y diez veces mayor que el error de medición; aumentar el número de repeticiones no la habría reducido de forma apreciable.

De esas dispersiones se sigue el efecto mínimo detectable con potencia de 80 %: 1.75 % en F0, 3.38 % en F1, 6.05 % en F2 y 9.5 % en F3. La lectura correcta del resultado es, entonces, que con este catálogo quedan descartadas mejoras uniformes superiores a 3.4 % en F1, y que una mejora del orden del 1 %, el efecto mínimo relevante declarado, es indetectable por construcción con 19 familias. Esto acota el resultado en lugar de dejarlo abierto, y es consistente con el piso de potencia descrito arriba: el margen físico disponible es menor que la resolución que el diseño puede ofrecer.

Dado que la clase agregada mezcla cargas con respuestas de signo opuesto, se evaluó además si una regla más fina, condicionada a una variable del vector de entrada, permitía

actuar sobre una subpoblación. La regla se congeló antes de medir y se probó en familias nuevas, ajenas a las 30 de entrenamiento, en dos pruebas externas independientes; ambas quedaron refutadas según los criterios declarados de antemano. El detalle queda registrado junto a la tabla de política en `policy_cpu.json`. La conclusión operativa de la fase es, por tanto, *no actuar* en ambas clases, sustentada en el piso de potencia medido y no en la ausencia de intentos de refinamiento.

3.1.6. Invarianza del clasificador de CPU frente a la acción del agente

La frecuencia observada es una de las seis variables del clasificador de CPU y el agente la modifica, de modo que se aplicó la misma auditoría contrafactual de la GPU. Sobre el modelo final y sus 1.31 millones de intervalos de 49 kernels, forzar la frecuencia a un valor fijo cambia entre 4 y 11 % de las predicciones (Tabla B.8), de modo que existe una dependencia moderada. Pero el cambio que el agente ejecuta es acotado, de F0 (3.2 GHz) a F1 (2.9 GHz): sobre las 179 774 filas observadas cerca de 3.2 GHz solo cambia el 0.42 % de las predicciones. Con ese espacio de acción la retroalimentación por esta variable es despreciable, y el modelo de CPU se despliega sin modificaciones.

3.2. Resultados de GPU

3.2.1. Validación de control y etiquetado

El actuador de GPU sostuvo el bloqueo del reloj de *streaming multiprocessors* bajo carga. Las pruebas con objetivos de 600 y 1200 MHz registraron, respectivamente, 305 y 174 lecturas activas coincidentes con el valor solicitado. La restauración de frecuencia se verificó al finalizar las campañas dedicadas.

La etiqueta Roofline se construyó con la precisión observada mediante Nsight Compute. Este control excluyó kernels con precisión mixta cuya intensidad operacional no admite comparación con un único techo y evitó atribuir a los contadores convencionales el trabajo ejecutado por Tensor Cores. En el nivel de referencia, los puntos de inflexión medidos fueron 14.965852 FLOP/byte para FP32 y 6.898377 FLOP/byte para FP64.

3.2.2. Diagnóstico del primer conjunto GPU

La primera campaña GPU reunió 540 corridas planificadas para 18 kernels. Después de calibrar el warmup, 334 corridas de 16 kernels y 11 familias quedaron disponibles para el

diagnóstico inicial. El conjunto estaba desbalanceado (Figura 3.19), con 73 corridas *compute_bound* y 261 *memory_bound*, y la mayoría de las familias era homogénea respecto de su etiqueta.

El árbol de profundidad 1 obtuvo un F1 macro de 0.617 bajo validación *leave-one-familia-out*, frente a 0.553 del clasificador mayoritario. Este valor no se adopta como resultado final. La evaluación mostró que, al retirar familias completas, muchos pliegues quedaban sin ejemplos de una clase. Además, la utilización de SM no separó de forma universal los dos regímenes. *rodinia_gaussian*, por ejemplo, fue *memory_bound* según Roofline, pero alcanzó utilizaciones de SM suficientemente altas para que el modelo lo confundiera con una carga *compute_bound*.

La evidencia no indicó que bastara ajustar el modelo sobre el mismo conjunto. Mostró que faltaban familias *compute_bound* variadas y alejadas del punto de inflexión. Por esta razón se cambió la estrategia desde la búsqueda de hiperparámetros hacia la ampliación experimental del catálogo y la revisión del protocolo de validación.

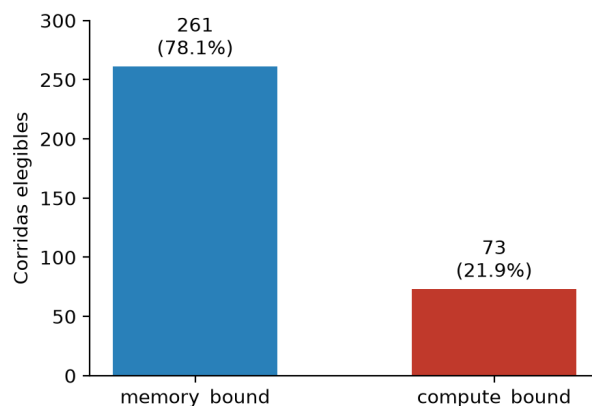


Figura 3.19: Distribución de las 334 corridas elegibles del conjunto GPU empleado en el diagnóstico exploratorio. Fuente: elaboración propia.

3.2.3. Reanálisis de campañas históricas con NVML

El bloqueo de la nueva campaña multi-frecuencia impidió incorporar observaciones nuevas durante buena parte de esta fase. Se reanalizaron entonces campañas ya aceptadas, cuya etiqueta Roofline fue construida fuera de línea con *ncu* y cuya inferencia potencial dependía exclusivamente de NVML. El posproceso comparó esa intensidad con el *ridge* correspondiente a la frecuencia y precisión de cada corrida, y difundió la misma etiqueta a todas sus lecturas NVML. No se registraron eventos CUPTI de lanzamiento en estas campañas, así que cada

corrida se transformó en una sola observación mediante agregados robustos, sin duplicar ni sintetizar observaciones.

Un primer diagnóstico sobre esta representación mostró separación parcial entre clases (Figura 3.20): la proyección PCA conserva la estructura global y t-SNE aproxima los vecindarios locales de ocho variables estáticas (mediana e IQR de utilización GPU, utilización de memoria, potencia y reloj SM). Ambas vistas contienen regiones dominadas por una clase, pero también regiones con etiquetas mezcladas. La separación por clase, calculada en el espacio original estandarizado, tuvo un coeficiente de silueta de 0.092, que describe separación débil. Un clasificador de tres vecinos que nunca observa ejemplos de la familia evaluada alcanzó F1 macro de 0.616, coherente con una señal transferible parcial: existe información física real en los agregados NVML, pero no una frontera estable para todas las familias.

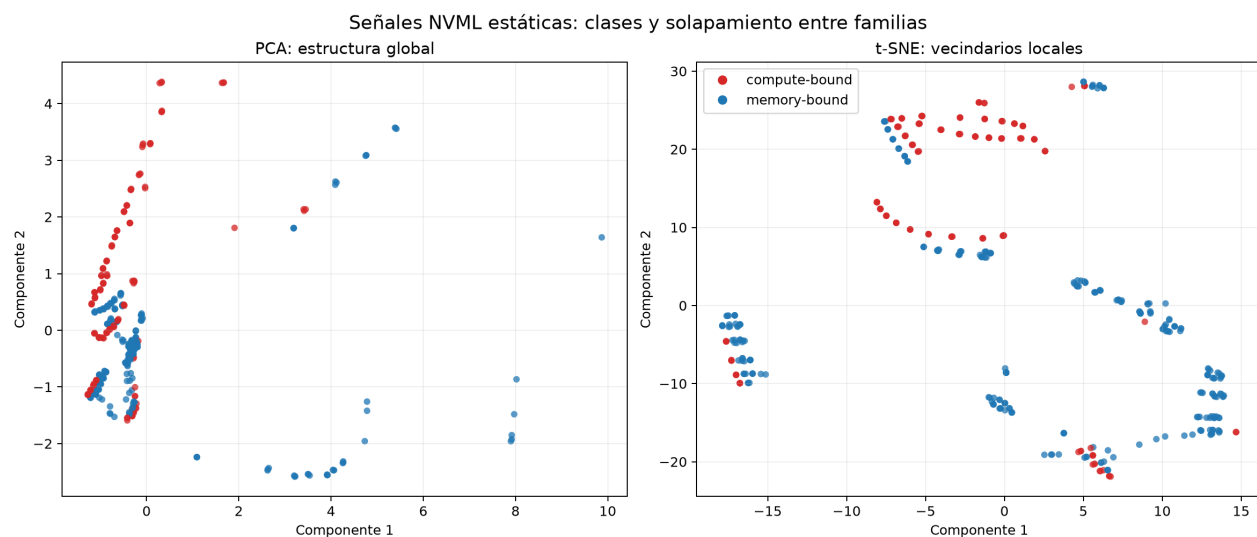


Figura 3.20: Proyecciones PCA y t-SNE de las medianas e IQR de utilización GPU, utilización de memoria, potencia y reloj SM. La proyección t-SNE es diagnóstica y no se utiliza para medir generalización. Fuente: elaboración propia.

A partir de ese diagnóstico se amplió el conjunto fusionando las campañas modernas aceptadas con tres campañas adicionales ya cerradas (*kmeans*, *miniBUDE* y *conv2d*), y se admitieron corridas que el gate de calidad había descartado únicamente por tener una fracción de muestras NVML posteriores al arranque menor al 50 %, cuando el conteo absoluto de muestras seguía siendo alto (cientos a miles por corrida). El conjunto resultante tiene 483 corridas y 16 familias algorítmicas, separando además los tres núcleos CUDA de RAJAPerf (GEMM, *stencil* de calor y Jacobi 2D) que antes compartían una sola familia pese a tener perfiles Roofline distintos entre sí. La composición es de 171 corridas *compute_bound* y 312 *memory_bound*, trece de las dieciséis familias homogéneas respecto de su etiqueta.

La métrica de evaluación pasó a ser la exactitud balanceada sobre celdas familia por clase, el promedio del *recall* de cada combinación observada. Esta métrica está bien definida incluso en familias de una sola clase y no depende de la prevalencia relativa de cada familia, a diferencia del F1 macro agrupado por volumen. La validación usó *leave-one-family-out*, cinco semillas de muestreo y un intervalo de confianza del 95 % por remuestreo *bootstrap* de familias (2000 réplicas). Se compararon siete modelos candidatos con hiperparámetros fijos (Tabla B.9 y Figura 3.21): ninguno domina de forma concluyente. *Random forest* (0.807), *extra trees* (0.806) y la regresión logística (0.792) resultaron estadísticamente indistinguibles entre sí por remuestreo pareado de familias (probabilidad de superioridad entre 0.53 y 0.60), aunque *random forest* sí superó con evidencia razonable a XGBoost (probabilidad de superioridad 0.961).

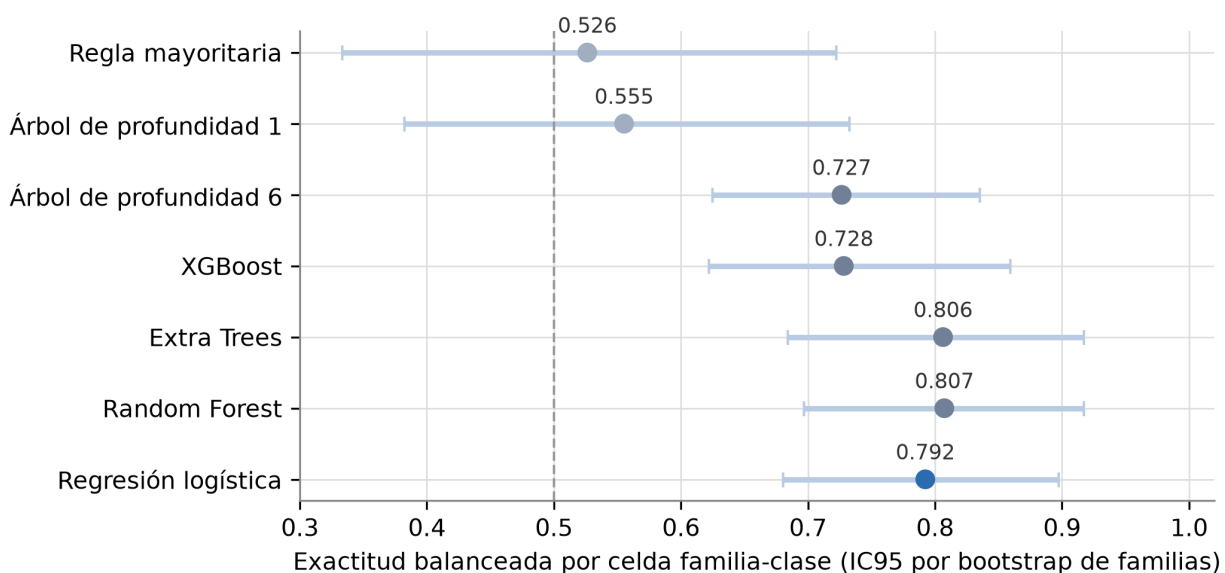


Figura 3.21: Exactitud balanceada por celda familia-clase de los siete modelos candidatos, con intervalo de confianza del 95 % por *bootstrap* de familias. Fuente: elaboración propia.

Las cinco variables del conjunto candidato (Tabla B.10) no presentan colinealidad relevante: el factor de inflación de varianza (VIF) máximo fue 2.26, muy por debajo del umbral convencional de 5 que suele usarse como señal de alerta.

Una búsqueda anidada de hiperparámetros con muestreador TPE (*Optuna*), quince pruebas por modelo dentro de un *leave-one-family-out* interno, empeoró el resultado en cuatro de los cinco modelos ajustables respecto de la configuración fija (regresión logística de 0.792 a 0.767, *random forest* de 0.807 a 0.778, *extra trees* de 0.806 a 0.748, XGBoost de 0.740 a 0.724; solo el árbol de profundidad 6 mejoró, de 0.727 a 0.783). El mismo mecanismo do-

cumentado para el clasificador de fase de CPU explica esta caída: con familias limitadas, el criterio interno que Optuna maximiza no discrimina de forma confiable la configuración que generaliza mejor. La búsqueda de hiperparámetros se descarta para el candidato GPU por la misma razón que para CPU.

Con el empate entre *random forest*, *extra trees* y regresión logística ya establecido, el criterio de desempate para la referencia estática fue la latencia de inferencia de una sola fila, medida en **paccaA100**, el nodo de destino real. La regresión logística respondió en 400.9, 413.8 y 466.9 microsegundos (percentiles 50, 95 y 99); *random forest* y *extra trees* tardaron entre diez y doce veces más (percentil 99 de 5512.2 y 4989.9 microsegundos, respectivamente). La regresión logística usa las cuatro señales NVML, utilización GPU, utilización de memoria, potencia y reloj SM, todas por mediana, más la desviación estándar intracorrida de la utilización de memoria como quinta variable. Esta última se incorporó porque las medianas por sí solas diluyen el efecto de cargas con lanzamientos cortos y repetidos, como *kmeans*, cuya intensidad operacional real es alta pero cuya firma NVML agregada se asemeja a una carga poco ocupada.

Dos familias no se resuelven con ninguna de las variables disponibles por una razón física, no por una limitación del modelo: el núcleo GEMM de RAJAPerf y *LUD* de Rodinia cambian de régimen Roofline según el nivel de frecuencia de la corrida, porque el punto de inflexión se desplaza con el reloj mientras la intensidad operacional de cada kernel permanece fija. *LUD*, por ejemplo, resultó *memory_bound* en los niveles de reloj más altos y *compute_bound* en los más bajos de la misma familia. Distinguir ese cambio exigiría una variable de distancia al *ridge*, que constituye fuga de la propia etiqueta y por tanto no es admisible como entrada del clasificador.

La Figura 3.22 desagrega el desempeño de la regresión logística estática por familia. El resultado no es parejo: ocho de las dieciséis familias superan 0.90 de exactitud, mientras *dual_cholesky* (0.23), el núcleo GEMM de RAJAPerf (0.50, la familia con cruce de *ridge* ya descrita) y *rodinia_gaussian* (0.53, la familia más cercana al *ridge* del catálogo) quedan por debajo del umbral trivial de 0.5. La matriz de confusión agregada (Figura 3.24, panel izquierdo) resume esto en 483 corridas: 125 de 171 *compute_bound* (73.1%) y 247 de 312 *memory_bound* (79.2%) correctamente clasificadas.

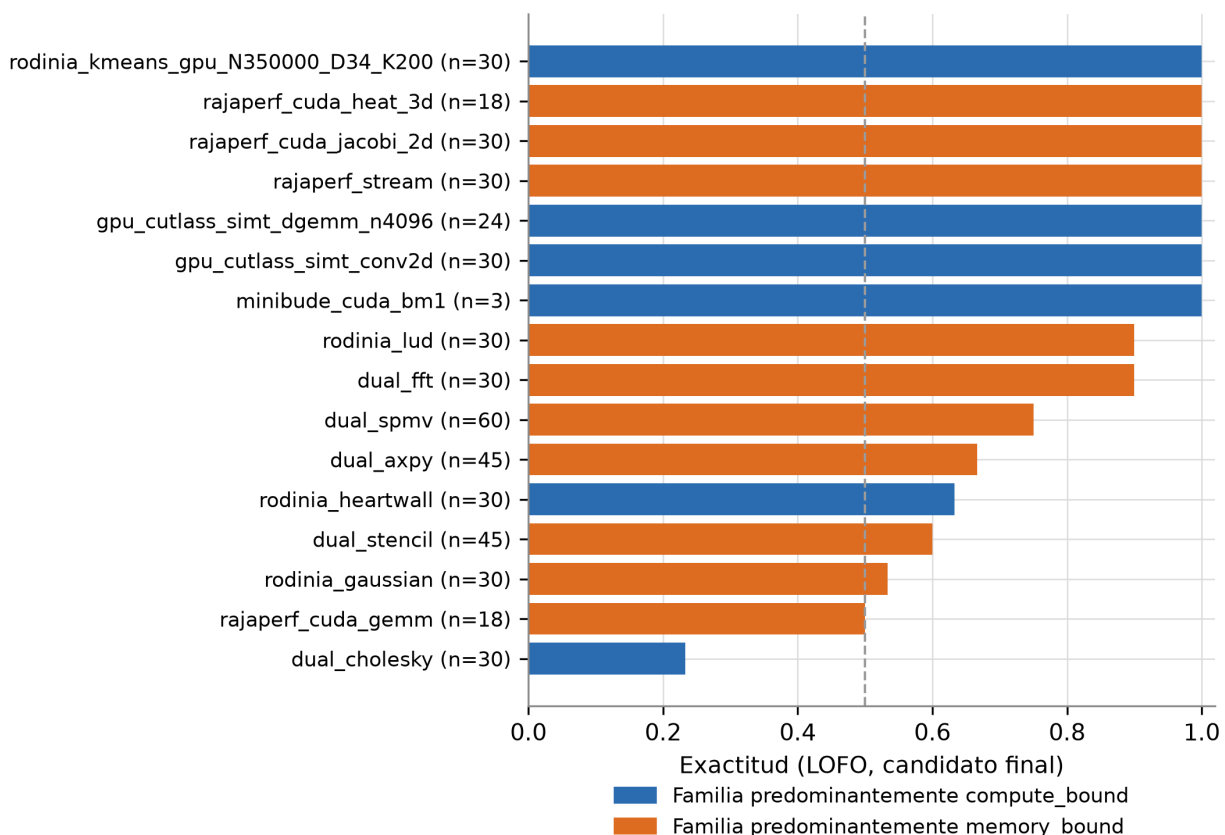


Figura 3.22: Exactitud de la regresión logística de cinco entradas por familia algorítmica, LOFO. El color indica la clase predominante real de cada familia. Fuente: elaboración propia.

La calibración de probabilidades de la regresión logística de cinco variables, usada aquí solo como candidato estático de referencia, tiene un error de calibración esperado (ECE, calculado sobre diez rangos de confianza de igual amplitud) de 0.057. Su umbral de 0.70 sube la exactitud balanceada por celda de 0.792 a 0.846 reteniendo el 76.7 % de las corridas (Figura 3.23). La selección del modelo desplegable se reporta por separado después de aplicar el contrato de entradas invariantes.

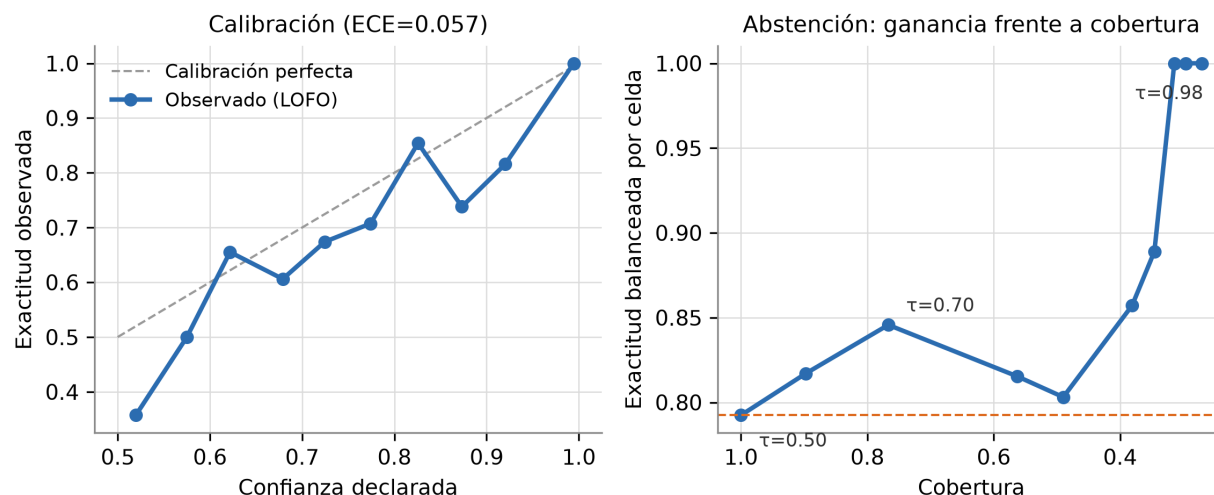


Figura 3.23: Calibración y abstención de la regresión logística de cinco entradas, candidata estática que no se despliega en el agente. Izquierda, confianza declarada contra exactitud observada, agrupada en diez rangos de igual amplitud. Derecha, exactitud balanceada por celda contra cobertura para distintos umbrales de decisión τ . Fuente: elaboración propia.

La regresión logística estática ya congelada, con sus cinco entradas, se evaluó una vez contra cuatro familias de Rodinia de una campaña de agosto (Backprop, DWT2D, LavaMD y Myocyte) que no participaron en ninguna decisión de esta ronda: ni en el umbral de aceptación de corridas, ni en la separación de familias de RAJAPerf, ni en la variable de variabilidad añadida, ni en la elección del modelo. Es el equivalente honesto a un conjunto de prueba sellado para esa referencia. Sobre 140 corridas, la exactitud global fue 0.779 (Figura 3.24, panel derecho), con *LavaMD* como la familia más débil (0.5) y las otras tres entre 0.81 y 0.97. Esta cifra es consistente con una validación externa independiente reportada en una versión anterior de este análisis sobre un conjunto ligeramente distinto (exactitud 0.743, misma familia más débil), lo que respalda que el resultado no es un artefacto de la partición específica de esta ronda.

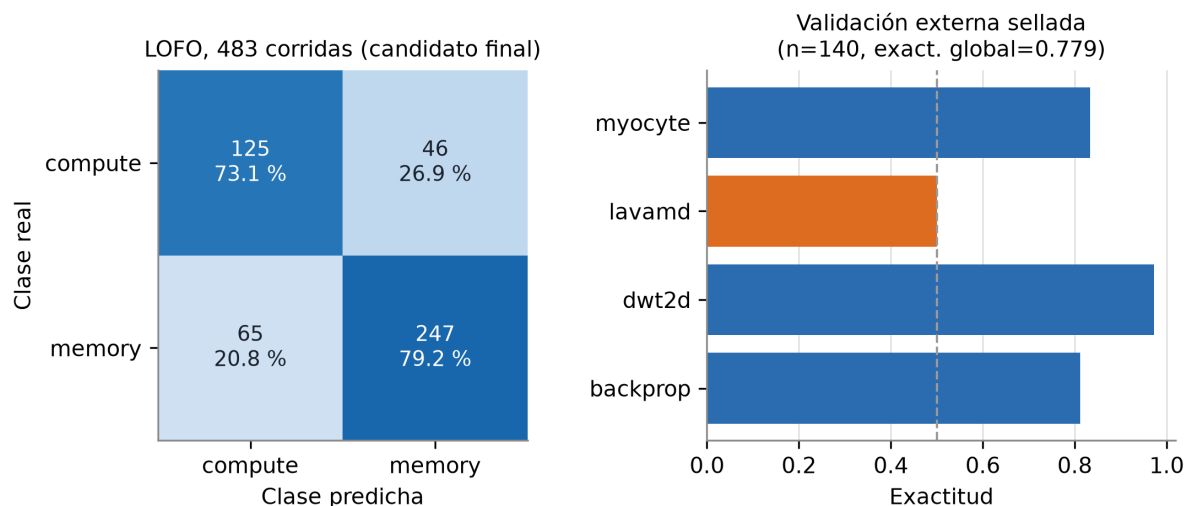


Figura 3.24: Izquierda, matriz de confusión agregada de la regresión logística de cinco entradas bajo *leave-one-family-out* (483 corridas). Derecha, exactitud por familia en la validación externa sellada (Backprop, DWT2D, LavaMD, Myocyte; 140 corridas nunca usadas en ninguna decisión de esta ronda). Fuente: elaboración propia.

3.2.4. Candidato operativo de GPU: invarianza frente a la acción del agente

La regresión logística de la subsección anterior es la referencia estática más rápida, pero el agente exige además que sus entradas no dependan del reloj que él mismo fija (sección 2.3.4). La auditoría, sobre las mismas 483 corridas y sin mediciones nuevas, muestra que ese requisito cambia el modelo seleccionado (Figura 3.25). Al fijar de forma contrafactual el reloj de cada observación en 1410 MHz, la regresión logística cambia de clase el 26.7% de sus predicciones, y su exactitud baja de 0.792 a 0.654 al excluir el reloj. El *random forest* conserva el desempeño al retirar las entradas afectadas por la actuación, con 0.807 usando cinco variables, 0.812 sin reloj y 0.819 sin reloj ni potencia.

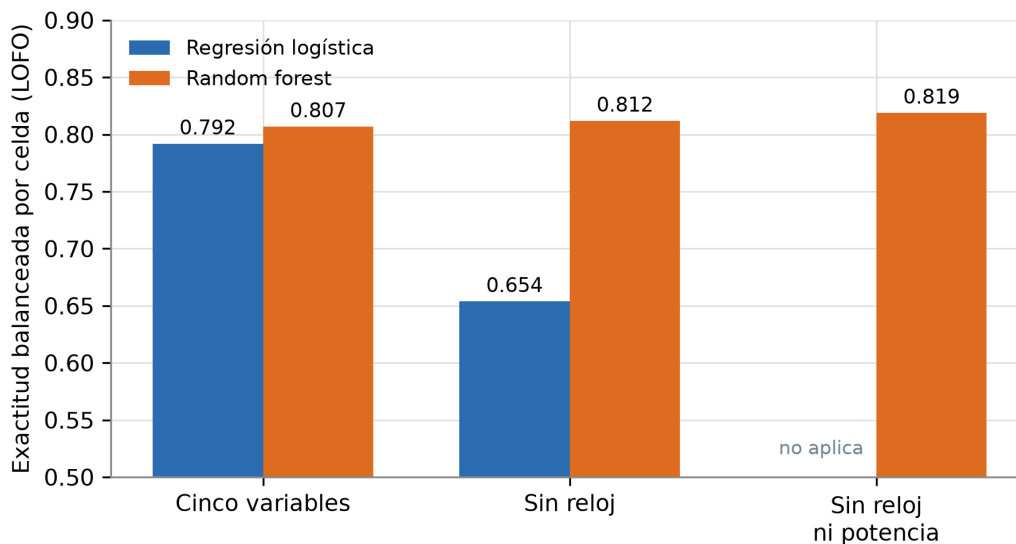


Figura 3.25: Exactitud balanceada por celda (LOFO, 483 corridas, 16 familias) al excluir las variables que dependen de la acción del agente. Fuente: elaboración propia.

El daemon consume el *random forest* de tres entradas invariantes de la Tabla 3.4. Su ECE LOFO es 0.0868 y su inferencia mediana tarda 4.7 ms, frente a los segundos requeridos para detectar y actuar sobre una fase. Con el umbral de 0.90 alcanza 0.900 de exactitud balanceada sobre el 65.7% de las corridas aceptadas. El agente decide una sola vez por fase, por lo que una abstención no se reevalúa.

Tabla 3.4: Modelos GPU después de aplicar el contrato de entradas del daemon. Las métricas se obtienen con LOFO sobre 483 corridas y 16 familias.

Modelo	Entradas	Exactitud balanceada	ECE	Estado
Regresión logística	Cinco señales NVML, incluidas potencia y reloj SM	0.792	0.0569	Referencia estática
<i>Random forest</i>	Utilización GPU, utilización de memoria y desviación estándar de esta última	0.819	0.0868	Desplegado

3.2.5. Política de frecuencia por clase en GPU

La tabla clase-frecuencia se derivó sobre las mismas 483 corridas de la sección anterior, con el kernel como unidad de comparación y el procedimiento de la sección A.3.3: mediana de repeticiones por kernel y nivel, prueba pareada contra REF e intervalo de confianza del 95 % por *bootstrap*. Un cálculo preliminar emparejó en cambio, kernel por kernel, sus tres repeticiones de cada nivel contra sus tres de REF ($n = 3$ por prueba); se descartó por responder una pregunta distinta de la que decide la tabla — si el ruido de medición dentro de un kernel es distinguible de cero, no si el efecto generaliza entre kernels — y se sustituye aquí por el resultado con el kernel, y su agregación por familia, como unidad.

De los 19 kernels con corridas aceptadas, uno (`minibude_cuda_bm1`) queda fuera por no tener más que una repetición en un solo nivel. De los 18 restantes, cuatro — el núcleo GEMM y el *stencil* de calor de RAJAPerf, y las variantes de menor tamaño de `dual_axpy` y `dual_stencil` — quedan fuera de la prueba por no tener corridas en REF bajo la rejilla disponible, no por decisión editorial: sin REF no hay con qué parrear. Restan 5 kernels *compute_bound* y 9 *memory_bound*, de los cuales tres pares (`dual_axpy`, `dual_spmv` y `dual_stencil`, en su variante con REF) son el mismo algoritmo en dos tamaños de problema; la Tabla 3.5 agrega esos pares dentro de su familia (8 familias *memory_bound*, 5 *compute_bound*), con validación *leave-one-familia-out* de la regla de nivel elegido.

Tabla 3.5: Síntesis de la política GPU. Las ganancias son de EDP frente a REF; la rejilla completa se conserva en la Tabla B.30 del Anexo B.

Clase	Evidencia decisiva	Política
Cómputo	F1: +2,1 % (IC95 $[-12,7; 14,2]$, $p = 0,438$); sin mejora confiable	No actuar
Memoria	F1: +8,9 % (IC95 $[3,0; 13,3]$, $p = 0,020$); siete de ocho familias mejoran	Fijar F1

El resultado es **no actuar** en *compute_bound* y **actuar en F1** para *memory_bound* (Figura 3.26), con una ganancia agregada de 8.9 % (IC95 de 3.0 a 13.3 %, $p = 0,020$, 7 de 8 familias mejoran). La validación *leave-one-familia-out* — aplicar la regla “elegir F1” a cada familia excluida, usando solo las siete restantes para elegirla — confirma la dirección: siete de ocho pliegues eligen F1 de forma independiente y la ganancia realizada fuera de muestra es 6.5 % (IC95 de $-4,0$ a 13.1 %, un intervalo que roza cero por tratarse de solo ocho familias, pero consistente en signo con el resultado de la Tabla 3.5). F2 muestra una ganancia de magnitud similar (7.4 %) pero no alcanza significancia con la familia como unidad, a diferencia

del resultado preliminar por kernel: la diferencia es exactamente el efecto de dejar de contar dos veces los mismos algoritmos en dos tamaños.

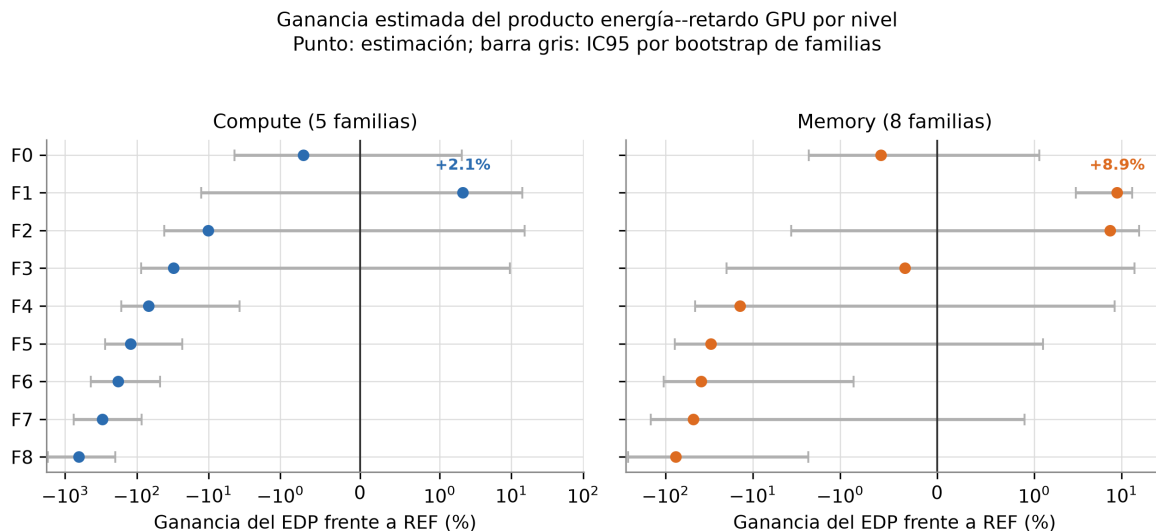


Figura 3.26: La rejilla completa de la Tabla B.30, en forma gráfica, con la familia como unidad. Cada punto es la ganancia estimada y la barra gris, su IC95 por *bootstrap*; los rótulos señalan F1. El eje horizontal es simétrico-logarítmico para mostrar a la vez F1, cercano a cero, y F8, con pérdidas de más de un orden de magnitud en *compute_bound*. Fuente: elaboración propia.

En *compute_bound*, F1 parece favorable en el promedio descriptivo (+2.1%), pero su IC95 va de -12,7% a +14,2% y la prueba pareada da $p = 0,438$. No constituye una mejora confiable. Además, la validación LOFO de la regla que escoge F1 produce una ganancia realizada de -4,8% (IC95 de -13,4 a 0%): al aplicarla a una familia que no participó en elegir el nivel, empeora en promedio. Por ello el daemon conserva REF en *compute_bound*; la conclusión coincide con la de CPU.

La pendiente mediana de sensibilidad del tiempo al reloj SM es 0.837 en cómputo ($n = 6$) y 0.189 en memoria ($n = 12$). Por ello, en memoria puede bajarse el reloj de cómputo sin prolongar proporcionalmente la espera por el dominio de memoria, que no se varió en la campaña. La excepción es *rodinia_gaussian*, etiquetada como memoria pero con una pendiente de 0.941 (véase el desglose en el Anexo B).

La Figura 3.27 hace visible el mismo contraste: en *compute_bound* el tiempo y el EDP crecen juntos desde F1, porque la energía apenas cae; en *memory_bound* el tiempo crece mucho más lentamente y la energía sí cae de forma sostenida hasta F3, de modo que el EDP se mantiene por debajo de 1 en F1–F2 antes de que la caída de energía se agote y el tiempo tome el control a partir de F5. Ajustando $P(f) = P_0 + c f^k$ a la potencia de cada kernel frente

al reloj SM — descartando seis de los 18 kernels cuyo rango de reloj disponible (menos de siete niveles con datos) o cuyo ajuste resultante ($R^2 < 0,8$) no identifica los tres parámetros de forma confiable —, el piso estático P_0 resulta de 38.5 W en mediana sobre los 12 kernels restantes, y representa el 55.8 % de la potencia al reloj más alto (Figura 3.28): un margen de potencia dinámica más amplio, proporcionalmente, que el 84 % estático de CPU, consistente con que *memory_bound* sí encuentre una región de mejora que CPU no encontró en ninguna clase.

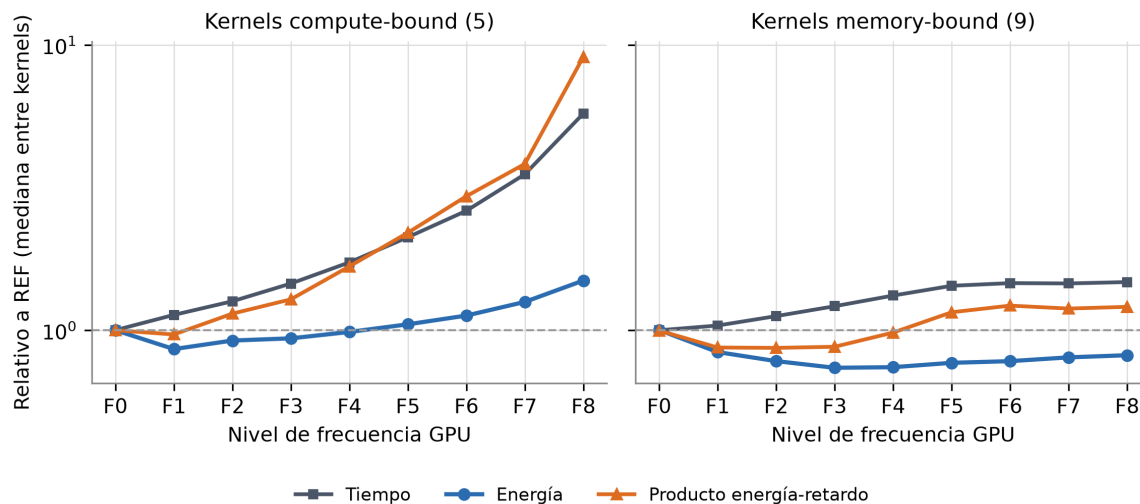


Figura 3.27: Mediana entre kernels del tiempo, la energía y el producto energía-retardo de cada nivel, relativos a REF, para las dos clases (5 y 9 kernels con corridas en REF, kernel como unidad, descriptivo; la significancia se evalúa por familia en la Tabla 3.5). El eje vertical es logarítmico. Fuente: elaboración propia.

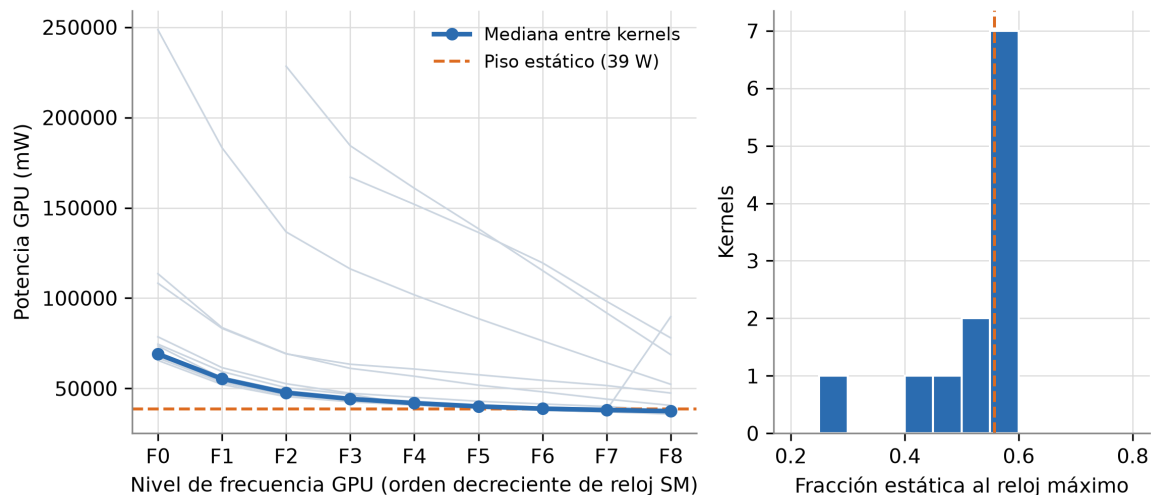


Figura 3.28: Izquierda: potencia media de GPU de cada kernel (líneas grises) y su mediana en cada nivel, con el piso estático ajustado (12 kernels con ajuste identificable). Derecha: distribución entre esos kernels de la fracción de la potencia al reloj más alto que corresponde al piso. Fuente: elaboración propia.

En la validación por familia retirada, siete de ocho familias de memoria realizaron ganancias de 4.2 a 14.8%; `rodinia_gaussian` perdió 27.3 %. Su tiempo responde al reloj como una carga de cómputo pese a su etiqueta de memoria. Se conservó en el análisis: excluirla después de observar la pérdida sesgaría la política.

3.2.6. Continuidad de la ruta CUPTI

La limitación observada en el reanálisis no invalida el uso de NVML como proxy ligero, pero delimita lo que puede esperarse de él. NVML mide efectos agregados de toda la GPU y no expone FLOPs, bytes movidos, ocupación ni la identidad del kernel activo en cada instante. Un sondeo más frecuente produce más lecturas del mismo estado agregado, no reconstruye los límites de los lanzamientos. Por ello se implementó una vía independiente, basada en la API de actividades de CUPTI, para ubicar en el tiempo real cada lanzamiento CUDA de una corrida y combinarlo con un modelo analítico de trabajo por lanzamiento [29]. La biblioteca de captura se inyecta en el proceso CUDA medido, no en el proceso recolector, porque el lanzador separa ambos con `fork+exec`. Una auditoría de cobertura, de tipo *fail-closed*, exige que todo kernel GPU del catálogo tenga registrado un modelo analítico de FLOPs y bytes por lanzamiento antes de permitir que una campaña con esta traza comience; diecinueve kernels del catálogo lo tienen.

Con esa infraestructura se ejecutó una campaña sobre el catálogo GPU completo, 544

corridas planificadas. La captura de la traza fue correcta: la corrida de *rodinia_gaussian* registró 8197 eventos CUPTI, consistentes con los aproximadamente 8190 lanzamientos esperados de **Fan1**/**Fan2** para una eliminación gaussiana de tamaño 4096. El mecanismo de captura por lanzamiento funciona como fue diseñado.

La campaña CUPTI capturó los lanzamientos, pero su eje de frecuencia no fue válido: con solicitudes de 700 y 1200 MHz, el reloj observado permaneció en 765 MHz. Ninguna ventana cumplió la tolerancia de reloj para entrenar por nivel. Se conservaron las trazas y calibraciones; una verificación posterior volvió a encontrar operativo el candado (sección 3.3.1). Por tanto, el fallo afecta esa campaña, no la política GPU derivada de las 483 corridas previas.

CUPTI mantiene dos funciones futuras diferenciadas, independientes del resultado anterior. Su perfilado de contadores puede construir una intensidad operacional y una etiqueta Roofline por kernel en una campaña controlada. Su API de actividades, ya implementada, puede registrar los límites temporales de lanzamientos y transferencias necesarios para asociar esa etiqueta con el segmento NVML de la misma fase. La granularidad fina requiere ambas fuentes en la misma campaña: sin el perfilado no se conoce la clase física de cada fase, y sin la traza no se sabe qué parte de la telemetría corresponde a esa fase. Ninguna de estas funciones convierte automáticamente a CUPTI en una entrada del clasificador en producción, que debe conservar únicamente señales ligeras de NVML. La reconstrucción del eje de frecuencia de esa campaña y una nueva evaluación de etiquetas por fase con CUPTI quedan como extensiones; la política de frecuencia GPU de este trabajo se obtuvo de la campaña anterior válida.

3.3. Resultados del agente de control

Esta sección reporta la verificación del agente de la Fase 3 en **paccaA100**, antes de la evaluación de la Fase 4. Todas las pruebas usaron el nodo en asignación exclusiva y restauraron el estado de frecuencia al terminar.

3.3.1. Candado de reloj de GPU y premisa física de la política

Con una carga sintética de fases controladas se comprobó que, al fijar el reloj con `nvidia-smi -lgc`, el reloj observado coincide con el solicitado en los seis valores probados (1410, 1260, 1110, 810, 510 y 210 MHz), y que el rendimiento responde con la forma que supone la política (Figura 3.29): el de una carga *compute_bound* escala aproximadamente con el reloj, mientras que el de una carga *memory_bound* no cae hasta llegar a 510 MHz.

Es la premisa física de bajar el reloj en la clase *memory_bound* sin costo de tiempo, medida sobre el mismo instrumento que aplicará el agente.

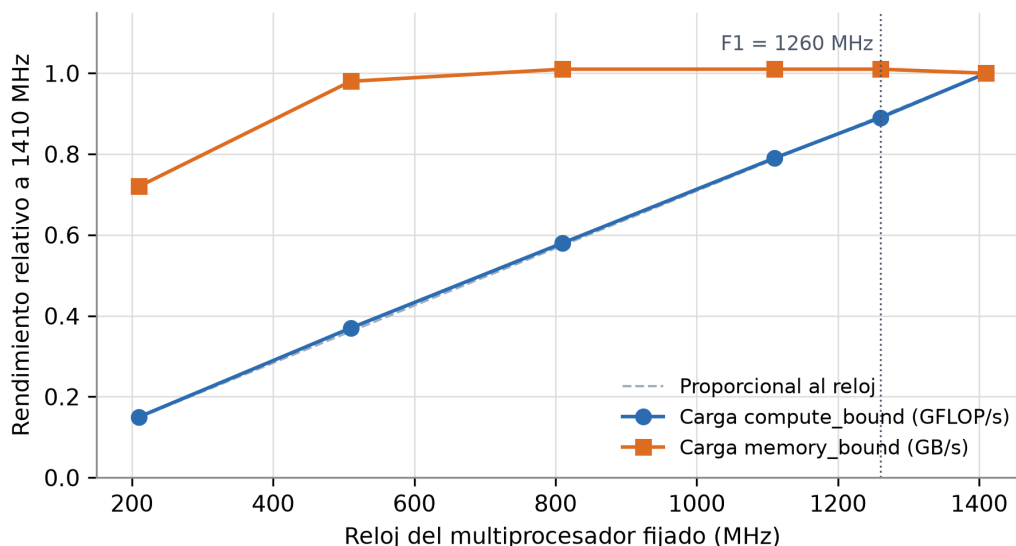


Figura 3.29: Rendimiento relativo al reloj de 1410 MHz con el reloj de GPU fijado, para una carga *compute_bound* y una *memory_bound*; la línea discontinua es la proporcionalidad exacta al reloj. Fuente: elaboración propia.

3.3.2. Latencias de conmutación y sobrecarga

La Tabla 3.6 resume las latencias de conmutación y la sobrecarga de cada proceso. La permanencia mínima entre cambios de reloj en GPU se fijó en 10 veces la latencia de conmutación observada, es decir, 3.7 s. La sobrecarga en energía de paquete del brazo *sombra* frente al *base* se mide en la Fase 4.

Tabla 3.6: Latencias de conmutación y sobrecarga del agente en **paccaA100**.

Dispositivo	Magnitud	Valor
GPU	Comando de fijado de reloj	≈ 50 ms
GPU	Reloj observado en el valor pedido (tras el comando)	70 a 80 ms
GPU	Uso de CPU del proceso con la GPU en reposo	0.0 a 0.1 % de un núcleo
CPU	Escritura secuencial de límites en los núcleos delegados	27.9 ms
CPU	Escritura en paralelo, un hilo por núcleo	2.85 ms
CPU	Inferencia del clasificador (percentil 50 / 99)	16.4 / 19.3 μ s

3.3.3. Restauración bajo terminación anómala y coordinación

La Tabla B.28 resume las pruebas de restauración y de coordinación. En cada prueba de terminación anómala inducida (SIGTERM o SIGINT) el agente atendió la señal y devolvió el dispositivo a su estado inicial, con código de salida 0. La prueba de coexistencia ejecutó ambos procesos en brazo activo sobre la aplicación compuesta A de GPU con el piso de CPU en 3.2 GHz.

3.3.4. Detección y clasificación de fase de GPU sobre aplicaciones compuestas

La Tabla B.11 reporta la clasificación de fase de GPU del agente sobre las dos aplicaciones compuestas, con el umbral de abstención de 0.90. En la aplicación A, las decisiones correctas tuvieron confianza 1.00 y las abstenciones entre 0.57 y 0.67; en la aplicación B, *LavaMD*, una familia inédita, se clasificó correctamente con confianza 1.00. Los conteos son pequeños y sirven para verificar el funcionamiento del detector y de la abstención; la exactitud con muestra suficiente es lo que mide la Fase 4.

3.4. Evaluación del agente frente a REF (Fase 4)

Esta sección reporta la evaluación experimental del agente frente al *governor* nativo con turbo (REF) en *paccaA100*, según la matriz de la sección 2.5. Se reportan la matriz inicial (escenario A), sus repeticiones de comprobación, los escenarios C, D y E, el confirmatorio de E-A, CloverLeaf y la línea base secundaria sin turbo. Las razones de las tablas se calculan sobre la mediana de cada brazo y se expresan frente a la mediana de REF de la misma combinación experimental; un valor menor que 1 es una mejora. Cuando el diseño usa bloques, también se reportan las razones emparejadas por bloque.

3.4.1. Aplicaciones compuestas

Cada alcance usa dos aplicaciones compuestas que encadenan kernels del catálogo, con la clase de cada fase conocida de antemano. La aplicación A reúne familias vistas durante el entrenamiento y la B familias inéditas. La Figura 3.30 muestra la estructura de un ciclo y la Tabla B.12 el detalle por kernel. La fracción de tiempo en fases *memory_bound* varía entre 15 % (CPU, B) y 71 % (CPU, A), y es la magnitud que acota cuánto puede ahorrar cualquier política que solo actúe en esa clase.

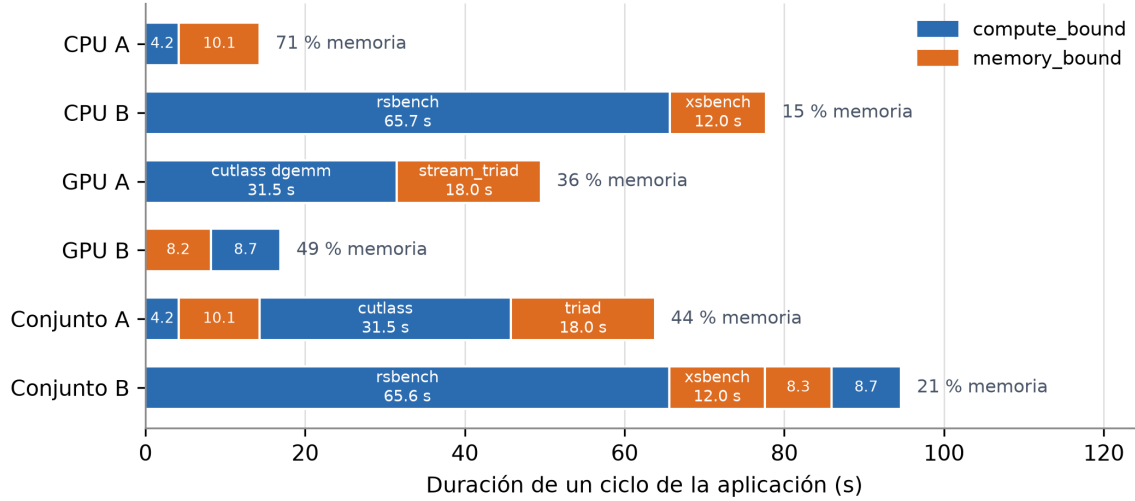


Figura 3.30: Estructura de un ciclo de cada aplicación compuesta. Los números dentro de las barras son la duración media de cada fase en segundos; a la derecha, la fracción del ciclo en fases *memory_bound*.

3.4.2. Clasificación de fase con el agente en el lazo

La Tabla B.13 puntúa cada decisión del agente contra la fase real en que cae su instante. En CPU las decisiones son por ventana de un milisegundo, de modo que hay decenas de miles por celda y la exactitud está ponderada por la duración de cada fase; en GPU hay una decisión por fase, con una permanencia mínima de 3.7 s.

En CPU, el agente acierta el 94 % de las decisiones sobre las familias vistas y más del 99.7 % sobre las inéditas. La diferencia favorable a las inéditas no debe leerse como mejor generalización: en la aplicación B una sola fase de cómputo ocupa el 85 % del tiempo y ponderar por ventanas la hace dominante, de modo que el resultado depende de cómo se separan esos dos kernels y no de familias arbitrarias. En GPU no hubo ninguna decisión errónea entre las que el agente tomó, pero se abstuvo con frecuencia: una de doce fases en A y entre cuatro y seis de doce en B (en el alcance conjunto, entre tres y siete de veinte). Toda fase, en cualquier alcance, recibió al menos una decisión. La exactitud del brazo activo y la del brazo sombra son indistinguibles, como corresponde a un clasificador cuya entrada no depende de la acción del agente.

3.4.3. Tiempo, energía y EDP frente a REF

La Tabla B.14 y la Figura 3.31 reportan la duración, las energías de CPU y de GPU por separado y las razones frente a REF de la matriz inicial. Tres hechos separan los alcances:

- En **GPU** el agente activo no cambia el resultado: el EDP del nodo queda en 1.00 frente a REF en A y en B (y el brazo sombra, en 1.01 y 1.00). El agente de GPU no impone sobrecarga medible.
- En **CPU** el brazo activo queda entre 1.7 y 2.0 veces peor que REF en EDP, con un aumento de 15 a 25 % en tiempo y de 55 a 63 % en energía de CPU.
- En el alcance **conjunto** el EDP queda entre 1.3 (A) y 1.6 (B) veces el de REF. Con cinco repeticiones por brazo, la prueba de Mann–Whitney da $p = 0,0079$ en todas las comparaciones contra REF, que es el mínimo alcanzable con cinco contra cinco: las cinco corridas de cada brazo quedaron por encima de las de REF.

La descomposición por brazo indica dónde se origina la diferencia. El brazo sombra, que decide pero no escribe ninguna frecuencia, ya presenta prácticamente todo el exceso de CPU. La Tabla B.15 compara el brazo activo con el sombra: el efecto atribuible a la acción de frecuencia está entre $-3,3\%$ y $+2,9\%$ del EDP, es decir, de pocos puntos y de signo mixto (peor en CPU con kernels vistos, mejor con los inéditos y en el alcance conjunto). Poner F0 también en *memory* (*activo-F0*) o quitar el piso de coordinación (*activo sin piso*) no cambia el resultado de forma apreciable.

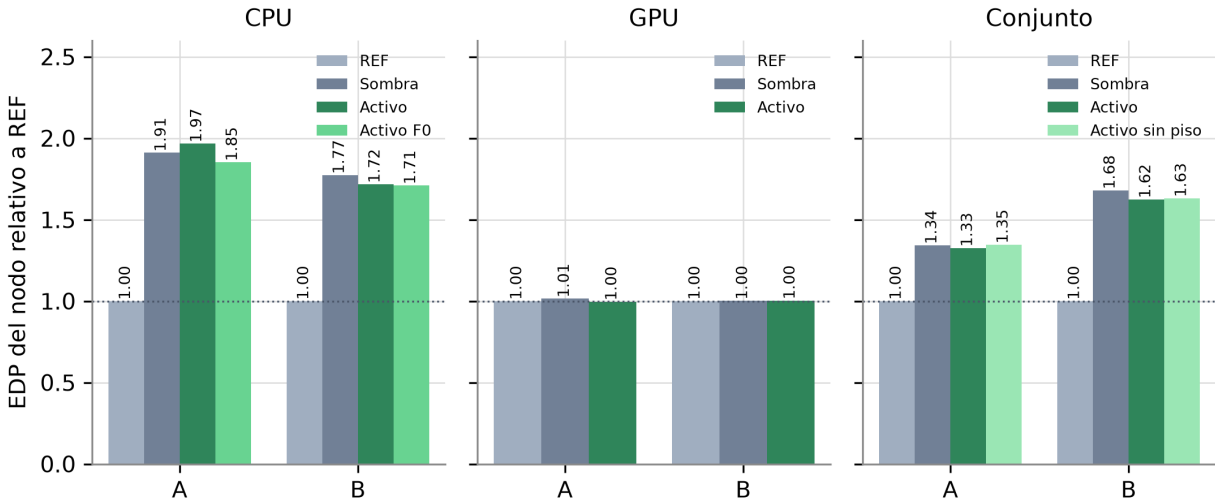


Figura 3.31: EDP del nodo, $(E_{\text{CPU}} + E_{\text{GPU}})T$, relativo a REF, por alcance, aplicación (A: vistos, B: inéditos) y brazo, en la matriz inicial. La línea punteada marca la paridad con REF.

3.4.4. Costo del agente de CPU y configuración de la inferencia

La Figura 3.32 desglosa el costo del agente. El brazo sombra permite medir el costo de mantener el agente en funcionamiento sin el efecto de la frecuencia. La causa del exceso de CPU no es el ritmo de muestreo: pasar de 1 a 10 ms reduce unas diez veces el número de decisiones y el costo no cambia (Tabla B.16). Depende de la configuración del motor de inferencia. Con el conjunto de hilos por defecto, el motor de inferencia crea un hilo por núcleo que espera de forma activa entre inferencias y compite con la aplicación, que está fijada a los núcleos 0 a 3. Un único hilo de inferencia sin espera activa elimina la diferencia de tiempo (1.00 frente a REF) y reduce el sobre costo de energía de CPU de 66 % a 6 %, y sube la exactitud de clasificación a 0.991; una explicación plausible, no verificada, es que las ventanas de contadores salen menos perturbadas cuando el agente no compite con la aplicación. Fijar el proceso consumidor a un núcleo no tiene efecto por sí solo. El agente de CPU se define, por tanto, con un hilo de inferencia; el agente de GPU no presentó este costo.

El sobre costo de energía de CPU que queda con un hilo de inferencia no depende del periodo de muestreo: con kernels vistos, el brazo sombra queda en 1.086, 1.058 y 1.066 veces la energía de CPU de REF con periodos de 1, 10 y 100 ms (tres repeticiones cada uno), con exactitud de 97.6, 99.4 y 98.4 %. Es un costo fijo de mantener el agente, del orden de 6 %. La comparación completa del brazo activo con el agente definido con un hilo de inferencia es el escenario C (sección 3.4.6).

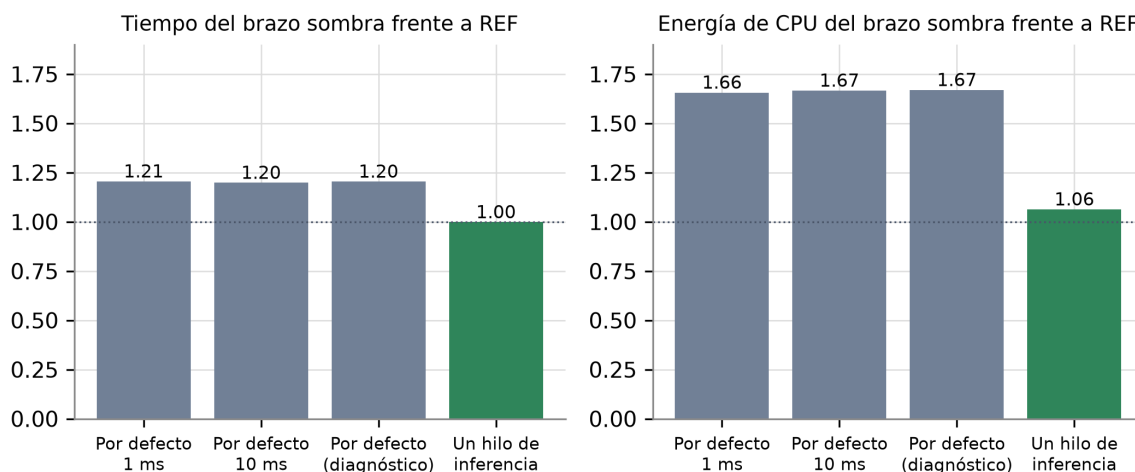


Figura 3.32: Costo del brazo sombra frente a REF en CPU con kernels vistos (A), por configuración de la inferencia y periodo de muestreo.

3.4.5. Potencial de la GPU

La ganancia máxima que la política de GPU puede aportar está acotada por la Fase 2: fijar el reloj en F1 (1260 MHz) durante fases *memory_bound* mejora el EDP en 8.9 % (IC95 3.0 a 13.3 %), y los niveles más bajos no mejoran o empeoran (Figura 3.33, izquierda). El efecto sobre una aplicación completa se reduce por la fracción de su tiempo en memoria. Con 36 % (A) y 49 % (B) de fases *memory_bound*, la ganancia esperada del nodo es de 3.2 % y 4.3 %. La ganancia medida fue de 0.4 % en A y de -0.3 % en B (Figura 3.33, derecha), indistinguibles de cero con tres repeticiones.

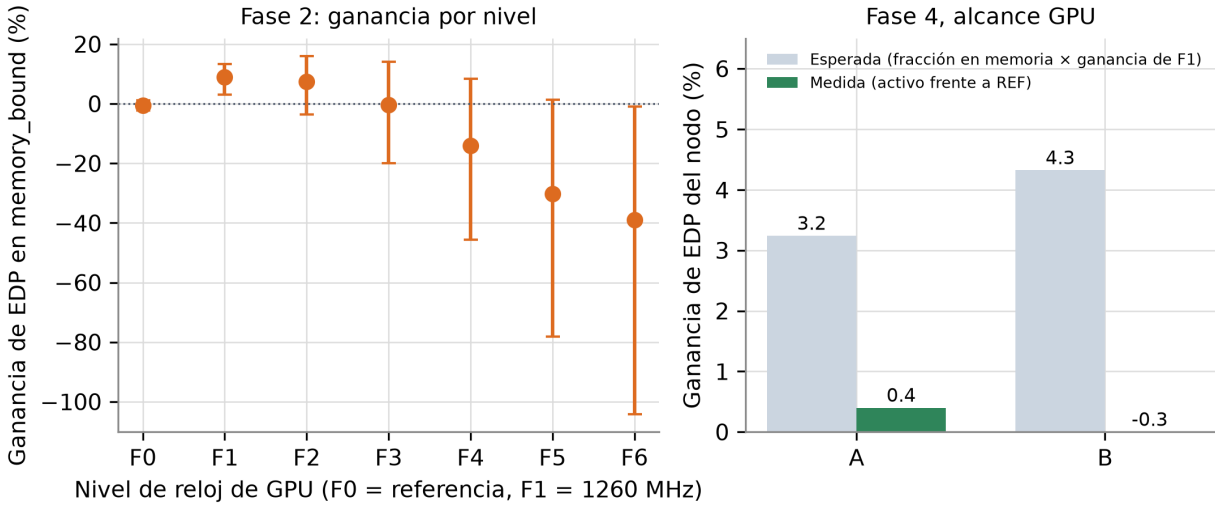


Figura 3.33: Izquierda: ganancia de EDP por nivel de reloj de GPU en fases *memory_bound* (Fase 2, IC95 sobre familias). Derecha: ganancia esperada (fracción de tiempo en memoria por la ganancia de F1) frente a la medida con el agente activo en el alcance GPU.

En A, el agente clasificó las dos fases de *triad* y aplicó F1 en ambas, y aun así la ganancia medida queda por debajo de la esperada: con tres repeticiones no puede separarse de cero y su causa no está aislada. En B, las fases de BabelStream, de 8 s, se resolvieron con una sola decisión cada una y en las celdas revisadas esa decisión fue una abstención, así que el reloj no se bajó. La ganancia posible de la GPU depende entonces de dos condiciones: que la aplicación pase una parte apreciable del tiempo en fases largas *memory_bound*, y que el agente no se abstenga en ellas. El escenario E de la sección 3.4.8 evalúa ambas.

3.4.6. Escenario C: agente de CPU con un hilo de inferencia

Este escenario repite el alcance de CPU y el conjunto con el agente en su configuración final (un hilo de inferencia, sección 3.4.4), con tres repeticiones por brazo (Tabla B.17 y

Figura 3.34). El costo de mantener el agente pasó a ser pequeño: el brazo sombra queda entre 1.036 y 1.094 del EDP de REF, con un exceso de tiempo de 0.4 a 1.4% y un exceso de energía de CPU de 5 a 9%. El brazo activo queda entre 1.05 y 1.16 y no mejora sobre sombra: el cociente activo/sombra es 1.06, 1.00, 1.02 y 1.01. En CPU con kernels vistos, bajar a F1 en memoria (1.161) es peor que mantener F0 (1.096). Quitar el piso de coordinación en el alcance conjunto empeora el EDP entre 0.8 y 1.3 puntos, de modo que el piso tiene un efecto pequeño pero favorable. La clasificación mantiene su exactitud con el agente corregido: 97.6% en A y 99.9% en B en CPU, sin errores en las decisiones de GPU (Tabla B.20).

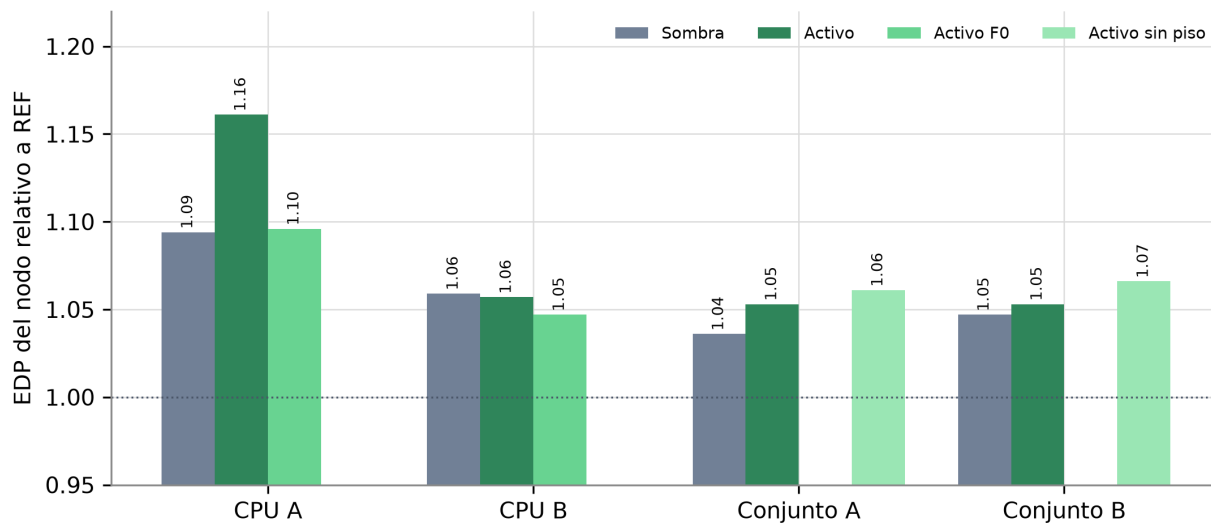


Figura 3.34: EDP del nodo relativo a REF en el escenario C, por alcance, aplicación y brazo. La línea punteada marca la paridad con REF.

3.4.7. Referencia sin turbo

REF sin turbo produjo razones medianas de EDP de 0.988 a 1.006 frente a REF en las seis combinaciones compuestas medidas (Tabla B.18). En estas cargas el turbo no elevó el reloj sobre el límite de 3.2 GHz; por ello, las conclusiones CPU no dependen de esa línea base. La comparación no se extiende a LAMMPS ni CloverLeaf, ni sustituye un brazo de GPU con F1 fijo.

3.4.8. Escenario E: GPU dominada por fases de memoria

El escenario E mide la política de GPU en las condiciones en que la Fase 2 predice ganancia: aplicaciones cuyo tiempo se reparte sobre todo en fases *memory_bound* largas. Es un escenario de aplicabilidad y no representa cualquier carga. Se definió antes de ejecutarse

con un criterio de selección explícito: familias *memory_bound* cuya ganancia de EDP con F1 en la Fase 2 fue de al menos 10 %, más una variante con familias no vistas en el entrenamiento. La duración de las fases sale de una medición previa: el agente de GPU decide una vez por fase y la primera decisión llega entre 13 y 16 s después de su inicio, de modo que fases de 18 s pasan solo la mitad de su tiempo en F1 y fases de 45 a 60 s pasan más del 70 %.

La Tabla 3.7 resume las dos aplicaciones declaradas antes de medir. En B, *myocyte* recibe una etiqueta de memoria por kernel pero casi no ocupa la GPU; solo el 42 % del ciclo corresponde a memoria con actividad GPU. Esta diferencia entre etiqueta y oportunidad de actuación condiciona la interpretación del resultado.

Tabla 3.7: Composición del escenario E. Duraciones aproximadas por fase medidas antes de la matriz.

Aplicación	Fases	Tiempo etiquetado memoria
A (vistas)	Stencil (58–60 s), SpMV (46 s), AXPY (47–48 s); DGEMM cómputo (31 s)	84 % del ciclo
B (inéditas)	BabelStream (36 s), Myocyte (40–41 s); LavaMD cómputo (9 s)	89 % etiquetado; 42 % con actividad GPU de memoria

Medición exploratoria en familias vistas (A). La medición exploratoria ejecutó tres repeticiones con REF, sombra y agente activo. El agente activo redujo la energía de GPU en 7.1 % (0.929 de REF), mantuvo la duración (1.000) y redujo el EDP del nodo a 0.975 y el EDP de GPU sola a 0.928 (Tabla B.19 y Figura 3.35). Sombra quedó en 0.995 del EDP de REF, lo que localiza la mayor parte del ahorro en la actuación F1. El agente clasificó y aplicó F1 en las fases de memoria previstas. La confirmación prospectiva de cinco bloques se reporta por separado a continuación.

El ahorro del nodo es modesto porque la CPU consume el 64 % de la energía en esta aplicación y no se modifica. Con el agente de CPU en observación, el EDP del nodo vuelve a 1.003, porque su costo (2.9 % más de energía de CPU y 1.0 % más de tiempo) iguala el ahorro de GPU. La configuración que mejora es, por tanto, la de un agente de GPU sin agente de CPU, coherente con la política de CPU de no actuar.

Confirmación prospectiva de E-A. La mejora frente a REF se repitió en un confirmatorio declarado antes de medir, con cinco bloques aleatorizados y cinco repeticiones válidas

por brazo. El brazo activo redujo la energía de GPU a 0.9282 de REF, mantuvo la duración en 0.9990 y dejó el EDP del nodo en 0.9716, equivalente a una reducción de 2.84 %. El brazo sombra quedó en 0.9935, por lo que el efecto no se explica por el costo de observación. Las cinco diferencias activo-REF fueron favorables; con bloques como unidad, la prueba exacta bilateral tiene $p = 0,0625$, el mínimo posible para este diseño. El brazo F1 fijo no produjo celdas válidas, porque el reloj observado bajo carga quedó fuera del intervalo de 1230–1290 MHz exigido por el protocolo. Por ello, el confirmatorio reproduce la mejora frente a REF, pero no permite establecer la ventaja del cambio por fase frente a mantener F1 durante toda la aplicación.

Premedición de F1 fijo. Antes del confirmatorio se ejecutó la medición diagnóstica F, con cinco repeticiones de REF y de una solicitud de F1 en DGEMM de CUTLASS, Conv2D y LavaMD. No se integra con E-A ni se usa para estimar el efecto del agente. En Conv2D y LavaMD, el EDP de GPU sola fue 0.894 y 0.960 de REF, respectivamente, pero el EDP del nodo fue 1.017 y 1.024 por el aumento del tiempo. En DGEMM, el reloj observado con F1 solicitado fue de 1140 a 1155 MHz, indistinguible de REF, por lo que esa comparación no valida una actuación a 1260 MHz. Este antecedente motivó el criterio de reloj bajo carga que después invalidó el brazo fijo del confirmatorio E-A.

Familias inéditas (B). La mediana quedó cerca de paridad (EDP de nodo 1.014 y energía GPU 1.003 de REF). BabelStream recibió una decisión de memoria en una de tres repeticiones y allí ahorró 12 % de energía GPU y 5 % de EDP del nodo; en las otras dos se abstuvo. *Myocyte* no superó sostenidamente el umbral de actividad GPU y no recibió decisión. Una demora aislada de esa fase ocurrió sin cambio de reloj atribuible al agente. El límite observado es la cobertura de decisión sobre familias inéditas, no un empeoramiento sistemático por F1.

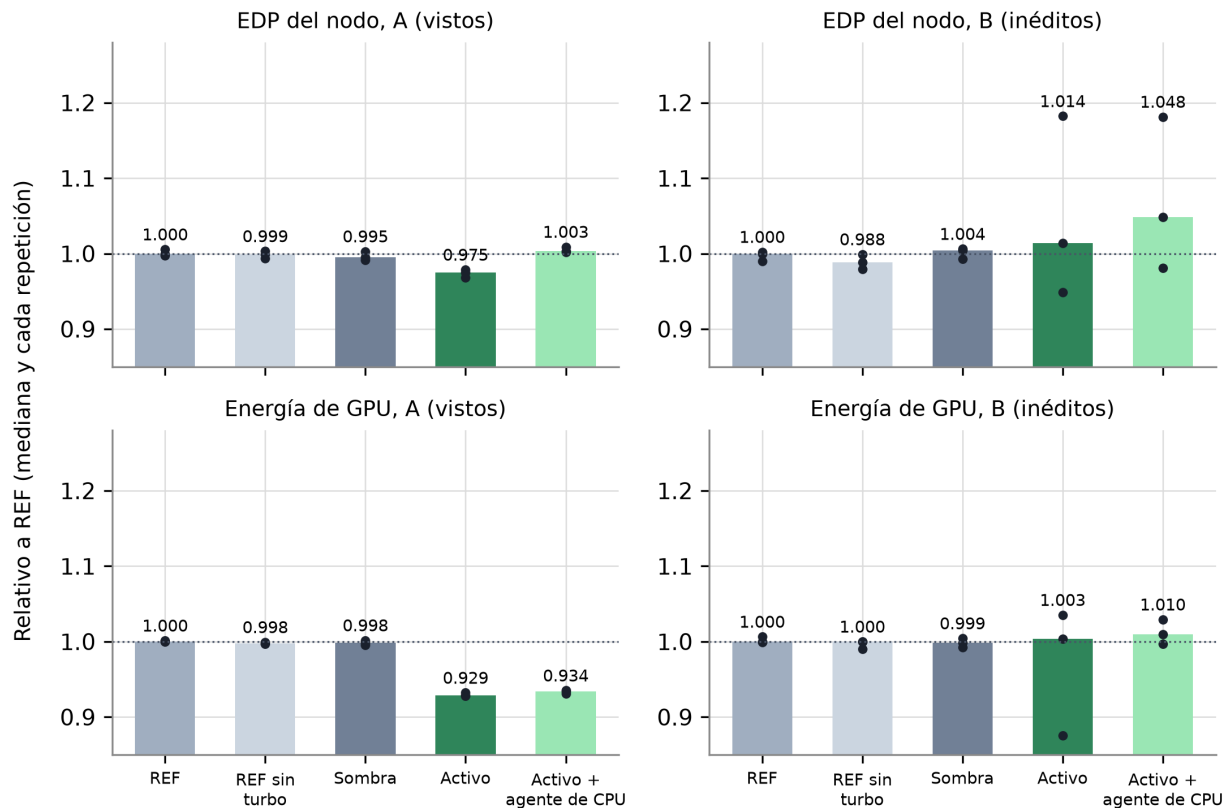


Figura 3.35: Escenario E. EDP del nodo (arriba) y energía de GPU (abajo) relativos a REF, con la mediana (barra) y cada repetición (punto), para las aplicaciones de familias vistas (A) e inéditas (B).

3.4.9. Escenario D: LAMMPS como aplicación de terceros

El escenario D evalúa LAMMPS 2023 con el paquete GPU del clúster, sin imponer fronteras de fase al programa. Se ejecutaron tres bloques de las entradas `chain`, `eam`, `lj` y `rhodo`, con REF, sombra, agente de GPU activo y una variante que mantiene el agente de CPU en observación. Todas las 48 celdas terminaron con código de aplicación y de daemon válido, y restauraron el estado de frecuencia. La Figura 3.36 y la Tabla B.21 muestran que el brazo activo de GPU queda entre 0.998 y 1.008 de REF en EDP, según la entrada. No se observa una mejora persistente y la variante con el agente de CPU en observación incrementa el EDP entre 3.0 y 4.3%. En `chain` y `rhodo` el agente no emitió decisiones y en `lj` y `eam` solo emitió `compute_bound`, por lo que no aplicó F1. Este resultado extiende la evaluación a una aplicación externa, pero no mide el efecto de la política GPU sobre una fase actuada.

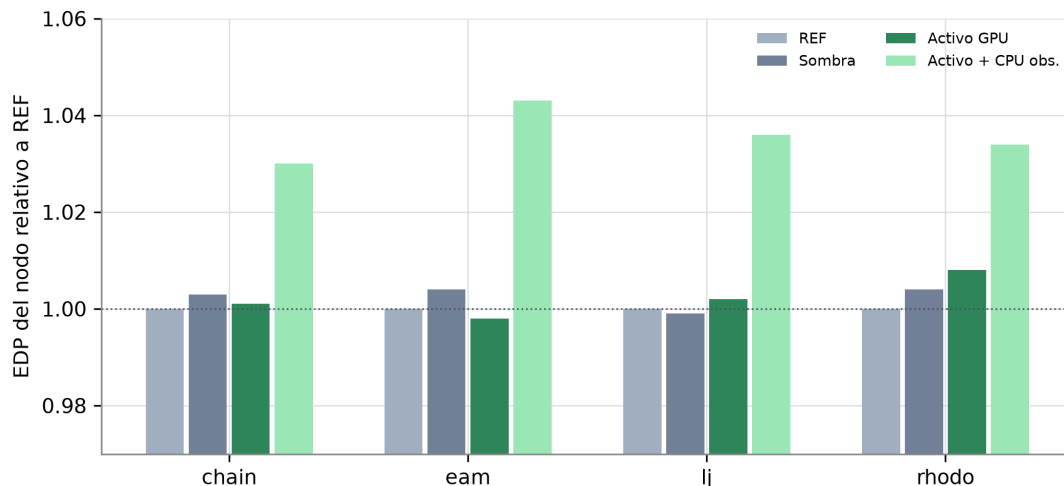


Figura 3.36: EDP mediano de LAMMPS por entrada y brazo, relativo a REF. Fuente: elaboración propia.

3.4.10. CloverLeaf CUDA Fortran como aplicación continua

CloverLeaf CUDA Fortran se evaluó como una aplicación continua de dinámica de fluidos, sin fronteras de fase impuestas al programa. El confirmatorio ejecutó cinco bloques aleatorizados con REF, sombra y activo. Las quince corridas terminaron con código de aplicación válido, restauración correcta del reloj y solución numérica reproducible; la energía cinética final fue idéntica dentro de la tolerancia relativa de 10^{-8} . En cada corrida activa el agente emitió una decisión *memory_bound* con confianza 1.00 tras aproximadamente 3 s y solicitó 1260 MHz. Tras esa decisión, las 433 o 434 muestras NVML bajo carga de cada bloque permanecieron en 1260 MHz. CloverLeaf no aporta una etiqueta Roofline independiente por fase, de modo que este experimento verifica la actuación y su efecto energético, pero no puntúa la exactitud de esa decisión. La Tabla 3.8 resume las medianas y las razones frente a REF.

El brazo activo mantuvo la duración en 1.0015 de REF, redujo la energía de GPU a 0.9393 y dejó el EDP del nodo en 0.9625, una reducción de 3.75 %. El brazo sombra quedó en 0.9958 de EDP, por lo que no se observa un costo apreciable de mantener el agente sin actuar. Las cinco diferencias de EDP activo–REF fueron favorables; la prueba exacta bilateral por bloques da $p = 0,0625$, que no alcanza el umbral de 0.05. La Figura 3.37 muestra las cinco observaciones emparejadas por bloque y la Figura 3.38 descompone el resultado entre CPU y GPU. En este caso, la actuación reduce la energía del dispositivo sin modificar de forma apreciable el tiempo de ejecución.

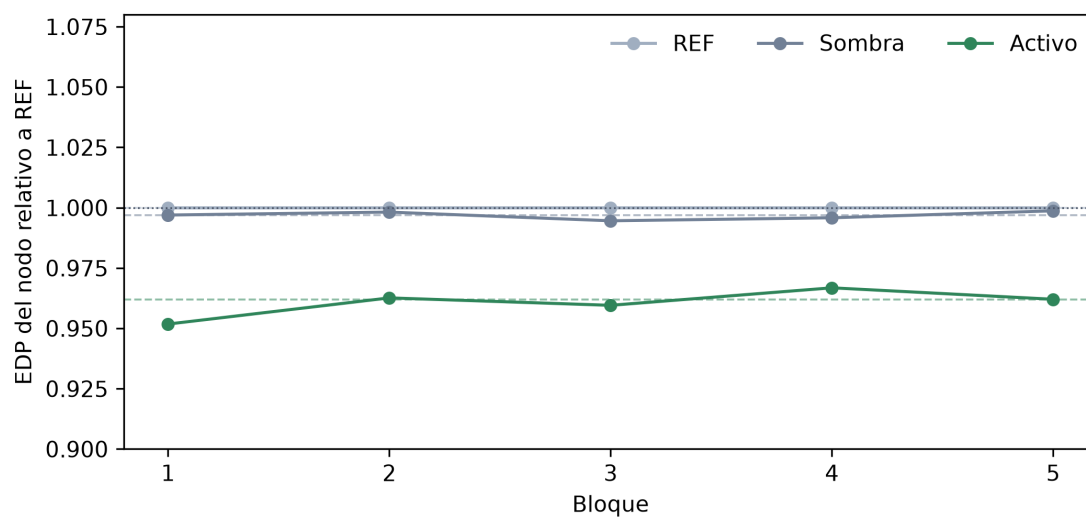


Figura 3.37: EDP del nodo de CloverLeaf relativo a REF en cada bloque. Las líneas discontinuas indican la mediana de cada brazo y la línea punteada marca la paridad con REF. Fuente: elaboración propia a partir de cinco bloques del nodo `paccaA100`.

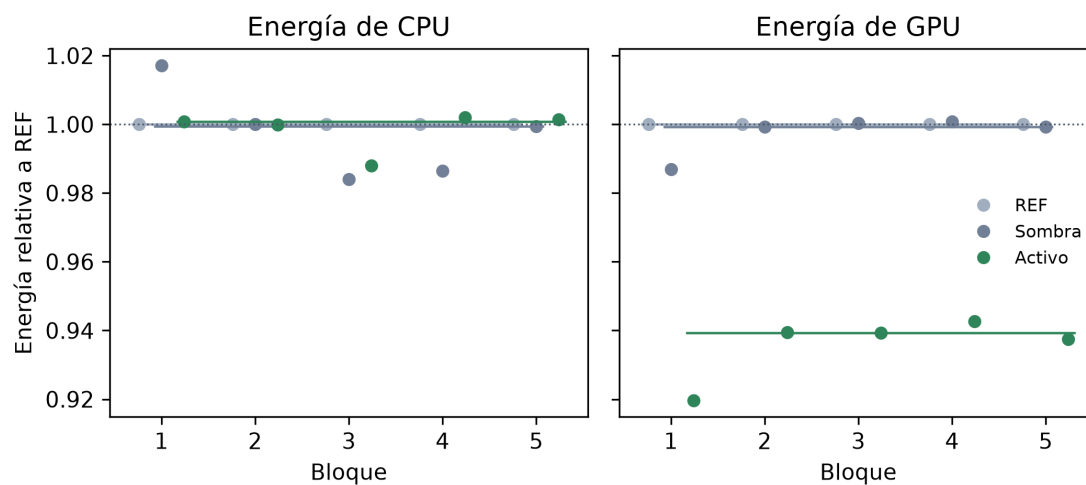


Figura 3.38: Energía de CPU y GPU de CloverLeaf relativa a REF en los cinco bloques. Los puntos representan bloques individuales y las líneas horizontales, las medianas por brazo. Fuente: elaboración propia.

Tabla 3.8: Resultados del confirmatorio de CloverLeaf CUDA Fortran. Duración y energías son medianas de cinco bloques; las razones se calculan frente a REF.

Brazo	n	T (s)	E_{CPU} (J)	E_{GPU} (J)	EDP/REF	$E_{\text{GPU}}/\mathbf{REF}$
REF	5	112.032	16939.3	27729.5	1.0000	1.0000
Sombra	5	111.975	16923.4	27733.6	0.9958	1.0001
Activo	5	112.201	16957.7	26044.9	0.9625	0.9393

3.4.11. Balance de la Fase 4

La Tabla 3.9 reúne la respuesta a cada pregunta de la Fase 4 y distingue los resultados favorables, neutros y no demostrados.

Tabla 3.9: Balance de la Fase 4. El desglose por pregunta se conserva en la Tabla B.31 del Anexo B.

Pregunta	Resultado
Clasificación	Alta en aplicaciones con verdad conocida; la GPU se abs- tiene con frecuencia ante familias inéditas
Costo de observación	GPU sin costo medible; CPU con un hilo añade 5 a 9 % de energía
Actuación CPU	Sin mejora de EDP frente a REF ni frente a sombra
Actuación GPU	E-A: $-2,84\%$ de EDP del nodo; CloverLeaf: $-3,75\%$; LAMMPS: paridad
F1 fijo	La comparación directa quedó inválida por reloj no soste- nido bajo carga

Capítulo 4

Discusión

4.1. Validez de la medición

Los enfoques de selección de frecuencia revisados [2, 15, 34] centran su evaluación en la decisión. En este trabajo también se comprobó su premisa física: que el hardware ejecuta la frecuencia solicitada. Restringir solo los procesadores lógicos delegados de CPU no alteró el rendimiento; fue necesario restringir sus hermanos de *SMT*. En GPU, el candado sostuvo los relojes solicitados durante la prueba de actuación, aunque el brazo F1 fijo del confirmatorio posterior falló la verificación bajo carga. Estas observaciones justifican aceptar una corrida por el reloj medido y no por el comando de actuación. La verificación independiente de los techos Roofline y del presupuesto de contadores cumple la misma función de protección de la etiqueta.

El catálogo GPU mostró, además, que el tipo de dato declarado por un kernel no siempre describe la precisión que ejecuta el hardware. En una ruta de biblioteca acelerada, más del 70 % del trabajo aritmético medido siguió una precisión distinta de la esperada. Para un kernel cercano al punto de inflexión, esa diferencia puede cambiar la etiqueta Roofline. La caracterización de precisión efectiva debe preceder al entrenamiento cuando la etiqueta depende del número de operaciones por precisión.

4.2. Generalización de los clasificadores

El resultado de CPU cambia al modificar la unidad de separación: mezclar intervalos al azar da 0.874 de exactitud balanceada; dejar fuera un kernel, 0.733; dejar fuera una familia, 0.728. Los 1.31 millones de intervalos proceden de 30 familias, por lo que el último protocolo

mide mejor la llegada de un algoritmo nuevo. Cambiar el modelo, ampliar las variables o buscar hiperparámetros no produjo una mejora distinguible bajo ese protocolo. La diversidad de familias sí elevó el resultado. La abstención reduce algunos errores, pero no corrige los de alta confianza en familias inéditas.

En GPU, la selección cambió al considerar el lazo de control. La regresión logística de cinco señales tenía buena exactitud y calibración estática, pero leía reloj y potencia, dos consecuencias de la acción que debía decidir. El bosque desplegado usa tres entradas invariantes y evita esa retroalimentación; su umbral de 0.90 sacrifica cobertura para sostener la precisión de las decisiones emitidas. La latencia de inferencia es pequeña frente a los segundos que consume detectar una fase y actuar. Por ello, mejorar solo el tiempo de predicción tendría poco efecto sobre fases breves; la abstención, que el agente no reevalúa en la misma fase, sigue siendo la limitación principal ante memoria inédita.

4.3. Alcance de la ganancia energética

La diferencia entre CPU y GPU procede de la respuesta física de cada dispositivo. En CPU, el piso de unos 85 W representa el 84 % de la potencia al mayor nivel fijo. Reducir el reloj deja poco margen energético y prolonga la ejecución, de modo que la política por clase es *no actuar*. En GPU, el piso representa 55.8 % de la potencia al reloj máximo y las fases limitadas por memoria responden poco al reloj SM; F1 mejoró su EDP agregado 8.9 % en la campaña de kernels. La etiqueta Roofline identifica una oportunidad, pero no garantiza que todos los kernels de la clase respondan igual: `rodinia_gaussian` perdió en la validación por familia retirada.

La Fase 4 delimitó dónde se traduce esa oportunidad en una aplicación. El brazo sombra separó el costo de observar del efecto de actuar: el agente GPU no mostró costo medible, mientras que el de CPU añadió entre 5 y 9 % de energía incluso con un hilo de inferencia. E-A redujo el EDP del nodo 2.84 % y CloverLeaf 3.75 % frente a REF; el ahorro del dispositivo se diluyó porque la energía CPU apenas cambió. Los cinco bloques de cada caso favorecieron al agente, con $p = 0,0625$ en la prueba exacta bilateral. CloverLeaf verifica la actuación y la salida numérica, aunque carece de una etiqueta física independiente para puntuar la clasificación. La invalidez del brazo F1 fijo de E-A deja abierta la ventaja de conmutar frente a mantener ese nivel. LAMMPS y las familias inéditas quedaron cerca de paridad porque el agente no aplicó F1 de forma sostenida. La mejora observada corresponde a fases de memoria largas que reciben una decisión confiable, no a cualquier aplicación GPU.

Capítulo 5

Conclusiones

El instrumento CPU–GPU quedó validado mediante comprobaciones de contadores, frecuencia efectiva y calibración Roofline antes de usar sus medidas para entrenar. La campaña CPU produjo 1470 corridas aceptadas y 1 309 755 intervalos elegibles; la campaña GPU reunió 483 corridas de 16 familias. Esta base permitió separar la calidad de las etiquetas, la generalización de los modelos y el efecto energético de la actuación, tres propiedades que no se deducen unas de otras.

En CPU, el clasificador XGBoost obtuvo 0.728 de exactitud balanceada por celda al dejar una familia algorítmica fuera y 0.738 sobre el 72.2 % de los intervalos aceptados con el umbral de confianza de 0.85. La diversidad de familias influyó más que cambiar el algoritmo o ajustar sus hiperparámetros. La política energética resultó *no actuar* en las dos clases Roofline: unos 85 W de potencia de paquete, equivalentes al 84 % de la potencia al mayor nivel fijo, permanecen al reducir el reloj. Incluso la elección ideal del mejor nivel para cada kernel ofrece solo 2.2 % de ganancia media. La Fase 4 confirmó que, en las aplicaciones compuestas medidas, actuar sobre la frecuencia CPU no mejoró el EDP frente a REF.

En GPU, la regresión logística de cinco señales fue una referencia estática competitiva (exactitud balanceada 0.792; ECE 0.0569), pero el reloj y la potencia entre sus entradas cambian cuando actúa el agente. El modelo desplegado es un *random forest* de tres señales invariantes, con exactitud balanceada de 0.819 y ECE de 0.0868. Con el umbral operativo de 0.90, su exactitud sobre decisiones aceptadas llegó a 0.900, con cobertura media por celda de 65.7 %. La política asociada seleccionó F1 para *memory_bound*, con una mejora de EDP de GPU de 8.9 % (IC95 de 3.0 a 13.3 %) en la campaña de kernels; para *compute_bound* conservó REF. Este contraste entre dispositivos responde al menor peso del piso de potencia GPU y a la escasa sensibilidad al reloj de cómputo de las cargas limitadas por memoria.

El agente quedó implementado y evaluado en los brazos REF, sombra y activo. En la Fase 4, el confirmatorio prospectivo de E-A redujo la energía GPU 7.18 % y el EDP del nodo 2.84 % frente a REF. CloverLeaf CUDA Fortran, una aplicación externa continua, redujo esas magnitudes 6.07 y 3.75 %, respectivamente. Los cinco bloques de cada experimento favorecieron al agente, pero la prueba exacta bilateral dio $p = 0,0625$; por ello no se afirma significancia al nivel 0.05. En CloverLeaf se verificaron la actuación y la salida numérica, pero no existe una etiqueta física independiente para puntuar su clasificación de fase. El brazo F1 fijo del confirmatorio E-A no sostuvo el reloj exigido bajo carga, de modo que la ventaja del cambio por fase frente a ese nivel fijo quedó sin establecer. LAMMPS y las aplicaciones con familias GPU inéditas mostraron resultados cercanos a paridad; en estas últimas, la abstención limitó la oportunidad de ahorro. La mejora demostrada es, por tanto, condicional a fases de memoria suficientemente largas y a decisiones de alta confianza.

5.1. Limitaciones

Los intervalos CPU proceden de 30 familias y no son unidades independientes de generalización. Su etiqueta tiene la granularidad de *uncore*, cercana a 10 ms, y la abstención no elimina errores de alta confianza sobre familias nuevas. El modelo GPU se evaluó con 16 familias y solo cuatro familias externas inéditas; NVML no observa los contadores que definen directamente el régimen Roofline, y algunos kernels cambian de régimen al variar la frecuencia. La política F1 se validó con ocho familias *memory_bound*, tres de ellas variantes de un mismo algoritmo. Los dos experimentos favorables de aplicación tuvieron cinco bloques cada uno y el contraste contra F1 fijo quedó inválido. Estas condiciones acotan la extrapolación a otras aplicaciones y plataformas.

5.2. Trabajo futuro

La siguiente validación debería ampliar las familias GPU externas, especialmente fases *memory_bound* que hoy provocan abstención, y evaluar una sola vez el modelo y el umbral congelados. También conviene repetir el contraste entre REF, agente y F1 fijo con verificación continua del reloj bajo carga. Para mejorar la respuesta en fases breves, se puede medir por separado la latencia de detección, recolección de señales y actuación, y estudiar una reevaluación controlada de las abstenciones. CUPTI Activity podría aportar fronteras de fase para construir etiquetas independientes, previa medición de su intrusión, mientras NVML permanece como entrada ligera del agente. En CPU, cualquier revisión de la política requiere primero reducir el costo de observación y demostrar un margen de ahorro superior al piso de

potencia medido.

Apéndice A

Detalle técnico de la metodología

Este anexo reúne los elementos de consulta que respaldan el capítulo 2. Conserva condiciones de validez, tablas de control y procedimientos cuyo detalle sería desproporcionado en el cuerpo principal.

A.1. Diseño de la medición

A.1.1. Dominio de frecuencia y topología SMT

El alcance de una solicitud de frecuencia no coincide necesariamente con la afinidad de ejecución de una carga. Linux organiza CPUFreq en políticas asociadas a una o varias CPU lógicas, identificables mediante `affected_cpus` y `related_cpus` [22].

Con *simultaneous multithreading* (SMT), dos procesadores lógicos comparten un núcleo físico. Aunque expongan límites de rendimiento separados a nivel de software, su frecuencia efectiva puede depender de las solicitudes de ambos. En `intel_pstate`, una solicitud de mayor rendimiento de un hermano puede elevar el nivel efectivo del otro por encima de su límite individual [23].

Por ello, restringir la carga a un hilo lógico por núcleo evita la contención entre hermanos, pero no basta para aislar el estado de frecuencia. El protocolo controla los procesadores lógicos delegados y sus hermanos, y verifica el reloj efectivo durante la carga.

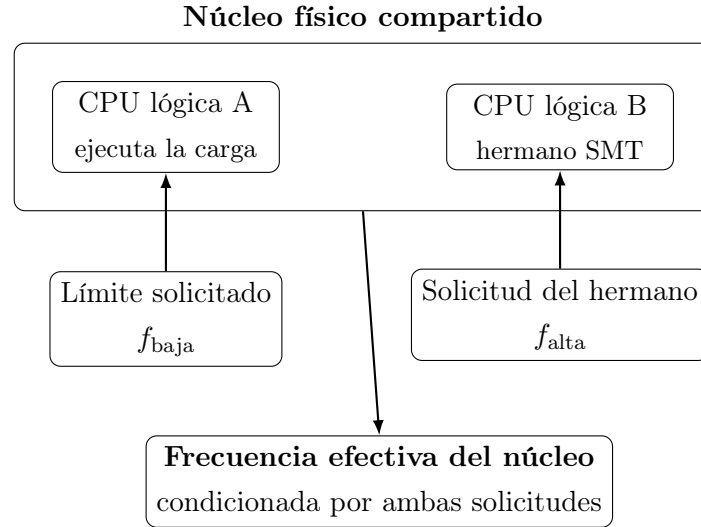


Figura A.1: Relación conceptual entre las solicitudes de rendimiento de dos procesadores lógicos hermanos y la frecuencia efectiva de un núcleo físico con SMT habilitado. Fuente: elaboración propia con base en la documentación de `intel_pstate` [23].

A.1.2. Instrumento y señales

El instrumento separa la medición de baja latencia, implementada en C++, de la planificación y validación de campañas, implementadas en Python. El ejecutor detiene el proceso hijo antes de su primera instrucción, abre los contadores con herencia y después lo reanuda. Un productor lee las señales y un consumidor las persiste sin competir con la cadencia de lectura.

Los eventos de núcleo se atribuyen a la carga medida. Los comandos CAS del controlador de memoria integrado pertenecen al dominio *uncore*, compartido en el zócalo, por lo que la campaña requiere asignación exclusiva del nodo.

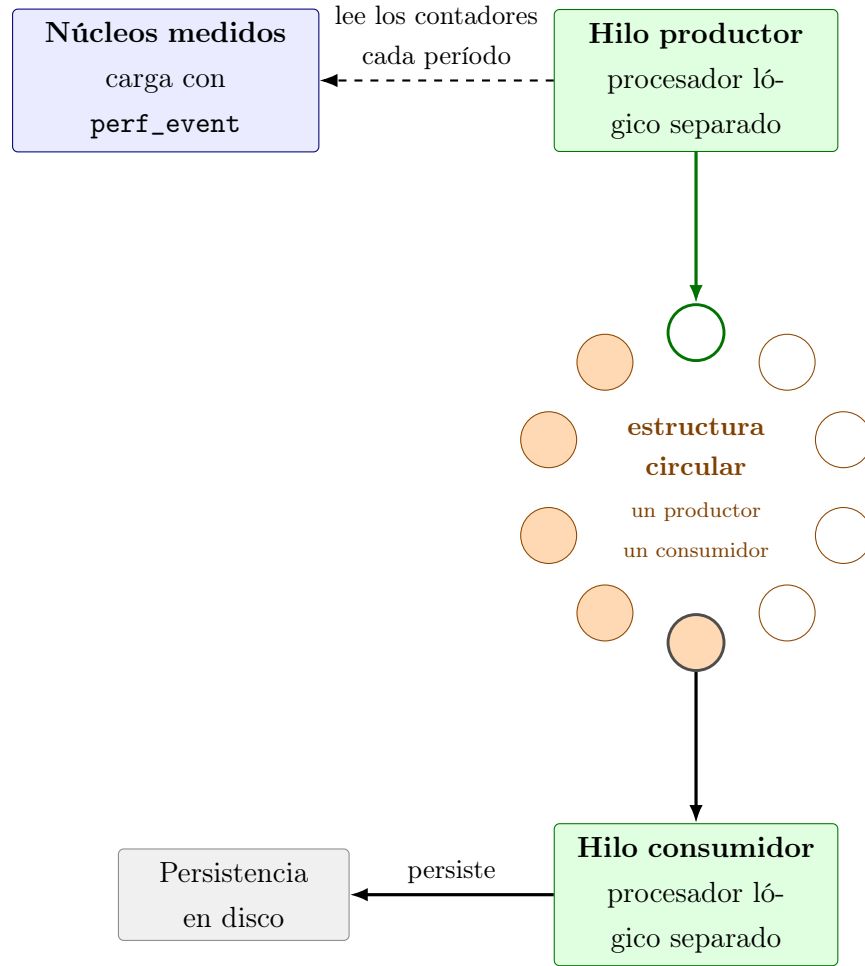


Figura A.2: Ruta de una muestra de telemetría desde la lectura de los contadores hasta su persistencia. Los hilos de medición y persistencia se fijan a procesadores lógicos distintos de los que ejecutan la carga. Fuente: elaboración propia.

A.1.3. Granularidad, calidad y reproducibilidad

En CPU, la fila de entrenamiento corresponde al intervalo del controlador de memoria, no a la ventana de núcleo. Los FLOPs se agregan sobre ese intervalo para alinearlos con los bytes de DRAM y se recalculan las tasas a partir de sus deltas. En GPU, la fila corresponde a una corrida completa porque NVML entrega señales agregadas y la etiqueta Roofline se obtiene fuera de línea por kernel.

Las ventanas de transición, calentamiento, frecuencia no verificable, energía inválida, contadores degradados o intensidad indefinida se conservan con su causa y se excluyen solo en el análisis correspondiente. Una corrida completa exige, además, criterio de éxito, suma de verificación del binario y ausencia de pérdidas en la estructura circular.

El transitorio se estima después de adquirir la traza completa, sin condicionar la captura. Sobre IPC en CPU y utilización en GPU, primero se busca el primer instante en que el coeficiente de variación no supera 5 % en dos ventanas móviles consecutivas. Si una corrida contiene n muestras, cada ventana móvil tiene $w = \max(3, \min(15, \lfloor n/4 \rfloor))$ muestras. Así, el análisis usa aproximadamente un cuarto de la traza cuando esta es corta, nunca menos de tres muestras ni más de quince. El instante detectado recibe un margen de seguridad de 20 %. En GPU se exige además que la media de ambas ventanas alcance al menos 5 % de utilización, para que el reposo inicial no se interprete como estabilidad.

Si ese criterio no encuentra un estado estable, se aplica detección recursiva de puntos de cambio sobre la misma señal. Cada partición debe reducir al menos 10 % la suma de errores cuadráticos, conserva segmentos de $\max(5, \lfloor n/50 \rfloor)$ muestras y alcanza como máximo seis niveles de recursión. Entre los segmentos obtenidos se selecciona el primero cuya media sea al menos 80 % de la meseta de mayor carga de la corrida. Durante la calibración de GPU, el intervalo se contrasta además con el ascenso de potencia hacia el régimen sostenido.

Cada campaña persiste un manifiesto versionado con cargas, frecuencias, repeticiones, afinidad, semilla y referencias de calibración. Una huella del protocolo impide mezclar resultados de manifiestos distintos al reanudar una campaña. La calibración ya aceptada se reutiliza tras verificar que corresponde a los niveles declarados.

A.2. Fase 1. Recolección y caracterización

A.2.1. Verificación previa

Antes de cada campaña se ejecutan verificaciones bloqueantes sobre el nodo, el catálogo y la instrumentación. Una falla detiene la campaña antes de producir datos.

Tabla A.1: Verificaciones previas agrupadas por familia.

Familia	Verificación bloqueante
Entorno	Turbo, afinidad NUMA, dominio real de frecuencia, política SMT y presupuesto de contadores.
Catálogo	Binario, suma de verificación, criterio de éxito y memoria suficiente.
Instrumentación	Dominios RAPL, NVML, manejo de desbordamiento y almacenamiento disponible.
Actuación	Permiso de escritura y rango soportado de frecuencia cuando la campaña lo requiere.

A.2.2. Catálogo y cribado

El catálogo incluye microbenchmarks de calibración, que no entrenan al clasificador, y familias de benchmarks con trabajo de punto flotante verificable. La familia algorítmica, no la variante de tamaño, es la unidad de diversidad para selección y validación.

Tabla A.2: Catálogo resumido de cargas.

Grupo	Origen	Papel
STREAM y ERT	Referencias CPU	Calibración de ancho de banda y cómputo.
Microbenchmarks FP y BW	Elaboración propia	Calibración GPU y controles.
NAS, BLAS y LavaMD	NPB, BLAS y Rodinia	Familias CPU de referencia.
RAJAPerf, CHOLMOD, LULESH, HPCG, HPCCG y PageRank	Suites científicas	Diversidad de patrones CPU.
Rodinia y cuBLAS	Rodinia y NVIDIA	Familias GPU de referencia.

Cada candidato pasa por un cribado de tres niveles de frecuencia. Se comprueba la calidad de señal, la cobertura respecto del punto de inflexión y la representación de ambas clases por familia y dispositivo. Solo las familias elegibles entran a la matriz definitiva.

A.2.3. Calibración y matriz

La intensidad operacional se compara con techos Roofline calibrados por dispositivo, precisión y nivel de frecuencia. En GPU, `ncu` mide FLOPs y bytes fuera de línea sobre ejecuciones repetidas del kernel. Se acepta la intensidad cuando los dos puntos de mayor trabajo difieren menos de 1 %; las cargas de precisión mixta no resoluble o sin trabajo de punto flotante se excluyen.

$$\text{FLOPs}_i^{\text{HW}} = \Delta N_{\text{esc},i} + 2\Delta N_{128,i} + 4\Delta N_{256,i} + 8\Delta N_{512,i}. \quad (\text{A.1})$$

$$I^{\text{GPU}} = \frac{\sum_{\ell=1}^L (N_{\ell}^{\text{add}} + N_{\ell}^{\text{mul}} + 2N_{\ell}^{\text{fma}})}{\sum_{\ell=1}^L B_{\ell}}. \quad (\text{A.2})$$

La matriz cruza familia, frecuencia y repeticiones independientes. El orden se aleatoriza con semilla registrada, se intercalan cargas de referencia y cada dispositivo no intervenido permanece en su gobierno nativo. La huella del protocolo protege las campañas reanudables contra mezclas de condiciones.

Tabla A.3: Rejilla fina de frecuencia de la campaña definitiva.

Dispositivo	Niveles de reloj en MHz
CPU	REF, F0 (3200), F1 (2900), F2 (2600), F3 (2300), F4 (2000), F5 (1700), F6 (1400), F7 (1100) y F8 (800).
GPU	REF, 1410, 1350, 1290, 1230, 1170, 1110, 810, 510 y 210.

A.3. Fase 2. Desarrollo y validación de los modelos

A.3.1. Variables y prevención de fuga

El modelo CPU usa IPC, MPKI, tasa de fallos de caché, fracción de ciclos detenidos por memoria, instrucciones por segundo y frecuencia observada. El modelo GPU usa agregados robustos de utilización de GPU y memoria, potencia y reloj, junto con la variabilidad de la utilización de memoria. La intensidad, los FLOPs, los bytes, el punto de inflexión y la etiqueta están prohibidos como entradas.

La selección se congela en un contrato versionado por dispositivo. Las columnas ausentes, constantes o no finitas se tratan explícitamente y la presencia de una variable prohibida aborta el entrenamiento.

A.3.2. Auditoría, muestreo y validación

Antes de entrenar se calculan correlaciones de Pearson y Spearman y el factor de inflación de varianza. Cuando dos columnas describen la misma señal se conserva la medición físicamente más directa.

La validación deja fuera familias completas. El ajuste se pondera para no sobrerrepresentar familias o clases con más observaciones, y la métrica principal es la exactitud balanceada por celda familia-clase:

$$w_i = \frac{1/n_{fc}}{(1/N) \sum_{k=1}^N 1/n_{f_k c_k}}. \quad (\text{A.3})$$

$$\text{EB} = \frac{1}{|\mathcal{C}|} \sum_{(f,c) \in \mathcal{C}} \frac{a_{fc}}{n_{fc}}. \quad (\text{A.4})$$

El candidato se congela antes de examinar errores por familia o frecuencia.

El límite del resultado se diagnostica mediante una referencia de vecinos cercanos, perfiles de familias con señales similares y clases opuestas, desempeño según distancia al *ridge* y curva de aprendizaje por número de familias. Estas pruebas separan falta de capacidad del modelo de falta de información en las señales.

La probabilidad fuera de muestra se evalúa agrupando las predicciones en rangos de confianza de igual amplitud y mediante el error de calibración esperado. El umbral de abstención se selecciona sobre las predicciones de validación por familia, con cobertura mínima declarada, y queda congelado con el candidato.

A.3.3. Derivación de la política

La política CPU se deriva del EDP por corrida, no de las ventanas ni del clasificador. Para cada kernel se compara la mediana de cada nivel contra su propio REF; la agregación se realiza por clase y se complementa por familia para evitar pseudorreplicación. Un nivel solo se adopta si supera el efecto mínimo relevante y la prueba pareada declarada. De lo contrario, la salida es *no actuar*.

La política GPU usa el mismo principio con el kernel como unidad de comparación. La clase del kernel se determina por mayoría de sus corridas y se marca ambigua si el margen es menor de 0.75. La regla se contrasta mediante *leave-one-familia-out*; los kernels sin referencia nativa no participan en pruebas apareadas.

A.4. Detalles adicionales del diseño

Tabla A.4: Señales de telemetría del instrumento.

Dominio	Señales
Núcleo (CPU)	Instrucciones, ciclos, referencias y fallos de caché, ciclos detenidos por memoria, sub-eventos de punto flotante de doble precisión (para FLOPs), frecuencia observada (<code>scaling_cur_freq</code>)
Zócalo (CPU)	Comandos CAS de lectura y escritura del controlador de memoria (bytes a DRAM), energía RAPL de paquete y DRAM
GPU	NVML: utilización de cómputo y de memoria, potencia, reloj del multiprocesador, temperatura y energía acumulada

Tabla A.5: Unidad de observación del conjunto de entrenamiento.

Dispositivo	Unidad de observación	Origen de I	Tamaño
CPU	Intervalo del controlador de memoria	En vivo, por intervalo	1 309 755 intervalos, 30 familias
GPU	Corrida completa	Fuera de línea, por kernel	483 corridas, 16 familias

A.5. Parámetros y controles del agente

Tabla A.6: Amenazas a la validez y medidas de control.

Amenaza	Medida de control
Interferencia de procesos ajenos en RAPL y en la GPU	Asignación exclusiva del nodo; ninguna carga adicional durante una medición
Atribución incorrecta de contadores	Proceso hijo detenido hasta abrir los contadores, con herencia
Multiplexación de eventos	Nueve eventos dentro del presupuesto verificado; las ventanas degradadas se excluyen
Frecuencia efectiva distinta de la pedida	Turbo apagado, rango por núcleo y hermanos <i>SMT</i> , verificación por ventana
Fuga de la etiqueta hacia las variables	Lista de columnas prohibidas comprobada por código
Pseudo-replicación entre tamaños y repeticiones	La familia como unidad de validación (LOFO) y de comparación de la política
Ajuste sobre datos de prueba	Congelación antes de auditar errores y prueba externa sellada única
Retroalimentación del agente sobre el modelo	Auditoría de invarianza de las variables frente a la acción del agente
Sesgo temporal o de orden	Orden aleatorizado con semilla registrada y repeticiones independientes
Deriva del estado del nodo	Snapshot y restauración del estado de frecuencia por celda
Muestra intencional en un solo nodo	Se declara como limitación; no se generaliza a otras plataformas

A.6. Análisis y decisiones complementarias

A.6.1. Derivación detallada de la política

La tabla que traduce cada clase de régimen en una frecuencia se deriva del barrido de la Fase 1 y no del clasificador. Para cada kernel y nivel de frecuencia, se compara la mediana de sus repeticiones contra su propio REF. Así se forman diferencias pareadas entre el mismo kernel medido en el nivel evaluado y en la referencia. Si hay menos de ocho pares o las diferencias no cumplen el criterio de normalidad establecido, se aplica Wilcoxon de rangos

con signo; en los demás casos se utiliza la prueba t pareada. El intervalo de confianza del 95% de la ganancia agregada se estima mediante *bootstrap* de kernels con 4000 réplicas, y se interpreta junto con el efecto mínimo relevante y el análisis de potencia. Cuando dos tamaños corresponden al mismo algoritmo, se agregan por familia y la regla “elegir el nivel de mayor ganancia media” se valida con *leave-one-family-out*. Cualquier refinamiento por subpoblación se declara antes de medir y se confirma una sola vez sobre familias externas selladas. En GPU, la clase de un kernel es la mayoritaria de sus corridas y se marca como ambigua si el margen es menor a 0.75, sin excluirlo, porque el punto de inflexión cambia con el reloj y un kernel cercano puede cambiar de clase entre niveles (anexo, secciones A.3.3 y A.3.3).

A.7. Protocolos completos de la Fase 4

A.8. Detalles adicionales de método y diagnóstico

A.8.1. Selección detallada del catálogo

La población es el conjunto de aplicaciones ejecutables en sistemas heterogéneos CPU–GPU. Como no es abarcable, se trabaja con una muestra intencional de benchmarks y micro-benchmarks que inducen regímenes de ejecución contrastantes, coherente con la necesidad de cargas controladas y reproducibles. Una carga entra al catálogo si cumple tres condiciones: dispone de una verificación interna de corrección que permite descartar corridas inválidas; su trabajo aritmético es observable por el propio instrumento (contadores de hardware en CPU, perfilado por contadores en GPU) y no depende de lo que la carga reporte de sí misma; y ese trabajo es efectivamente de punto flotante, de modo que la intensidad operacional tenga significado físico. La unidad de diversidad es la **familia algorítmica**, el patrón de cómputo y de acceso a memoria compartido por todas las variantes de tamaño de un mismo algoritmo. Tratar cada tamaño como un ejemplo distinto inflaría el conjunto sin añadir diversidad, y dejar fuera un tamaño no prueba generalización hacia un patrón nuevo. Por eso la familia gobierna la selección de cargas y, más adelante, la validación. Los microbenchmarks propios sirven como controles y no forman parte del conjunto de entrenamiento. Los detalles del catálogo y de su cribado están en el anexo (sección A.2.2).

Tabla A.7: Protocolos complementarios ejecutados en la Fase 4.

Escenario	Qué cambia	Motivo
A: matriz inicial	Ninguno	Referencia, tal como se diseñó
C: agente de CPU con un hilo de inferencia	Repite A en CPU y en el alcance conjunto con la configuración final del agente	Separa el efecto de actuar del costo de mantener el agente
E: GPU dominada por memoria	Aplicaciones de GPU con tres o dos fases <i>memory_bound</i> largas (45 a 60 s) y una de cómputo, de familias vistas (E-A) e inéditas (E-B); la CPU sin agente o con el agente en observación	El techo medido en la Fase 2 es 8.9% de EDP en GPU y el efecto escala con la fracción de tiempo en memoria; es un escenario de aplicabilidad
Referencia sin turbo	Sin agente, con turbo de CPU apagado, en las combinaciones compuestas de C y E	Línea base secundaria para aislar la contribución del turbo de CPU; no sustituye un brazo de GPU a F1 fijo
D: aplicación real de terceros	LAMMPS 2023 con paquete GPU, cuatro entradas de referencia y fases naturales	Validez externa; tres bloques por entrada y brazo
F: premedición de F1 fijo	REF frente a la solicitud de F1 en DGEMM de CUTLASS, Conv2D y LavaMD, cinco repeticiones por nivel	Comprobar que una comparación posterior contra un nivel fijo tendría un reloj efectivo distinto de REF
Confirmatorio E-A	Cinco bloques aleatorizados de REF, sombra, agente GPU activo y F1 fijo para la aplicación E-A	Confirmar la mejora frente a REF y contrastarla con F1 fijo; el brazo fijo se declara inválido si no sostiene el reloj bajo carga
CloverLeaf CUDA Fortran	Aplicación externa continua, cinco bloques aleatorizados con REF, sombra y agente GPU activo; una decisión inicial del agente por corrida	Verificar la política sobre una carga de terceros sostenida, con energía de nodo, salida numérica y reloj auditados

Apéndice B

Resultados complementarios

Este anexo conserva las tablas de desglose, sensibilidad y trazabilidad que respaldan el capítulo 3. El cuerpo principal mantiene las tablas de política, el modelo operativo y los resultados confirmatorios.

B.1. Detalle del clasificador de CPU

Tabla B.1: Variantes del vector de entrada bajo el mismo modelo y el mismo protocolo. La diferencia se calcula frente al vector de 6 variables.

Vector de entrada	Exact. bal.	F1 macro	Diferencia (IC95)
6 variables (configuración final)	0.728	0.760	referencia
12 variables (seis transformaciones adicionales)	0.724	0.783	-0,004 (-0,026 a +0,011)
6 variables sin frecuencia	0.720	0.752	-0,008 (-0,018 a +0,001)
Solo frecuencia observada	0.592	0.504	-0,136 (-0,208 a -0,065)

Tabla B.2: Efecto del esquema de partición sobre el resultado, con el mismo modelo y los mismos datos. El $\pm$ es la desviación estándar entre las cinco semillas de muestreo.

Esquema de partición	Qué mide	Exactitud balanceada
Intervalos repartidos al azar (80/20)	Memorización de la carta	0.874 ± 0.024
Dejar un kernel fuera (49 pliegues)	Otro tamaño o variante	0.733 ± 0.016
Dejar una familia fuera (30 pliegues)	Algoritmo nunca visto	0.728 ± 0.015

Tabla B.3: Métricas de evaluación empleadas y pregunta que responde cada una.

Métrica	Definición	Pregunta que responde y límite
Exactitud	$\frac{\sum_c VP_c}{N}$	¿Qué fracción de todos los intervalos se clasifica bien? Premia predecir la clase mayoritaria y la dominan las familias con más intervalos
Precisión de la clase c	$\frac{VP_c}{VP_c + FP_c}$	Cuando el modelo dice c , ¿cuántas veces acierta?
Sensibilidad de la clase c	$\frac{VP_c}{VP_c + FN_c}$	De los intervalos que son c , ¿cuántos detecta?
F1 de la clase c	$\frac{2 P_c S_c}{P_c + S_c}$	Media armónica de precisión y sensibilidad de esa clase
F1 macro agrupada	$\frac{1}{2}(F1_{\text{compute}} + F1_{\text{memory}})$	Promedio de las F1 de las dos clases, calculadas sobre todos los intervalos concatenados
Exactitud balanceada clásica	$\frac{1}{2}(S_{\text{compute}} + S_{\text{memory}})$	Promedio de las dos sensibilidades; corrige el desbalance entre clases pero no el volumen entre familias
AUC-ROC	Área bajo la curva sensibilidad frente a tasa de falsos positivos	¿Ordena el modelo los intervalos <i>memory-bound</i> por encima de los <i>compute-bound</i> ? No depende del umbral de decisión
Coefficiente de Matthews (MCC)	Correlación de Pearson entre la clase predicha y la clase real	Vale 0 para un modelo sin información y 1 para uno perfecto; usa las cuatro casillas de la matriz de confusión
Exactitud balanceada por celda	$\frac{1}{ C } \sum_{(f,c) \in C} \frac{a_{fc}}{n_{fc}}$	¿Con qué fracción de aciertos se resuelve una celda típica de familia y clase? Ecuación (A.4)

Tabla B.4: Valores de todas las métricas para el modelo final, sin abstención. Media de cinco semillas de muestreo, desviación entre semillas e intervalo de confianza del 95 % por remuestreo de las 30 familias.

Métrica	Valor	Desv. semillas	IC95 familias
Exactitud	0.769	0.011	0.678 a 0.840
Precisión <i>compute-bound</i>	0.712	0.016	–
Sensibilidad <i>compute-bound</i>	0.714	0.026	–
F1 <i>compute-bound</i>	0.713	0.015	–
Precisión <i>memory-bound</i>	0.808	0.013	–
Sensibilidad <i>memory-bound</i>	0.806	0.017	–
F1 <i>memory-bound</i>	0.807	0.010	–
F1 macro agrupada	0.760	0.012	–
Exactitud balanceada clásica	0.760	0.012	0.670 a 0.846
AUC-ROC agrupada	0.842	0.008	–
Coefficiente de Matthews	0.520	0.023	–
Exactitud balanceada por celda	0.728	0.015	0.653 a 0.785

Tabla B.5: Resultado del modelo final por nivel de frecuencia. La cobertura es la fracción de intervalos con confianza $\geq 0,85$; la exactitud es global y depende de la composición de clases de cada nivel, a diferencia de la exactitud balanceada por celda.

Nivel	Intervalos	Frac. memoria	Exact. bal. celda	Exactitud	Cobertura
REF	94 426	0.710	0.823	0.860	0.848
F0	85 447	0.685	0.777	0.849	0.844
F1	90 079	0.682	0.768	0.842	0.843
F2	96 045	0.664	0.753	0.818	0.819
F3	103 404	0.641	0.730	0.799	0.766
F4	114 253	0.588	0.751	0.753	0.718
F5	129 985	0.580	0.702	0.728	0.684
F6	153 276	0.564	0.692	0.712	0.668
F7	186 309	0.537	0.657	0.710	0.649
F8	256 531	0.536	0.659	0.757	0.643

Tabla B.6: Rejilla del umbral de confianza. Exactitud balanceada por celda sobre lo decidido y cobertura media por celda, promedio de cinco semillas. La regla elige el máximo con cobertura de al menos 0.60; se congeló 0.85.

Umbral	Cobertura por celda	Exact. bal. sobre lo decidido
0.50	1.000	0.728
0.80	0.770	0.742
0.85 (congelado)	0.714	0.749
0.90 (regla)	0.649	0.752
0.93	0.600	0.753
0.95	0.557	0.757
0.98	0.453	0.764

Tabla B.7: Efecto de la decisión selectiva con umbral de confianza 0.85.

Medida	Sin abstención	Con umbral 0.85
Cobertura	1.000	0.722
Cobertura media por familia	1.000	0.751
Cobertura de la familia peor cubierta	1.000	0.293
Exactitud balanceada por celda	0.728	0.738
F1 macro agrupada	0.760	0.842
Exactitud	0.769	0.850
Sensibilidad <i>compute-bound</i>	0.714	0.762
Sensibilidad <i>memory-bound</i>	0.806	0.910

Tabla B.8: Proporción de predicciones del clasificador de CPU que cambia de clase al forzar la frecuencia.

Frecuencia forzada	Predicciones que cambian
3.2 GHz	10.6 %
2.9 GHz	10.1 %
2.0 GHz	4.5 %
0.8 GHz	8.8 %
De F0 a F1, sobre filas observadas a $\approx$ 3.2 GHz (179 774)	0.42 %

B.2. Detalle del clasificador de GPU

Tabla B.9: Comparación de modelos candidatos sobre el conjunto GPU ampliado (483 corridas, 16 familias), exactitud balanceada por celda familia por clase.

Modelo	Exact. balanceada	IC95
Mayoritaria	0.526	[0.333, 0.722]
Árbol profundidad 1	0.555	[0.382, 0.733]
Regresión logística	0.792	[0.680, 0.898]
Árbol profundidad 6	0.727	[0.625, 0.835]
<i>Random forest</i>	0.807	[0.697, 0.917]
<i>Extra trees</i>	0.806	[0.684, 0.917]
XGBoost	0.740	[0.634, 0.872]

Tabla B.10: Factor de inflación de varianza (VIF) de las cinco variables de la referencia estática GPU.

Variable	VIF
Utilización GPU (mediana)	1.89
Utilización de memoria (mediana)	1.76
Potencia (mediana)	2.26
Reloj SM (mediana)	1.37
Desviación estándar de utilización de memoria	1.32

B.3. Detalle de la evaluación del agente

Tabla B.11: Decisiones del agente de GPU sobre las aplicaciones compuestas (umbral de abstención 0.90).

Aplicación	Brazo	Correctas	Erróneas	Abstenciones
A: dgemm de CUTLASS y <i>triad</i> (3 ciclos)	sombra	3	0	2
A: dgemm de CUTLASS y <i>triad</i> (3 ciclos)	activo	3	0	2
B: BabelStream y LavaMD (2 ciclos)	sombra	2	0	2

Tabla B.12: Fases de las aplicaciones compuestas. Duración media por fase medida en las celdas del brazo REF (6 a 10 fases por kernel); cada aplicación ejecuta dos ciclos.

Alcance	Aplicación	Kernel	Clase	Duración (s)
CPU	A	DGEMM $n = 2048$	compute	4.2
		NPB CG	memory	10.1
CPU	B	RSBench	compute	65.7
		XSbench	memory	12.0
GPU	A	DGEMM de CUTLASS ($n = 4096$)	compute	31.5
		<i>triad</i> de RAJAPerf	memory	18.0
GPU	B	BabelStream	memory	8.2
		LavaMD (entrada escalada)	compute	8.7
Conjunto	A	DGEMM, NPB CG, CUTLASS, <i>triad</i>	c, m, c, m	4.2, 10.1, 31.5, 18.0
Conjunto	B	RSBench, XSbench, BabelStream, LavaMD	c, m, m, c	65.6, 12.0, 8.3, 8.7

Tabla B.13: Clasificación de fase del agente en la matriz inicial (muestreo de 1 ms en CPU). Correctas y erróneas cuentan decisiones; la exactitud es sobre las decididas y la cobertura es el número de fases con al menos una decisión.

Alcance	Kernels	Agente	Brazo	Correctas	Erróneas	Abst.	Exactitud	Cobertura
CPU	A	CPU	Activo	65919	3906	0	0.944	12/12
CPU	A	CPU	Sombra	66422	4215	0	0.940	12/12
CPU	B	CPU	Activo	465161	734	0	0.998	12/12
CPU	B	CPU	Sombra	457006	1070	0	0.998	12/12
GPU	A	GPU	Activo	11	0	1	1.000	12/12
GPU	A	GPU	Sombra	11	0	1	1.000	12/12
GPU	B	GPU	Activo	6	0	6	1.000	12/12
GPU	B	GPU	Sombra	8	0	4	1.000	12/12
Conjunto	A	CPU	Activo	111875	6729	0	0.943	20/20
Conjunto	A	CPU	Sombra	110917	6372	0	0.946	20/20
Conjunto	A	GPU	Activo	14	0	6	1.000	20/20
Conjunto	A	GPU	Sombra	13	0	7	1.000	20/20
Conjunto	B	CPU	Activo	770777	1348	0	0.998	20/20
Conjunto	B	CPU	Sombra	761367	2537	0	0.997	20/20
Conjunto	B	GPU	Activo	10	0	10	1.000	20/20
Conjunto	B	GPU	Sombra	11	0	9	1.000	20/20

Tabla B.14: Resultados de la matriz inicial. Duración y energías son medianas; las razones son frente a REF. $E_{\text{tot}} = E_{\text{CPU}} + E_{\text{GPU}}$. El valor p es de Mann–Whitney bilateral y solo se calcula con al menos cinco repeticiones por brazo.

Alc.	Kern.	Brazo	n	T (s)	E_{CPU} (J)	E_{GPU} (J)	T/T_{REF}	$E_{\text{tot}}/\text{REF}$	EDP/REF	p
CPU	A	REF	3	33.1	5617	1152	1.000	1.000	1.000	–
CPU	A	Sombra	3	39.9	9302	1411	1.205	1.585	1.910	–
CPU	A	Activo	3	41.3	9164	1460	1.248	1.576	1.966	–
CPU	A	Activo F0	3	40.0	8939	1434	1.207	1.534	1.852	–
CPU	B	REF	3	159.9	26054	5582	1.000	1.000	1.000	–
CPU	B	Sombra	3	184.5	42217	6506	1.154	1.536	1.773	–
CPU	B	Activo	3	185.3	40508	6444	1.159	1.481	1.717	–
CPU	B	Activo F0	3	184.8	40442	6398	1.155	1.479	1.709	–
GPU	A	REF	3	103.4	15176	19489	1.000	1.000	1.000	–
GPU	A	Sombra	3	103.8	15206	19730	1.003	1.007	1.014	–
GPU	A	Activo	3	103.3	15249	19253	0.999	0.994	0.996	–
GPU	B	REF	3	38.2	5469	4593	1.000	1.000	1.000	–
GPU	B	Sombra	3	38.3	5557	4496	1.002	0.999	1.001	–
GPU	B	Activo	3	38.3	5487	4580	1.001	1.001	1.003	–
Conjunto	A	REF	5	136.0	20708	20690	1.000	1.000	1.000	–
Conjunto	A	Sombra	5	143.4	31740	20974	1.055	1.270	1.342	0.0079
Conjunto	A	Activo	5	145.0	30668	20882	1.066	1.245	1.326	0.0079
Conjunto	A	Activo sin piso	5	146.7	30803	20887	1.079	1.247	1.346	0.0079
Conjunto	B	REF	5	197.7	31447	10050	1.000	1.000	1.000	–
Conjunto	B	Sombra	5	223.0	50548	10934	1.128	1.483	1.679	0.0079
Conjunto	B	Activo	5	223.9	48460	10999	1.132	1.430	1.624	0.0079
Conjunto	B	Activo sin piso	5	224.6	48419	11065	1.136	1.430	1.631	0.0079

Tabla B.15: Efecto atribuible a la acción de frecuencia: EDP frente a REF del brazo sombra (sin escribir) y del activo, y su cociente. Un cociente menor que 1 indica que actuar mejora el EDP respecto de solo observar.

Alcance	Kernels	Sombra/REF	Activo/REF	Activo/sombra
CPU	A	1.910	1.966	1.029
CPU	B	1.773	1.717	0.968
Conjunto	A	1.342	1.326	0.988
Conjunto	B	1.679	1.624	0.967

Tabla B.16: Costo del brazo sombra frente a REF en CPU, aplicación A (tres repeticiones por configuración), según la configuración del motor de inferencia y del proceso consumidor.

Configuración	T/REF	$E_{\text{CPU}}/\text{REF}$	EDP/REF	Exactitud
Pool por defecto, 1 ms	1.205	1.656	1.910	0.940
Pool por defecto, 10 ms	1.198	1.666	1.903	0.964
Pool por defecto, 10 ms (diagnóstico)	1.204	1.669	1.911	0.969
Consumidor fijado a un núcleo	1.191	1.652	1.871	0.969
Un hilo de inferencia	0.998	1.063	1.047	0.991
Un hilo y consumidor fijado	0.999	1.059	1.045	0.994

Tabla B.17: Escenario C (tres repeticiones por brazo). Duración y energías son medianas y las razones son frente a REF.

Alc.	Kern.	Brazo	n	T (s)	E_{CPU} (J)	E_{GPU} (J)	T/REF	$E_{\text{CPU}}/\text{REF}$	$E_{\text{tot}}/\text{REF}$	EDP/REF
CPU	A	REF	3	33.0	5530	1142	1.000	1.000	1.000	1.000
CPU	A	Sombra	3	33.4	6009	1214	1.013	1.086	1.081	1.094
CPU	A	Activo	3	34.7	6118	1219	1.050	1.106	1.104	1.161
CPU	A	Activo F0	3	33.5	5996	1213	1.014	1.084	1.081	1.096
CPU	B	REF	3	159.9	25953	5539	1.000	1.000	1.000	1.000
CPU	B	Sombra	3	162.1	27281	5643	1.014	1.051	1.046	1.059
CPU	B	Activo	3	162.7	27008	5639	1.018	1.041	1.038	1.057
CPU	B	Activo F0	3	161.1	27163	5606	1.007	1.047	1.039	1.047
Conjunto	A	REF	3	136.2	20600	20536	1.000	1.000	1.000	1.000
Conjunto	A	Sombra	3	136.7	21716	20629	1.004	1.054	1.029	1.036
Conjunto	A	Activo	3	138.2	21998	20707	1.015	1.068	1.038	1.053
Conjunto	A	Activo sin piso	3	140.1	22075	20700	1.029	1.072	1.038	1.061
Conjunto	B	REF	3	198.4	31459	10097	1.000	1.000	1.000	1.000
Conjunto	B	Sombra	3	199.7	32904	10141	1.006	1.046	1.036	1.047
Conjunto	B	Activo	3	201.0	33126	10098	1.013	1.053	1.040	1.053
Conjunto	B	Activo sin piso	3	202.0	33361	10135	1.018	1.060	1.047	1.066

Tabla B.18: REF sin turbo relativo a REF, por aplicación compuesta y alcance incluido en C o E. Las razones son medianas de tres repeticiones.

Alcance	Kernels	n	T	E_{CPU}	E_{GPU}	EDP
CPU	A	3	1.001	1.002	1.003	1.006
CPU	B	3	0.995	0.997	0.998	0.991
Conjunto	A	3	0.999	0.999	0.995	0.996
Conjunto	B	3	1.005	1.004	1.003	1.006
GPU (memoria)	A	3	0.999	1.001	0.998	0.999
GPU (memoria)	B	3	0.993	0.994	1.000	0.988

Tabla B.19: Escenario E (tres repeticiones por brazo). Duración y energías son medianas; las razones son frente a REF y la última columna frente a REF sin turbo.

Kern.	Brazo	n	T (s)	E_{CPU} (J)	E_{GPU} (J)	T/REF	E_{GPU}/REF	EDP/REF	EDP GPU/REF	EDP/REF sin turbo
A	REF	3	186.7	28691	16404	1.000	1.000	1.000	1.000	1.001
A	REF sin turbo	3	186.6	28707	16366	0.999	0.998	0.999	0.996	1.000
A	Sombra	3	186.3	28650	16378	0.998	0.998	0.995	0.994	0.997
A	Activo	3	186.6	28720	15234	1.000	0.929	0.975	0.928	0.976
A	Activo + CPU obs.	3	188.6	29517	15320	1.010	0.934	1.003	0.942	1.005
B	REF	3	89.4	13147	8913	1.000	1.000	1.000	1.000	1.012
B	REF sin turbo	3	88.7	13070	8911	0.993	1.000	0.988	0.993	1.000
B	Sombra	3	89.6	13212	8900	1.002	0.999	1.004	1.001	1.016
B	Activo	3	90.1	13267	8943	1.008	1.003	1.014	1.012	1.026
B	Activo + CPU obs.	3	90.8	13774	8998	1.016	1.010	1.048	1.027	1.061

Tabla B.20: Clasificación de fase del agente en los escenarios C y E, contra las fronteras reales de cada fase.

Esc.	Alcance	Kernels	Agente	Brazo	Correctas	Erróneas	Abst.	Exactitud	Cobertura
C	CPU	A	CPU	Activo	63051	1452	0	0.977	12/12
C	CPU	A	CPU	Sombra	54688	1939	0	0.966	12/12
C	CPU	B	CPU	Activo	465002	744	0	0.998	12/12
C	CPU	B	CPU	Sombra	403524	1132	0	0.997	12/12
C	Conjunto	A	CPU	Activo	64479	1288	0	0.980	12/12
C	Conjunto	A	CPU	Sombra	57444	1965	0	0.967	12/12
C	Conjunto	A	GPU	Activo	11	0	1	1.000	12/12
C	Conjunto	A	GPU	Sombra	10	0	2	1.000	12/12
C	Conjunto	B	CPU	Activo	469229	741	0	0.998	12/12
C	Conjunto	B	CPU	Sombra	403151	971	0	0.998	12/12
C	Conjunto	B	GPU	Activo	8	0	4	1.000	12/12
C	Conjunto	B	GPU	Sombra	6	0	6	1.000	12/12
E	GPU (memoria)	A	GPU	Activo	12	0	0	1.000	12/12
E	GPU (memoria)	A	GPU	Sombra	12	0	0	1.000	12/12
E	GPU (memoria)	B	GPU	Activo	4	0	2	1.000	6/9
E	GPU (memoria)	B	GPU	Sombra	3	0	3	1.000	6/9

Tabla B.21: Resultados del escenario D con LAMMPS. Las razones se calculan frente a REF para cada entrada.

Entrada	Brazo	n	T (s)	E_{CPU} (J)	E_{GPU} (J)	EDP/REF	$E_{\text{GPU}}/\mathbf{REF}$
chain	REF	3	60.183	9056	2223	1.000	1.000
	Sombra	3	60.120	9092	2226	1.003	1.001
	Activo GPU	3	60.208	9075	2231	1.001	1.004
	Activo + CPU obs.	3	60.465	9322	2240	1.030	1.008
eam	REF	3	61.880	9395	2488	1.000	1.000
	Sombra	3	61.936	9419	2503	1.004	1.006
	Activo GPU	3	61.816	9391	2495	0.998	1.003
	Activo + CPU obs.	3	62.355	9779	2511	1.043	1.009
lj	REF	3	61.578	9371	2376	1.000	1.000
	Sombra	3	61.512	9360	2383	0.999	1.003
	Activo GPU	3	61.587	9367	2391	1.002	1.006
	Activo + CPU obs.	3	62.227	9632	2399	1.036	1.010
rhodo	REF	3	60.523	9109	2313	1.000	1.000
	Sombra	3	60.635	9129	2329	1.004	1.007
	Activo GPU	3	60.812	9139	2328	1.008	1.006
	Activo + CPU obs.	3	61.160	9352	2341	1.034	1.012

B.4. Tablas complementarias de CPU

Tabla B.22: Cobertura y composición del conjunto de intervalos CPU.

Medida	Valor
Configuraciones de kernel	49
Niveles de frecuencia	10
Corridas aceptadas	1470 de 1470
Intervalos procesados	1 424 531
Intervalos elegibles	1 309 755 (91.9 %)
Intervalos <i>compute_bound</i>	526 279 (40.2 %)
Intervalos <i>memory_bound</i>	783 476 (59.8 %)

Tabla B.23: Intervalos procesados, rechazados por causa y elegibles en la campaña CPU.

Categoría	Intervalos	Proporción
Procesados	1 424 531	100.0 %
Rechazados por frecuencia no utilizable	71 658	5.03 %
Rechazados por ventana de origen no válida	43 118	3.03 %
Elegibles	1 309 755	91.9 %

Tabla B.24: Flujo de observaciones desde la campaña CPU hasta la evaluación del modelo.

Etapas	Intervalos	Cálculo o criterio
Intervalos elegibles	1 309 755	Estado de entrenamiento correcto y frecuencia utilizable
Intervalos evaluados	1 309 755	Todos los elegibles: las seis variables de entrada están definidas en cada intervalo
Muestra reproducible	42 458	Suma de $\min(n_{fc}, 1000)$ para cada celda observada
Clase <i>compute-bound</i> en la muestra	21 350	Conteo posterior al límite por familia y clase
Clase <i>memory-bound</i> en la muestra	21 108	Conteo posterior al límite por familia y clase

Tabla B.25: Sensibilidad del resultado al tope de intervalos por celda de familia y clase.

Tope por celda	Intervalos de ajuste	Exactitud balanceada
100	4 728	0.732
300	13 430	0.729
1000	42 458	0.723
3000	122 135	0.739
10 000	355 228	0.737
30 000	730 974	0.720

Tabla B.26: Configuración final del clasificador CPU.

Elemento	Valor
Modelo	XGBoost, 100 árboles, profundidad máxima 6, objetivo <code>binary:logistic</code>
Entradas	6 variables PMU de la Tabla 3.1
Ponderación	Peso total igual por celda de familia y clase, ecuación (A.3), con <code>scale_pos_weight = 1</code>
Salida	Probabilidad de <i>memory-bound</i> ; clase con confianza $\geq 0,85$ y <i>revisar</i> por debajo
Semilla	20260918

Tabla B.27: Métricas por clase del modelo final sobre los 1 309 755 intervalos evaluados.

Clase	Intervalos	Precisión	Sensibilidad	F1
<i>compute-bound</i>	526 279	0.712	0.714	0.713
<i>memory-bound</i>	783 476	0.808	0.806	0.807

B.5. Verificaciones complementarias del agente

Tabla B.28: Restauración bajo terminación anómala y coordinación entre procesos.

Prueba	Resultado
Proceso de GPU, candado puesto	El reloj vuelve al valor nativo; código de salida 0
Proceso de CPU, frecuencia modificada	Turbo y rango de frecuencia idénticos al estado inicial; código de salida 0
Ambos procesos a la vez	Ambos salen con código 0; estado de CPU idéntico al inicial y reloj de GPU liberado
Piso de CPU con GPU activa (3.2 GHz)	Respetado en el 99.2 % de las muestras; las restantes (cuatro, de ≈ 0.24 s) están en los flancos de subida

B.6. Análisis y decisiones complementarias

B.6.1. Sensibilidad del tamaño de la muestra CPU

El conjunto contiene 30 familias algorítmicas. El modelo se ajusta con una muestra estratificada de 42 458 intervalos, con un tope de 1000 por celda de familia y clase, y se evalúa sobre *todos* los intervalos elegibles de la familia retirada (Tabla B.24 y Figura 3.5). El tope se verificó y no descarta información útil: al repetir la evaluación con topes entre 100 y 30 000, es decir, entre 4728 y 730 974 intervalos de ajuste, el resultado se mantiene entre 0.720 y 0.739 sin tendencia creciente (Tabla B.25). El valor de 0.723 que muestra la tabla para el tope de 1000, ligeramente por debajo del 0.728 del modelo congelado con ese mismo tope (Tabla B.4), proviene de esta corrida de sensibilidad y no de la evaluación final; la diferencia, de 0.005, es menor que la desviación entre semillas de muestreo (0.015–0.016) y no altera la conclusión de que el tope no mueve el resultado. El sorteo del tope no reparte las filas por nivel de frecuencia, de modo que en las celdas donde actúa el nivel más poblado llega a tener en mediana 4.1 veces las filas del menos poblado; con un tope repartido por igual entre los 10 niveles la exactitud balanceada por celda cambió en $-0,003$ (IC95 de $-0,026$ a $+0,017$), una diferencia compatible con cero, por lo que se conservó el sorteo simple. Un millón de intervalos no equivale a un millón de ejemplos independientes, porque son observaciones repetidas de 30 algoritmos.

B.6.2. Variables adicionales del modelo CPU

El vector final deriva de este conjunto ampliado a doce entradas, que añadía cocientes y transformaciones de los mismos contadores, entre ellas $\log(1 + \text{MPKI})$ (una transformación monótona de MPKI) y las referencias de caché por millar de instrucciones ya mencionadas. Esas seis variables adicionales no mejoran el resultado: con las seis base la exactitud balanceada por celda es 0.728 y con las doce es 0.724, una diferencia de $-0,004$ cuyo intervalo de confianza del 95 % (de $-0,026$ a $+0,011$) contiene el cero (Tabla B.1). La F1 macro agrupada favorece nominalmente a las doce (0.783 frente a 0.760), pero es una métrica de referencia y no la que gobierna esta comparación. La redundancia sí afecta a la regresión logística, que es un modelo lineal: con doce variables queda 0.067 por debajo de XGBoost (IC95 de $-0,105$ a $-0,030$, que no toca el cero) y con seis la brecha es de 0.048, con un IC95 que apenas toca el cero (de $-0,092$ a $0,000$); la redundancia entre variables agrava, en lugar de crear, una diferencia ya presente con el vector reducido. El costo de inferencia de XGBoost es el mismo con seis o con doce variables (mediana de 243 y 245 μs , y percentil 99 de 0.40 y 0.41 ms, con seis y con doce variables); se conservaron las seis porque el agente calcula menos columnas y

no hay evidencia de que las adicionales aporten.

B.6.3. Desglose de la política CPU

El resultado es *no actuar* en ambas clases (19 kernels compute y 28 memory, estos últimos con `ptrchase`, medido solo para esta política por ser un recorrido de memoria por latencia fuera del dominio del clasificador). En compute, el nivel más alto (F0, 3.2 GHz, el mismo reloj que la referencia con el turbo desactivado) mejora el producto 0.56 % (IC95 de 0.27 a 1.38 %, $p = 0,008$): la diferencia es significativa pero menor que el efecto mínimo relevante y proviene de fijar el reloj frente al gobernador, no de reducirlo; de F1 a F8 la diferencia es significativa ($p < 0,001$) y desfavorable, y el nivel más bajo multiplica el producto por 9.0 en mediana. En memory, ningún nivel bajo mejora el producto agregado: de F3 a F8 la diferencia es significativa, también en sentido desfavorable. `ptrchase`, con $\alpha = 0,22$, apenas gana 1.0 % en F1. Cinco kernels del tipo STREAM, con exponente de tiempo α entre 0.18 y 0.22, alcanzan ganancias de entre 4 y 11 % en alguno de los niveles F1 a F3, pero esas cifras corresponden al mejor nivel de cada kernel y no describen una tendencia: en ninguno de ellos la ganancia varía de forma monótona con la frecuencia, como exigiría una curva de producto energía–retardo suave. El mismo kernel que gana 8.0 % en F2 gana 0.3 % en F3, y varios de ellos pierden en uno de los tres niveles mientras ganan en otro. Su magnitud es del orden de la dispersión entre repeticiones y niveles, de modo que se reportan como observación puntual y no como evidencia de una región aprovechable. Aquí α es la pendiente negativa de $\log T$ frente a $\log f$. La Figura 3.17 muestra por qué: al bajar el reloj la energía casi no disminuye (la mediana más baja es 0.983 de la referencia en memory, y en compute nunca baja de 0.998), mientras el tiempo crece, de modo que el producto energía–retardo solo empeora.

B.6.4. Modelo del piso de potencia CPU

La causa es el piso de potencia. Ajustando $P(f) = P_0 + c f^k$ a cada kernel (error mediano de 0.34 W), P_0 resulta de 85 W en mediana, casi idéntico entre kernels (entre 80 y 100 W), y representa el 84 % de la potencia a 3.2 GHz; con $k \approx 2$, reducir el reloj a 0.8 GHz ahorra solo 15 % de la potencia. Con ese modelo, bajar la frecuencia mejora el producto únicamente si α es menor que un valor crítico, cuya mediana es 0.14 y que en ningún kernel supera 0.33. Los kernels compute densos tienen el mayor margen de potencia, pero un α cercano a 1. Incluso con un clasificador perfecto que eligiera el mejor nivel de cada kernel, la ganancia media sería de 2.2 % y la máxima de 12.5 %; ese techo es además optimista, porque se obtiene tomando el mínimo del producto entre los nueve niveles medidos de cada kernel, y ese mínimo captura parte de la dispersión de la medición además del efecto real. De hecho, 22 de los 47 kernels

tienen una ganancia de oráculo inferior al efecto mínimo relevante de 1 %, es decir que para casi la mitad del catálogo ni siquiera un selector perfecto alcanzaría el umbral declarado. Este desenlace es un resultado válido de la evaluación: la decisión de actuar recae en identificar la región de α bajo, y la clase Roofline por sí sola no la determina. La Figura 3.18 muestra la potencia medida de cada kernel en función de la frecuencia: las curvas casi se aplanan sobre ese piso, y la fracción estática a 3.2 GHz es de entre 0.6 y 0.95 según el kernel.

B.6.5. Contrato operativo del modelo GPU

En consecuencia, el modelo desplegado en el agente es un *random forest* con tres variables, la mediana de la utilización de GPU, la mediana de la utilización de memoria y la desviación estándar intracorrida de esta última, sin reloj ni potencia (Tabla 3.4). Se ajustó sobre las 483 corridas disponibles después de cerrar la evaluación LOFO. Su inferencia de una fila en **paccaA100** es de 4.7 ms en el percentil 50 y 5.0 ms en el 99. Es pequeña frente a los segundos necesarios para detectar una fase, reunir sus señales y aplicar un reloj; esa latencia de respuesta completa se mide en la Fase 4. El agente toma como máximo una decisión por fase detectada, de modo que una abstención no se reevalúa dentro de la misma fase. Su ECE LOFO, calculado sobre las probabilidades fuera de familia de cinco semillas y diez rangos de confianza, es 0.0868. Es mayor que el 0.0569 de la regresión logística, pero corresponde al contrato de tres entradas que el daemon puede observar sin realimentarse. Con el umbral congelado en 0.90, la exactitud balanceada por celda sube de 0.819 a 0.900 reteniendo el 65.7 % de las corridas. Este es el modelo y el umbral que consume el agente.

B.6.6. Sensibilidad del tiempo al reloj GPU

Por qué *memory_bound* sí responde y *compute_bound* no. El mecanismo es el mismo que en CPU (sección 3.1.5): la sensibilidad del tiempo de ejecución al reloj. Ajustando α , la pendiente negativa de $\log(\text{tiempo})$ frente a $\log(\text{reloj SM})$, sobre 18 de los 19 kernels con al menos tres niveles de reloj medidos (excluido **minibude_cuda_bm1**, que solo tiene **REF**) — este cálculo no exige corridas en **REF**, así que incluye también los cuatro kernels sin ella que la Tabla 3.5 no puede emparejar —, los kernels *compute_bound* tienen α mediano de 0.837 (rango 0.344–1.216, $n = 6$): su tiempo es casi inversamente proporcional al reloj, de modo que bajar la frecuencia ahorra poca energía por unidad de tiempo perdido. Los kernels *memory_bound* tienen α mediano de 0.189 (rango 0.093–0.941, $n = 12$): su tiempo apenas responde al reloj del multiprocesador de flujo, porque su cuello de botella — el ancho de banda hacia la memoria del dispositivo — está gobernado por un dominio de reloj separado que la matriz experimental no varía (Tabla A.3). Reducir el reloj SM en ese régimen retira

potencia dinámica del cómputo sin alargar proporcionalmente la espera por memoria, lo que abre el margen de mejora del producto energía-retardo que la Tabla 3.5 mide. La excepción es *rodinia_gaussian* ($\alpha = 0,941$ pese a ser *memory_bound*), la familia más cercana al punto de inflexión del catálogo (sección 3.2.3), consistente con que su tiempo responda casi tanto al reloj como el de un kernel genuinamente limitado por cómputo.

B.6.7. Excepción en la política GPU

La única familia que empeora en la validación LOFO. De las ocho familias *memory_bound*, siete escogen F1 al aplicar la regla “mayor ganancia media en las demás familias” a su propio pliegue, y la realizan con ganancias de entre 4.2 % y 14.8 %. La excepción es *rodinia_gaussian*, cuyo pliegue escoge F2 a partir de las otras siete familias y realiza una pérdida de -27,3 %: es precisamente la familia con $\alpha = 0,941$ señalada en el párrafo anterior, la más cercana al punto de inflexión del catálogo (sección 3.2.3), cuyo tiempo responde al reloj casi como el de un kernel *compute_bound* pese a estar etiquetada *memory_bound* sin ambigüedad (margen de clase 1.0). No es el mismo mecanismo que limita al clasificador: *rodinia_lud*, la familia con menor margen de mayoría de clase de todo el conjunto (0.60, la que sí cruza el punto de inflexión entre niveles) realiza en este pliegue una ganancia positiva de 13.9 % y no es problemática para la política, aunque sí lo sea para el clasificador de la sección 3.2.3. La política de frecuencia y el clasificador de régimen comparten un límite de fondo — ninguno de los dos recibe la distancia al punto de inflexión como variable — pero les afecta por vías distintas: al clasificador lo confunde un kernel cuya etiqueta cambia con el nivel; a la política la sorprende un kernel cuya etiqueta es estable pero cuya física de reloj se parece a la de la otra clase. Ninguna de las dos familias se excluyó de su respectivo análisis por la misma razón: excluirlas seleccionaría datos por el resultado que ellas mismas producen.

B.6.8. Referencia sin turbo

Como línea base secundaria se midió el nodo sin agente y con el turbo de CPU apagado (*REF sin turbo*), para determinar si la referencia nativa obtiene una ventaja atribuible a ese mecanismo. REF con turbo sigue siendo la referencia principal. La Tabla B.18 cubre las seis combinaciones compuestas medidas en C y E, con tres repeticiones por combinación. Sus razones medianas de EDP frente a REF están entre 0.988 y 1.006, de modo que la mayor diferencia observada es -1,2 % y no muestra una ventaja sistemática de ninguna referencia. En los núcleos que ejecutan las aplicaciones de CPU y conjunto, el límite superior configurado es 3.2 GHz; por ello, el turbo no eleva el reloj de la aplicación y tampoco aparece una diferencia apreciable de tiempo o energía. Esta evidencia se limita a las cargas compuestas evaluadas y

no se extrapola a LAMMPS ni a CloverLeaf. Las conclusiones de las secciones anteriores no cambian con la referencia sin turbo: el brazo activo de CPU queda entre 1.05 y 1.14 de REF sin turbo, frente a 1.04 y 1.15 de REF. Esta línea base conserva el reloj nativo de la GPU y, por tanto, no prueba la ventaja de conmutar por fase frente a un nivel fijo de GPU.

B.6.9. Análisis por repetición de E-B

Familias inéditas (B). La mediana no muestra mejora: el brazo activo queda en 1.014 del EDP de REF y 1.003 en energía de GPU, y el agente con observación de CPU en 1.048. Los resultados por repetición explican por qué. El clasificador de GPU decide con la ventana inicial de cada fase, y en BabelStream su probabilidad de memoria fue 1.00 en una repetición y 0.43 y 0.33 en las otras dos, de modo que solo en la primera superó el umbral de 0.90 y aplicó F1; en las otras dos se abstuvo y el reloj quedó en el valor nativo. En la repetición en que el agente actuó, la energía de GPU fue 12 % menor que la de REF con la misma duración y el EDP del nodo 5 % menor; en las otras dos el resultado fue neutro. Con *myocyte* el agente no tomó ninguna decisión en ninguna repetición, porque la utilización de GPU (máxima de 11 %, mediana de 0 %) no supera el umbral de 5 % de forma sostenida con que el agente abre una fase. Además, en la segunda repetición de los dos brazos activos la fase de *myocyte* duró 49.9 s frente a 40 a 42 s en las demás, con el reloj nativo y sin que el agente hubiera cambiado nada; esa repetición explica el valor de 1.18 de REF en el EDP y su causa no está identificada, aunque no hay evidencia de que la produzca el agente. La conclusión es que con memoria inédita el resultado depende de que el clasificador decida: cuando decide, el ahorro es del mismo orden que con familias vistas, y cuando se abstiene no hay efecto, sin que en ninguna repetición se atribuya un empeoramiento al agente. Es el límite de generalización del clasificador de GPU visible en la Fase 2.

B.7. Tablas completas de síntesis y política

Tabla B.29: Ganancia agregada del producto energía-retardo frente a la referencia nativa (%), intervalo de confianza de 95 % por *bootstrap* de kernels y valor p de Wilcoxon pareado. Un valor negativo indica empeoramiento. Compute: 19 kernels; memory: 28 kernels. Fuente: elaboración propia.

Nivel	Compute			Memory		
	Ganancia	IC 95 %	p	Ganancia	IC 95 %	p
F0	0,56	[0,27; 1,38]	0.008	0,21	[−0,41; 0,39]	0.938
F1	−11,6	[−14,2; −11,0]	< 0,001	−5,4	[−6,8; −3,4]	0.120
F2	−30,7	[−34,7; −29,2]	< 0,001	−13,4	[−18,5; −11,2]	0.073
F3	−58,0	[−66,6; −54,6]	< 0,001	−26,2	[−34,0; −20,5]	0.018
F4	−97,5	[−111,2; −91,8]	< 0,001	−44,0	[−54,8; −20,9]	0.009
F5	−159,9	[−181,9; −150,7]	< 0,001	−72,8	[−91,9; −43,7]	< 0,001
F6	−268,3	[−305,8; −252,6]	< 0,001	−124,6	[−153,0; −82,3]	< 0,001
F7	−476,0	[−541,0; −448,6]	< 0,001	−219,2	[−273,5; −155,7]	< 0,001
F8	−966,6	[−1098,7; −911,3]	< 0,001	−445,9	[−561,9; −337,3]	< 0,001

Tabla B.30: Ganancia agregada del producto energía-retardo frente a REF (%), intervalo de confianza de 95 % por *bootstrap* de familias y valor p de la prueba pareada. Compute: 5 familias; memory: 8 familias. Fuente: elaboración propia.

Nivel	Compute			Memory		
	Ganancia	IC 95 %	p	Ganancia	IC 95 %	p
F0	−0,7	[−4,3; 2,0]	0.438	−0,6	[−2,3; 1,1]	0.770
F1	2,1	[−12,7; 14,2]	0.438	8,9	[3,0; 13,3]	0,020
F2	−10,2	[−41,2; 15,3]	0.688	7,4	[−3,6; 15,9]	0.156
F3	−30,8	[−87,3; 9,4]	0.844	−0,3	[−20,0; 14,1]	0.422
F4	−68,6	[−165,7; −3,7]	0.938	−14,0	[−45,7; 8,3]	0.680
F5	−122,0	[−276,6; −23,4]	0.969	−30,3	[−78,1; 1,3]	0.902
F6	−183,9	[−438,9; −47,3]	0.969	−38,9	[−104,2; −0,9]	0.926
F7	−300,4	[−761,1; −86,1]	1.000	−48,2	[−147,4; 0,9]	0.902
F8	−645,1	[−1727,4; −200,0]	1.000	−76,3	[−268,2; −2,3]	0.902

Tabla B.31: Balance de la Fase 4 por pregunta.

Pregunta	Resultado	Sección
¿Clasifica bien la fase en aplicaciones con verdad conocida?	Sí: CPU 97 a 99.9 %; GPU sin errores en lo que decide, con abstenciones	3.4.2, 3.4.8
¿Es barato mantener el agente?	GPU sin costo medible; CPU con un hilo de inferencia, +5 a 9 % de energía de CPU y +0.4 a 1.4 % de tiempo	3.4.4, 3.4.6
¿Actuar sobre la frecuencia de CPU mejora el EDP?	No: el brazo activo queda entre 1.04 y 1.16 de REF, y no mejora sobre sombra	3.4.6
¿Conmutar GPU por fase gana a F1 fijo?	No quedó demostrado: el brazo F1 fijo de E-A fue inválido porque no sostuvo el reloj bajo carga	3.4.8
¿Actuar sobre la frecuencia de GPU mejora el EDP?	E-A y CloverLeaf reducen la energía de GPU en 7.18 y 6.07 %, y el EDP del nodo en 2.84 y 3.75 %; en ambos casos $p = 0,0625$ bilateral. La comparación E-A con F1 fijo quedó inválida y LAMMPS permanece en paridad	3.4.8, 3.4.10, 3.4.9

B.8. Detalles adicionales de método y diagnóstico

B.8.1. Auditoría de frecuencia de la campaña CPU

Toda corrida se ejecutó con turbo desactivado y con la frecuencia solicitada verificada en cada ventana, con una tolerancia de $\pm 5\%$; una ventana cuya frecuencia observada no cumple ese criterio no recibe etiqueta. La comprobación excluye dos márgenes. Tras cada cambio de nivel se descartan ~ 10 ms, porque en mediciones repetidas (dos corridas idénticas en el nodo de la campaña) un procesador delegado tardó entre 10 y 11 ms en engancharse al límite de frecuencia justo después de arrancar el kernel real, aunque el asentamiento previo ya se había confirmado. Además se descartan 0.15 s al inicio y al final de cada corrida, porque en los kernels de una sola región paralela larga un hilo que termina antes queda ocioso esperando a sus pares y diluye el promedio de frecuencia en la unión final. La actuación de frecuencia se comprobó midiendo su efecto sobre el rendimiento: al restringir solo los procesadores lógicos

delegados, la razón de rendimiento entre el nivel más alto y el más bajo se mantuvo en 0.995, es decir, sin escalar; al restringir también sus hermanos de *SMT*, subió a 3.78, cercana a la razón de cuatro esperada entre esos niveles. Las 1470 corridas conservaron una observación de frecuencia válida o, en el nivel nativo, el estado no aplicable a una frecuencia fija. De los 1 424 531 intervalos procesados se rechazó el 8.1 %, por dos causas (Tabla B.23): que la frecuencia observada no fuera utilizable, y que alguna ventana de origen no superara la validación de calidad (sección A.1.3).

B.8.2. Selección detallada del umbral CPU

El modelo puede abstenerse cuando su confianza, definida como $\max(p, 1-p)$, no alcanza un umbral. El umbral se eligió con un LOFO interno sobre las 30 familias y la misma métrica principal, la exactitud balanceada por celda sobre lo decidido, con una cobertura media por celda mínima de 0.60. Esta selección es un promedio a través de los pliegues de la rejilla (Tabla B.6) y no coincide exactamente con la evaluación externa del modelo final que se reporta más abajo, porque una promedia el criterio interno de cada pliegue y la otra evalúa el modelo ya congelado. La regla propuso 0.90. Se congeló 0.85 en su lugar, por tres razones. Primera, la curva es casi plana: pasar de 0.85 a 0.90 sube esa exactitud de selección de 0.749 a 0.752, una diferencia dentro del ruido (con 30 familias, las diferencias menores que 0.03 no son distinguibles), y baja la cobertura por celda de 0.714 a 0.649, es decir, 6.5 puntos más de intervalos que el agente no resuelve por sí mismo. Segunda, los intervalos con confianza entre 0.85 y 0.90 son el 6.3 % del total y se aciertan el 64 %, de modo que excluirllos descarta casos que en su mayoría el modelo acierta a medias. Tercera, ningún umbral mayor rescata la métrica de forma sustancial: a 0.93 la cobertura llega al piso de 0.60 y esa exactitud de selección es 0.753. Es una decisión de diseño posterior al resultado y se declara como tal; elegir el umbral por pliegue, dentro de cada familia retirada y con la rejilla ampliada hasta 0.98, produce prácticamente lo mismo (0.734 sobre lo decidido, con una cobertura media por familia de 0.69), de modo que congelar un único valor no infla el resultado.

B.8.3. Construcción detallada de la política CPU

La tabla clase–frecuencia de menor producto energía–retardo se derivó de la campaña con la energía y el tiempo de cada corrida completa. Toda corrida de un mismo kernel ejecuta el mismo trabajo en los diez niveles, de modo que el producto energía–retardo (energía de paquete más DRAM, multiplicada por el tiempo) es comparable entre niveles. Para cada clase se tomó la mediana de las tres repeticiones de cada kernel y se comparó cada nivel fijo con la referencia nativa mediante una prueba de Wilcoxon pareada por kernel ($\alpha = 0,05$).

Un nivel solo se adopta si mejora el producto agregado de forma significativa y en al menos 1 %, efecto mínimo relevante que se fijó después de ver el resultado y se declara como tal; en caso contrario la entrada de la clase es *no actuar*, con su motivo. La clase de cada kernel es la mayoritaria entre sus intervalos etiquetados y se excluyen las sondas de calibración. Se excluyen las tres variantes de *phasic*, cuya duración es fija (20.6 s en todos los niveles): su trabajo no es constante entre niveles y su producto no compara lo mismo. La Tabla 3.3 resume la decisión; la ganancia y su incertidumbre para cada nivel están en la Tabla B.29 del anexo.

B.8.4. Diagnóstico del eje de frecuencia CUPTI

El eje de frecuencia de esa misma campaña, sin embargo, resultó inválido. Una verificación posterior del candado de reloj (`nvidia-smi -lgc`), con carga computacional real y medición del rendimiento alcanzado, encontró que el reloj observado no cambiaba al solicitar 700 MHz ni al solicitar 1200 MHz: permaneció fijo en 765 MHz antes, durante y después de cada solicitud, y el rendimiento medido bajo carga coincidió en ambos casos con el pico documentado sin candado. Esto contrasta con la verificación reportada en la sección 3.2.1, donde el mismo mecanismo sí sostuvo objetivos de 600 y 1200 MHz. Como la verdad por ventana de la campaña CUPTI exige que el reloj observado coincida con el nivel solicitado dentro de una tolerancia, cada ventana quedó marcada con frecuencia inválida y ninguna resultó elegible para entrenamiento. Los artefactos de mayor costo, la traza de eventos y las calibraciones Roofline por nivel, permanecen íntegros; lo que falla es la interpretación del eje de frecuencia sobre esa traza, en principio recuperable reprocesando cada corrida como un único nivel de reloj nativo en vez de nueve niveles distintos, tarea que queda pendiente. Una verificación posterior, con la misma prueba directa bajo carga, encontró el candado operativo (sección 3.3.1), por lo que el eje de frecuencia de una campaña nueva vuelve a ser interpretable.

B.8.5. Composición detallada del escenario E

La aplicación A reúne tres fases *memory_bound* (`dual_stencil`, 58 a 60 s; `dual_spmv`, 46 s; `dual_axpy`, 47 a 48 s) y una fase de cómputo (DGEMM de CUTLASS, 31 s), con el 84 % del ciclo en memoria. La aplicación B reúne dos fases *memory_bound* inéditas (BabelStream escalado, 36 s, y *myocyte* de Rodinia escalado, 40 a 41 s) y una de cómputo (LavaMD escalado, 9 s), con el 89 % del ciclo en fases etiquetadas *memory_bound*; solo hay dos porque ningún otro kernel inédito de memoria del catálogo sostiene 25 s. La utilización de GPU medida sobre la secuencia muestra que solo BabelStream ocupa la GPU (utilización mediana 100 %); *myocyte* tiene utilización mediana de 0 % y máxima de 11 %, de modo que su eti-

queta *memory_bound*, derivada de una intensidad operacional por kernel, no corresponde a actividad de GPU, y la fracción de tiempo del ciclo con la GPU en una fase de memoria es 42 %. Cada aplicación se ejecuta en un ciclo. Los brazos son REF, REF sin turbo, sombra y dos variantes del brazo activo con la política de GPU congelada (F1 en *memory*, reloj nativo en *compute*): sin agente de CPU, porque la política de CPU es no actuar, y con el agente de CPU en observación, para medir su costo.

Bibliografía

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [2] G. Ali, M. Side, S. Bhalachandra, N. J. Wright, and Y. Chen. An automated and portable method for selecting an optimal gpu frequency. *Future Generation Computer Systems*, 149:71–88, 2023. doi: 10.1016/j.future.2023.07.011.
- [3] Oscar Antepara, Leonid Oliker, and Samuel Williams. Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200. In *Proceedings of the SC ’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1205–1216, 2025. doi: 10.1145/3731599.3767497.
- [4] F. Antici, A. Bartolini, Z. Kiziltan, O. Babaoglu, and Y. Kodama. Mcbound: An online framework to characterize and classify memory/compute-bound hpc jobs. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Atlanta, GA, USA, 2024. doi: 10.1109/SC41406.2024.00062.
- [5] James S. Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24, pages 2546–2554, 2011.
- [6] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.
- [7] E. Calore, A. Gabbana, S. F. Schifano, and R. Tripiccione. Evaluation of dvfs techniques on modern hpc processors and accelerators for energy-aware applications. *Concurrency and Computation: Practice and Experience*, 29(12):e4143, 2017. doi: 10.1002/cpe.4143.

- [8] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [9] M. T. Costa et al. Characterizing the impact of gpu power management on an exascale system. In *Proc. SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*, pages 1524–1533, New York, NY, USA, 2025. doi: 10.1145/3731599.3767702.
- [10] H. David, E. Gorbato, U. R. Hanebutte, R. Khanna, and C. Le. RAPL: Memory power estimation and capping. In *Proc. 16th ACM/IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, pages 189–194, 2010. doi: 10.1145/1840845.1840883.
- [11] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63(1):3–42, 2006. doi: 10.1007/s10994-006-6226-1.
- [12] R. Gonzalez, B. M. Gordon, and M. A. Horowitz. Supply and threshold voltage scaling for low power CMOS. *IEEE Journal of Solid-State Circuits*, 32(8):1210–1216, 1997. doi: 10.1109/4.604018.
- [13] Kazushige Goto and Robert A. van de Geijn. Anatomy of high-performance matrix multiplication. *ACM Transactions on Mathematical Software*, 34(3):1–25, 2008. doi: 10.1145/1356052.1356053. Article 12.
- [14] B. Gregg. *Systems Performance: Enterprise and the Cloud*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2020.
- [15] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás. DVFS-aware application classification to improve GPGPUs energy efficiency. *Parallel Computing*, 83:93–117, 2019. doi: 10.1016/j.parco.2018.02.001.
- [16] R. Hebbar and A. Milenković. PMU-events-driven DVFS techniques for improving energy efficiency of modern processors. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, 7(1):1–31, 2022. doi: 10.1145/3538645.
- [17] Intel Corporation. Roofline analysis with intel advisor. Intel Advisor User Guide, 2026. URL <https://www.intel.com/content/www/us/en/docs/advisor/user-guide/2026-0/roofline-perspective-in-intel-advisor.html>. Accessed: Aug. 12, 2026.

- [18] Intel Corporation. Intel 64 and ia-32 architectures software developer’s manual, volume 3b: System programming guide, part 2 (performance monitoring, nmi watchdog interaction with fixed-function and general-purpose counters), 2026. URL <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>. Accessed: Aug. 12, 2026.
- [19] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer Texts in Statistics. Springer, 2nd edition, 2021. doi: 10.1007/978-1-0716-1418-1.
- [20] M. Kerrisk. *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*. No Starch Press, San Francisco, CA, USA, 2010.
- [21] J. H. Laros III, K. Pedretti, S. M. Kelly, W. Shu, K. Ferreira, J. Van Dyke, and C. Vaughan. Energy delay product. In *Energy-Efficient High Performance Computing: Measurement and Tuning*, SpringerBriefs in Computer Science, chapter 6, pages 51–55. Springer, London, UK, 2013. doi: 10.1007/978-1-4471-4492-2_9.
- [22] Linux Kernel Documentation. CPU Performance Scaling, 2026. URL <https://docs.kernel.org/admin-guide/pm/cpufreq.html>. Accessed: Sep. 5, 2026.
- [23] Linux Kernel Documentation. intel_pstate cpu performance scaling driver, 2026. URL https://docs.kernel.org/admin-guide/pm/intel_pstate.html. Accessed: Sep. 5, 2026.
- [24] Linux man-pages project. perf_event_open(2) — linux manual page, 2026. URL https://man7.org/linux/man-pages/man2/perf_event_open.2.html. Accessed: Aug. 29, 2026.
- [25] Linux perf project. perf-stat(1) measurement overhead documentation, 2026. URL <https://man7.org/linux/man-pages/man1/perf-stat.1.html>. Accessed: Sep. 20, 2026. Man pages explicitly warn: “The overhead percentage could be high in some cases, for instance with small, sub 100ms intervals. Use with caution.” Intervals below 10ms incur syscall overhead that dominates measurement time in typical deployments.
- [26] T.-Y. Liu, J. Guo, and B. Huang. Efficient cross-platform multiplexing of hardware performance counters via adaptive grouping. *ACM Transactions on Architecture and Code Optimization*, 21(1):1–26, 2024. doi: 10.1145/3629525.
- [27] S. McCarty. Architecting containers part 1: Why understanding user space vs. kernel space matters. Red Hat Blog, 2015. URL <https://www.redhat.com/en/blog/>

[architecting-containers-part-1-why-understanding-user-space-vs-kernel-space-matters](#).

Accessed: Apr. 3, 2025.

- [28] National Energy Research Scientific Computing Center (NERSC). Roofline performance model. NERSC Documentation, 2026. URL <https://docs.nersc.gov/tools/performance/roofline/>. Accessed: Aug. 7, 2026.
- [29] NVIDIA. CUDA profiling tools interface (CUPTI). NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/cupti/>. Accessed: Sep. 18, 2026.
- [30] NVIDIA. Nsight compute. NVIDIA Developer Documentation, 2026. URL <https://docs.nvidia.com/nsight-compute/>. Accessed: Aug. 8, 2026.
- [31] O. S. Simsek, J.-G. Piccinali, and F. M. Ciorba. Increasing energy efficiency of astrophysics simulations through gpu frequency scaling. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1826–1834, Atlanta, GA, USA, 2024. doi: 10.1109/SCW63240.2024.00229.
- [32] P. Thamm and U. Leser. Strategies to measure energy consumption using RAPL during workflow execution on commodity clusters. *arXiv preprint arXiv:2505.09375*, 2025.
- [33] D. Velicka, O. Vysocky, and L. Riha. Methodology for GPU frequency switching latency measurement. In *Proc. 2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 830–839, 2025. doi: 10.1109/IPDPSW66978.2025.00133.
- [34] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang. DRLCap: Runtime GPU frequency capping with deep reinforcement learning. *IEEE Transactions on Sustainable Computing*, 9(5):712–726, 2024. doi: 10.1109/TSUSC.2024.3361596.
- [35] V. M. Weaver. Self-monitoring overhead of the linux perf event performance counter interface. In *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 102–111, 2015.
- [36] V. M. Weaver and S. A. McKee. Can hardware performance counters be trusted? In *Proc. IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–150, Seattle, WA, USA, 2008. doi: 10.1109/IISWC.2008.4636099.
- [37] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Repositorio y recursos complementarios

Repositorio público del proyecto:

<https://github.com/AdrianCCRS/hyperion>